

HOSTED BY



ELSEVIER

Contents lists available at ScienceDirect

# Journal of King Saud University - Computer and Information Sciences

journal homepage: [www.sciencedirect.com](http://www.sciencedirect.com)

## Review article

# RNN-LSTM: From applications to modeling techniques and beyond—Systematic review<sup>☆</sup>

Safwan Mahmood Al-Selwi<sup>a,b,\*</sup>, Mohd Fadzil Hassan<sup>a,b</sup>, Said Jadid Abdulkadir<sup>a,b</sup>,  
Amgad Muneer<sup>a</sup>, Ebrahim Hamid Sumiea<sup>a,b</sup>, Alawi Alqushaibi<sup>a,b</sup>, Mohammed Gamal Ragab<sup>a,b</sup>

<sup>a</sup> Department of Computer and Information Sciences, Universiti Teknologi PETRONAS, Seri Iskandar, 32610, Perak, Malaysia

<sup>b</sup> Center for Research in Data Science (CeRDs), Universiti Teknologi PETRONAS, Seri Iskandar, 32610, Perak, Malaysia

## ARTICLE INFO

Dataset link: <https://github.com/SafwanAlselwi/RNN-LSTM-SLR>

### Keywords:

Machine learning  
Deep learning  
Recurrent neural networks  
Long short-term memory  
Weights initialization  
Weights optimization  
Systematic literature review

## ABSTRACT

Long Short-Term Memory (LSTM) is a popular Recurrent Neural Network (RNN) algorithm known for its ability to effectively analyze and process sequential data with long-term dependencies. Despite its popularity, the challenge of effectively initializing and optimizing RNN-LSTM models persists, often hindering their performance and accuracy. This study presents a systematic literature review (SLR) using an in-depth four-step approach based on the PRISMA methodology, incorporating peer-reviewed articles spanning 2018–2023. It aims to address how weight initialization and optimization techniques can bolster RNN-LSTM performance. This SLR offers a detailed overview across various applications and domains, and stands out by comprehensively analyzing modeling techniques, datasets, evaluation metrics, and programming languages associated with these networks. The findings of this SLR provide a roadmap for researchers and practitioners to enhance RNN-LSTM networks and achieve superior results.

## 1. Introduction

Deep learning (DL) has revolutionized the field of artificial intelligence in recent years, resulting in state-of-the-art performance on a variety of tasks (Al-Qubaydhi et al., 2024; Li et al., 2024), from image classification to natural language processing (Lauriola et al., 2022; Singh et al., 2022). When processing sequential data, such as text or time series, special architectures are needed to understand the relationships within the sequence. While traditional feed-forward networks struggle with this processing, Recurrent Neural Networks (RNNs) are suitable for capturing these relationships (Yu et al., 2019; Alhumoud and Al Wazrah, 2022; Beltozar-Clemente et al., 2024).

RNN, a neural network model introduced in the 1980s for time-series data modeling (Elman, 1990; Rumelhart et al., 1986; Werbos, 1988), preserves historical information through connections between

hidden units coupled with the time delay. This capability allows it to detect temporal correlations between distant events in the data. Despite its primary purpose being to learn long-term dependencies, both theoretical and experimental evidence indicate that effectively processing information over an extended period can be challenging (Bengio et al., 1994). A solution to this challenge involves adding explicit memory to the network. In 1997, Hochreiter and Schmidhuber introduced the first model of this type, called long short-term memory (LSTM), which incorporates specific hidden units that naturally tend to retain inputs for an extended period (Hochreiter and Schmidhuber, 1997). Later, Cho et al. (2014) proposed the gated recurrent unit (GRU) in late 2014, allowing each recurrent unit to adaptively capture relationships between time-scale differences. Similar to the LSTM unit, the GRU unit

<sup>☆</sup> Universiti Teknologi PETRONAS fully supports this research under the Yayasan Universiti Teknologi PETRONAS Fundamental Research Grant (YUTP-FRG 1/2021 (015LC0-370)).

\* Corresponding author at: Department of Computer and Information Sciences, Universiti Teknologi PETRONAS, Seri Iskandar, 32610, Perak, Malaysia.

E-mail addresses: [safwan\\_21002827@utp.edu.my](mailto:safwan_21002827@utp.edu.my), [saf1.alselwi@gmail.com](mailto:saf1.alselwi@gmail.com) (S.M. Al-Selwi), [mfadzil\\_hassan@utp.edu.my](mailto:mfadzil_hassan@utp.edu.my) (M.F. Hassan), [saidjadid.a@utp.edu.my](mailto:saidjadid.a@utp.edu.my) (S.J. Abdulkadir), [muneeragad@gmail.com](mailto:muneeragad@gmail.com) (A. Muneer), [ebrahim\\_22006040@utp.edu.my](mailto:ebrahim_22006040@utp.edu.my) (E.H. Sumiea), [alawi\\_18000555@utp.edu.my](mailto:alawi_18000555@utp.edu.my) (A. Alqushaibi), [mohd.gamal\\_20497@utp.edu.my](mailto:mohd.gamal_20497@utp.edu.my) (M.G. Ragab).

Peer review under responsibility of King Saud University.



Production and hosting by Elsevier

<https://doi.org/10.1016/j.jksuci.2024.102068>

Received 12 February 2024; Received in revised form 15 May 2024; Accepted 16 May 2024

Available online 21 May 2024

1319-1578/© 2024 The Author(s). Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

uses gating mechanisms to manage information flow without requiring distinct memory cells (Bengio et al., 1994).

Recently in 2017, Transformers architecture was introduced by Vaswani et al. (2017) that use attention mechanisms to focus on important parts of the sequence efficiently and have shown impressive results over LSTM in various sequential tasks that require parallel processing (Trujillo-Guerrero et al., 2023; Li et al., 2024; Ma et al., 2021). However, their complicated attention mechanism, demands extensive datasets and considerable computational resources for training, reflecting its complexity as a neural network architecture (Reza et al., 2022). On the other hand, LSTM-based models outperform Transformer in the context of multi-agent navigation and obstacle avoidance, offering a more favorable balance across various sequence lengths (Zhao et al., 2024). Ezen-Can (2020) revealed that on small datasets, bidirectional LSTM models are more accurate and quicker to train than pre-trained Transformers-based models. In financial time series forecasting, LSTM models show robustness and efficiency across multiple prediction tasks, maintaining their superiority in the field of electronic trading (Bilokon and Qiu, 2023). To sum up, selecting the right architecture among RNNs, LSTMs, and Transformers for data processing tasks is crucial. RNNs and LSTMs excel in tasks with a clear temporal sequence, with LSTMs being particularly superior at handling long-term dependencies, while Transformers are more suited for large-scale tasks that benefit from parallel processing and the ability to manage complex, long-term dependencies (Vaswani et al., 2017; Trujillo-Guerrero et al., 2023).

Network weights initialization is a crucial step applied before training the RNNs models. It has a significant impact on their convergence speed and overall training stability (Narkhede et al., 2022; Boulila et al., 2022). Inadequate initialization can lead to issues such as vanishing or exploding gradients, which hinder the model's ability to effectively capture long-term dependencies (Narkhede et al., 2022). The network's weights are initialized and then optimized repeatedly while training the developed model. This is an extensive and repeated process until the model's loss converges to a minimum loss value and an ideal matrix of weights is obtained (Bengio et al., 1994). Different initialization techniques, such as *Xavier/Glorot* (Glorot and Bengio, 2010) and *He* initialization (He et al., 2015), have been proposed to address these challenges by adapting the initial weights based on the network's architecture and activation functions.

On the other hand, network optimization is one of the core elements in DL applications, including RNN (Sun et al., 2020a). It refers to the continuous procedure of seeking the optimal values for the model's parameters of the network to minimize its loss and maximize its results (Mirjalili, 2016b). Traditional optimizers like stochastic gradient descent (SGD) (Robbins and Monro, 1951) have been the foundation. They rely on gradient information from the objective function during each iteration. While these techniques are effective for finding optimal solutions, they are susceptible to getting stuck in poor local minima. Additionally, gradient-based algorithms often face challenges in achieving high accuracy while suffering from slow training speeds (Chia et al., 2021). Later on, modern variants such as adaptive learning rate optimizers, such as *Adam* (Kingma and Ba, 2014), *RMSprop* (Tieleman and Hinton, 2012), and *AdaGrad* (Duchi et al., 2011) have been developed to overcome some of these limitations, by adaptively adjusting the learning rate during the training process (Prokhorov et al., 2002; Chia et al., 2021). Moreover, metaheuristic algorithms have special advantages over other optimization techniques, such as their flexibility, simplicity, and gradient-free nature (Dhiman and Kumar, 2019; Ragab et al., 2020). These algorithms do not involve computing gradients but focus on evaluating the objective function for feasible solutions, thereby reducing computational complexity significantly. Their adaptable nature allows easy customization for diverse problems by modifying the objective function, making them widely applicable across various fields (Bahreininejad, 2019; Mirjalili and Lewis, 2016). Finally, boosting algorithms, a family of ensemble learning techniques, utilize an iterative approach for generating a strong classifier (Zhang and Ma,

2012). The selection of an appropriate optimizer depends on factors such as the dataset, model complexity, and available computational resources.

Therefore, the motivation for this systematic literature review (SLR) is to provide a valuable resource for researchers and practitioners who are interested in applying RNN-LSTMs to long-term dependencies, by summarizing the current understanding of weights initialization and optimization techniques in RNN-LSTM. It is believed that the most recent developments in this field provide a powerful impetus for analyzing and discussing current techniques, applications, developments, research challenges, and potential future study orientations. Consequently, this research is structured around a compelling central point by defining specific research questions (Section 4.1.2) that strongly connect the concepts surrounding the primary objective. No comprehensive SLR has been undertaken to emphasize current domains, applications, datasets, and techniques in RNN-LSTM weights initialization and optimization using detailed facts and figures. This SLR examines the literature published over the past six years (2018–2023) and identifies the following as the most significant contributions:

1. Presenting a systematic literature review with an in-depth four-step approach to methodically include studies concerning the initialization and optimization of LSTM weights (shown in Fig. 3).
2. Conducting an extensive discussion on the current domains and applications into which LSTM is used, including their architectures, datasets, evaluation metrics, strengths, limitations, and their contribution to the field (Section 5.1).
3. Identification of the current dominated techniques and tools used to build the LSTM models, including the preferred and most used programming languages (Section 5.2.1) and the most used evaluation metrics in regression and classification problems (Section 5.2.2).
4. Providing an extensive analysis of various weight initialization techniques specifically designed for RNN-LSTM networks (Section 5.3). These techniques include among others, *random* initialization with uniform or Gaussian distributions, *Xavier/Glorot* initialization, and *He* initialization. The review examines the effects of different weight initialization techniques on the performance and convergence of LSTM networks in different applications.
5. Comprehensively summarizing numerous optimization algorithms used in optimizing LSTM networks (Section 5.4). These algorithms have been classified into five main groups: *adaptive learning rate*, *gradient descent-based*, *metaheuristic*, *boosting*, and *other algorithms*. The characteristics, advantages, and disadvantages of each one of these algorithms have been compared in multiple tables of comparisons to give the researchers a full understanding of these techniques.
6. Illustrating and emphasizing key insights from the synthesized data via a methodical mapping strategy to illustrate research intensity in the covered field (Section 5.5).

The findings of this SLR will provide a valuable resource for practitioners and researchers who are interested in applying RNN-LSTMs in their experiments, by summarizing the current understanding of this important area of DL and providing insights into how to choose the best weights initialization and optimization algorithm for LSTMs for different tasks. Moreover, insights from the dissection studies will form a paradigm for research challenges, suggestions, and future research directions. This SLR will help beginners and expert researchers grasp the current state of research, associated fundamental ideas, and future research directions in this area.

The review begins by introducing the background and context of LSTMs, including the RNN-LSTM overall training process, and LSTM operations. Next, this SLR presents a comprehensive systematic review of the existing current studies on LSTM, covering a wide range of applications and domains, including speech recognition, natural language processing, video analysis, and stock market prediction, among

others. These studies will be analyzed in terms of their methodologies, results, and contributions to the field, highlighting their strengths and limitations. Finally, the review will conclude with a discussion of the current state-of-the-art in the field of LSTMs, identifying the key challenges and opportunities for future research. This will include a critical assessment of the existing studies and a proposal for future directions for research, including the need for further investigation into the optimal design, training, and deployment of LSTMs for different tasks and domains.

The rest of this paper is structured as follows. Firstly, LSTM is discussed in Section 2. Next in Section 3, a summary of SLR/survey studies from the existing literature is provided. Then, Section 4 outlines the approach taken for this systematic study. The subsequent Section 5, addresses all the presented research questions by analyzing synthesized data from the included studies. Lastly, in Section 6, the study is concluded with theoretical implications, study limitations, and concluding remarks.

## 2. Long Short-Term Memory

LSTM has been specifically designed to address the issue of vanishing gradients, which makes vanilla RNNs unsuitable for learning long-term dependencies (Jaydip and Sidra, 2022). LSTMs possess the capacity to process sequential data and retain information from previous steps in the sequence, enabling them to predict future steps effectively. This characteristic makes them highly suitable for tasks involving long-term dependencies (Al-Selwi et al., 2023). LSTMs utilize a set of gates to regulate the information flow into and out of the memory cell, allowing them to mitigate the vanishing gradient problem that affects vanilla RNNs (Hochreiter and Schmidhuber, 1997; Muneer et al., 2022). As a result, LSTMs are proficient in learning long-term dependencies, where information from earlier time steps is important for predicting later time steps. Since their introduction in 1997 by Hochreiter and Schmidhuber (1997), LSTMs have become widely used and highly effective in various sequential data tasks, from speech recognition to machine fault prediction. They have been shown to outperform other types of RNNs and traditional ML methods in several tasks.

LSTM can be conceptualized as an RNN augmented with a memory pool and two key vectors: Short-term state, and Long-term state. The former preserves the current time-step's output, while the latter handles long-term information as it traverses through the network (Al-Selwi et al., 2023). During the progression of time, both the cell states and hidden states move forward, continuously updating and impacting each other at every time step. While the hidden state serves as an output, the cell state typically remains internal, being passed and modified along with the hidden states at each time step, thus influencing the hidden state in the process. Additionally, information flow within LSTM networks is governed by three internal gates during the learning process: the forget gate, the input gate, and the output gate as shown in the architecture of LSTM in Fig. 1. The forget gate ( $f_t$ ) determines which information from the cell state should be discarded. The input gate ( $i_t$ ) decides which new information will be added to the cell state. Finally, the output gate ( $o_t$ ) controls the output of the hidden state. This gating mechanism allows LSTM units to effectively manage long-term dependencies by selectively updating and retaining information over time.

This section further comprises two subsections: RNN overall training (2.1) and LSTM operations and computational complexity (2.2).

### 2.1. RNN training

The overall training process of RNN is shown in Fig. 2. It consists of a forward and backward pass as per the following steps:

- (a) Perform the forward pass using the initialized weights/ hidden states.

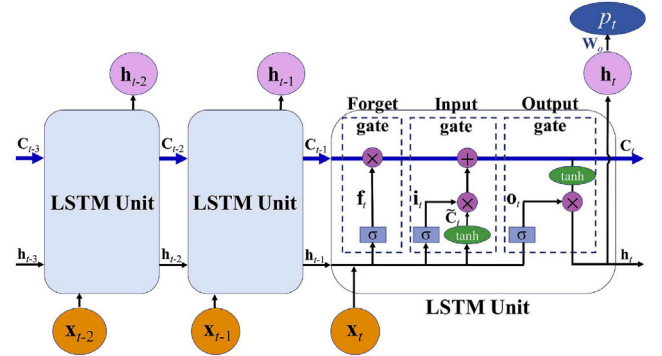


Fig. 1. LSTM architecture (Wei et al., 2021).

- (b) Compute the loss function using the given observed and predicted values:

$$J(W, b) = \frac{1}{n} \sum_{i=1}^n L(\hat{y}^i, y^i)^2 \quad (1)$$

Where:  $W$  denotes the weight matrix,  $b$  denotes the bias vector,  $n$  denotes the number of inputs,  $\hat{y}^i$  denotes the predicted value,  $y^i$  denotes the observed value, and  $L$  denotes the loss value between the predicted and the observed value. The cost function  $J$  is used to figure out how much the loss is after each iteration.

- (c) Perform the backward pass to get the required gradients to update the network weights and reduce (optimize) the loss function.
- (d) Update the weights using the acquired gradients and repeat steps (a) and (b).
- (e) Repeat steps (a) to (d) until an accepted loss threshold is reached.

### 2.2. LSTM operations and computational complexity

The functions of a single cell LSTM are detailed in Algorithm 1 (Thakkar and Chaudhari, 2021), where  $\sigma$  denotes the logistic sigmoid function,  $\tanh$  represents the hyperbolic tangent function, and  $\odot$  denotes the Hadamard product, which is the element-wise product (Hochreiter and Schmidhuber, 1997).

Based on experimental findings, LSTM demonstrates superior performance compared to Vanilla RNN. However, a drawback of LSTM lies in its complexity as it contains four times more parameters than Vanilla RNNs. Consequently, this leads to higher complexity in the hidden layers (Rashid et al., 2018). LSTM's increased number of trainable weights is noteworthy, featuring four separate sets of trainable input weights for input data:  $[W_f, W_i, W_o, W_c]$ , and four sets of trainable recurrent weights for hidden states:  $[U_f, U_i, U_o, U_c]$ . Additionally, there are four distinct trainable bias terms:  $[b_f, b_i, b_o, b_c]$ . This makes it crucial to set some initial values for the weights of the network before starting the experiment.

LSTM computational complexity can be estimated by calculating the number of operations and trainable parameters. For a network with input size  $m$  and output size  $n$ , the LSTM has matrices  $U$  of size  $nm$  and  $W$  of size  $mn$  for three gates and the cell state, plus  $n$  biases. The memory complexity is  $4(nm + n^2 + n)$ . The time complexity for one training step involves  $mn(n + (m - 1)) = n^2m + nm^2 - nm$  operations. This is done for each of the three gates, the cell state, and over  $k$  time steps, yielding  $4k(n^2m + nm^2 - nm)$  (Petrozziello et al., 2022). Consequently, this can result in longer training times, especially when dealing with large datasets or when using a high number of LSTM units. The necessity of processing the input sequentially also contributes to the extended training duration. As such, optimizing LSTM networks for large-scale data often requires significant computational resources and efficient implementation strategies to manage the increased computational load effectively.

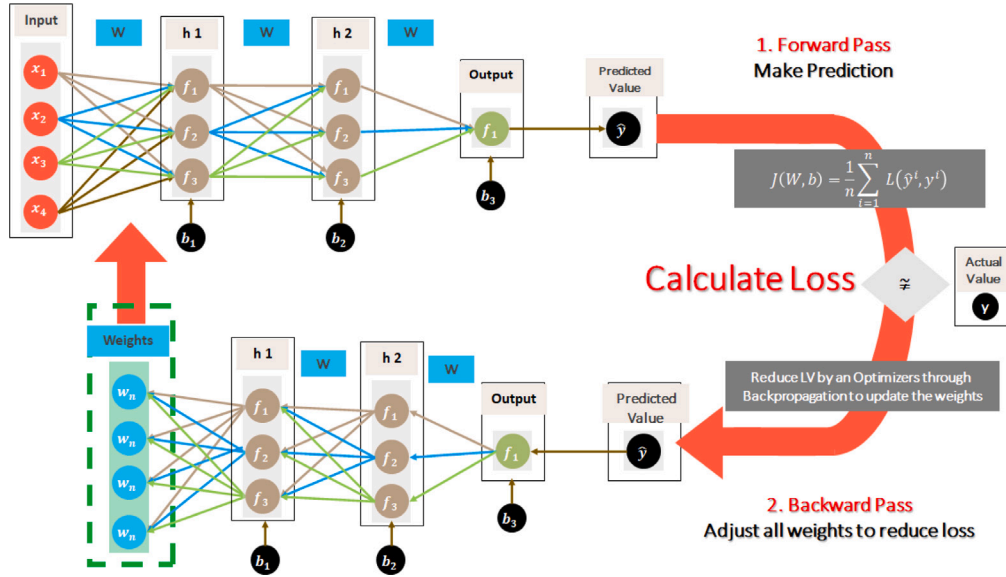


Fig. 2. Recurrent neural network overall training (Al-Selwi et al., 2023).

#### Algorithm 1 Single cell LSTM operations

Given

1.  $W$  Matrix of input-to-hidden weights
2.  $U$  Matrix of hidden-to-hidden weights
3.  $b$  Vector of Bias

Input

1.  $x_t$  Vector of input at time step  $t$
2.  $c_{t-1}$  Vector of previous memory cell state
3.  $h_{t-1}$  Vector of previous hidden state

Output

1.  $c_t$  Vector of memory cell state
2.  $h_t$  Vector of the hidden state

Process

1. Compute the input gate vector:  

$$i_t = \sigma(W^{(i)}x_t + U^{(i)}h_{t-1} + b^{(i)})$$
2. Compute the forget gate vector:  

$$f_t = \sigma(W^{(f)}x_t + U^{(f)}h_{t-1} + b^{(f)})$$
3. Compute the output gate vector:  

$$o_t = \sigma(W^{(o)}x_t + U^{(o)}h_{t-1} + b^{(o)})$$
4. Compute the memory cell state vector:  

$$c_t = i_t \odot u_t + f_t \odot c_{t-1}$$

where:

$$u_t = \tanh(W^{(u)}x_t + U^{(u)}h_{t-1} + b^{(u)})$$
5. Compute the hidden state vector:  

$$h_t = o_t \odot \tanh(c_t)$$

### 3. Literature review

This section investigates the related works from similar SLRs, reviews, and survey papers. For instance, a review on LSTM conducted by Van Houdt et al. (2020), discusses the architecture of the LSTM model, explaining its key components such as the memory cell, input gate, forget gate, and output gate. They also delve into the training process of LSTMs, exploring how the model is optimized using gradient descent and backpropagation through time. However, they have not provided any specific information about gradient descent-based optimization or any of the other optimization techniques mentioned in Table 1.

Similarly, Greff et al. (2017) employed gradient descent-based optimization as their primary optimization technique in their eight LSTM variants experiments. However, no specific information is provided regarding adaptive learning rate methods, LSTM-specific optimization techniques, weight initialization strategies, or metaheuristic algorithms in their study. In another study, Yu et al. (2019) published their work in 2019 about LSTM cells and architectures, and their work missing the details regarding the optimization techniques that can be applied to LSTM networks, specifically for adaptive learning rate methods, LSTM-specific optimization techniques, weight initialization strategies, and metaheuristic algorithms. Moreover, Narkhede et al. (2022) conducted their study in 2022 and their main focus was weight initialization techniques in RNNs. However, they have not covered any specific information regarding weight optimization techniques.

Furthermore, Prokhorov et al. (2002) provided an overview of the adaptive behavior of RNNs with fixed weights, explicitly discussing the RNN's optimization techniques. The scope of their paper is different from the optimization-based methods targeted in this SLR. In contrast, Weerakody et al. (2021) presented a review on handling irregular time series data using gated RNNs. Their review paper covers gradient descent-based optimization and also addresses LSTM-specific optimization techniques. However, it does not include adaptive learning rate methods, weight initialization strategies, or metaheuristic algorithms.

Besides, Abdulrahman et al. (2021) conducted a review focused on deep RNNs for electricity forecasting in residential buildings. Their review focused only on optimization of LSTM and RNNs architecture such as modifications to the LSTM architecture or training algorithms, and how they improve performance in the context of electricity forecasting



in residential buildings. The authors [Torres et al. \(2021\)](#) conducted a survey on DL techniques for time series forecasting and the authors acknowledged the importance of gradient descent-based optimization methods in DL, particularly for time series forecasting and they highlighted the relevance of some techniques in mitigating overfitting and improving generalization performance.

In addition, [Surakhi et al. \(2020\)](#) study covers several optimization techniques, including gradient descent-based optimization, adaptive learning rate methods, and LSTMs-specific optimization technique in the context of air pollution forecasting using ensemble RNNs. However, it suffers from the weight initialization strategies investigation. In another study, [Lipton et al. \(2015\)](#) critically reviewed RNNs for sequence learning and their review acknowledges the significance of gradient descent-based optimization and adaptive learning rate methods in RNN for sequence learning. Hence, further examination of the different optimization algorithms used in RNNs and their advantages and limitations would provide valuable insights into the literature. However, they have not extensively reviewed the optimization methods and weight initialization strategies.

[Bianchi et al. \(2017\)](#) conducted a comparative analysis in 2017 and the authors emphasized the importance of various optimization techniques in short-term load forecasting using RNNs models. The paper analyzed the gradient descent-based optimization, adaptive learning rate methods, and weight initialization strategies, specifically in the context of load forecasting. However, metaheuristic algorithms were not investigated. Hence, considering their potential application in improving RNN-based load forecasting models and comparing their performance with traditional optimization methods would be an interesting avenue for research.

Finally, [Salehinejad et al. \(2017\)](#) reviewed recent advances in RNNs, their initialization, optimization, and architectures. However, this study slightly discusses the metaheuristic algorithms. [Sezer et al. \(2020\)](#) conducted an SLR on DL for financial time series forecasting. [Gallicchio and Scardapane \(2020\)](#) focused on deep randomized neural networks. [Boulila et al. \(2022\)](#) discussed weight initialization techniques for DL in remote sensing. [Lalapura et al. \(2021\)](#) conducted a survey on RNNs for edge intelligence. [Vanitha and Jayashree. \(2023\)](#) conducted an SLR on the impact of DL in educational time series datasets. [Zeebaree et al. \(2021\)](#) reviewed the prediction process based on deep RNNs. [Fantin and Hadad \(2022\)](#) conducted a bibliometric analysis and SLR on stock price forecasting with LSTM networks. [Alhumoud and Al Wazrah \(2022\)](#) presented a review on Arabic sentiment analysis using RNNs. [Lukosevicius and Jaeger \(2009\)](#) discussed reservoir computing approaches to RNN training. However, all of these studies were either scoping review studies or not extensively focused on weight initialization and/or optimization techniques in LSTM models, unlike this proposed comprehensive SLR.

To sum up, 15 relevant reviews, survey studies, and SLRs published within the last 6 years, and 6 studies published in the past have been identified, so a total of 21 relevant reviews have been identified. This study aims to provide the most up-to-date information on recent advancements in the field of weight initialization and optimization by including the most recent articles and covering literature published between 2018 and 2023. Upon conducting a comparative analysis, several studies, namely [Torres et al. \(2021\)](#), [Surakhi et al. \(2020\)](#), [Lipton et al. \(2015\)](#), [Bianchi et al. \(2017\)](#), and [Salehinejad et al. \(2017\)](#), have been observed to encompass a wide range of optimization techniques. These techniques include gradient descent-based optimization, adaptive learning rate methods, some LSTM-specific optimization techniques, and weight initialization strategies. However, it is noteworthy that none of these studies specifically focus on metaheuristic algorithms, except for a slight discussion in [Salehinejad et al. \(2017\)](#).

This analysis reveals there is a need for an updated SLR that comprehensively addresses all aspects of weight initialization strategies and optimization methods, including the incorporation of metaheuristic algorithms, within the context of LSTM. This identified gap in the

existing literature serves as the impetus for undertaking this review, which aims to synthesize and analyze the most recent research findings. Through this endeavor, the aim is to provide valuable insights into the advancements, challenges, and potential future directions in optimizing RNNs for diverse applications.

Efficient weight initialization strategies and optimization methods hold significant importance in the training of RNNs and improving their overall performance. The success of RNNs models is intricately linked to the identification of appropriate weight initialization techniques and the selection of optimal optimization algorithms. By addressing these critical aspects, the aim is to recognize their significance in achieving superior results and overcoming challenges encountered during RNNs training. Finally, this study highlights the paramount importance of exploring weight initialization strategies and optimization methods within the context of RNNs. Notably, existing works of literature lack comprehensive coverage of metaheuristic algorithms. Therefore, this review endeavors to bridge this research gap by providing a comprehensive analysis that encompasses all relevant aspects. Leveraging the latest research findings, the aim is to offer valuable insights that can guide future research endeavors and foster advancements in the optimization of RNNs across a wide range of real-world applications.

In the subsequent Section 4, a comprehensive explanation is provided about the methodology employed to conduct this study with the aim of accomplishing the research objectives.

## 4. Methodology

The present SLR was carried out following the guidelines provided by the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) ([Page et al., 2021](#)). The study also adhered to the standard recommendations proposed by [Kitchenham and Charters \(2007\)](#). Additionally, to ensure comprehensive reporting, the structure, and approach of previous extensive studies were adopted ([Abdullah et al., 2023](#); [Talpur et al., 2023](#); [Sumiea et al., 2024](#); [Ragab et al., 2024](#)).

The mapping process used in this SLR consists of four connected steps, which are depicted in [Fig. 3](#) and further elaborated in Sections 4.1, 4.2, 4.3, and 4.4. For more comprehensive details regarding the mapping process, previous works by [Kitchenham and Charters \(2007\)](#) and [Page et al. \(2021\)](#) offer further insights.

### 4.1. Step 1: Preliminary study

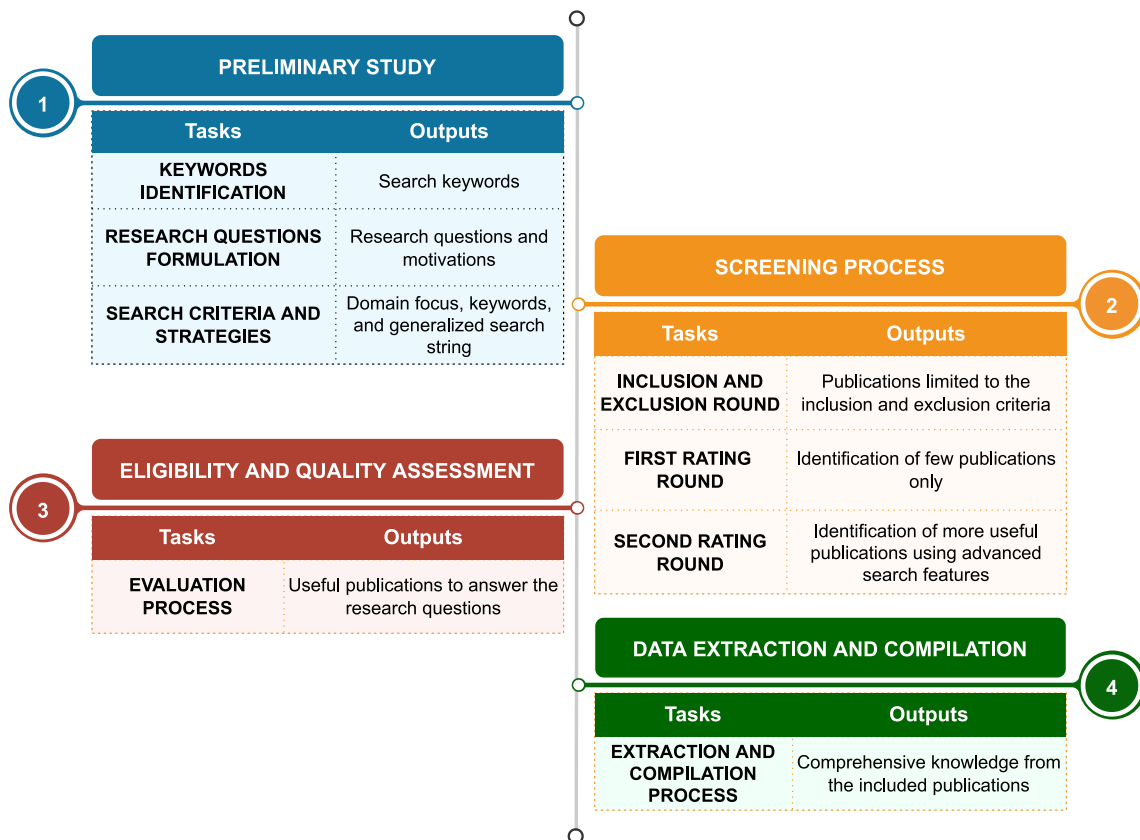
The preliminary study represents a crucial and primary phase during the SLR process. Consequently, an initial exploration was performed to gain a deeper understanding of the primary subject matter. This step additionally acts as a startup motivation, inspiring authors to identify solid research questions (RQs), keywords, and the scopes of their studies. As part of this initial phase, three tasks were undertaken: first, the identification of keywords [4.1.1](#); second, the formulation of SLR [4.1.2](#); and third, the identification of the search criteria and strategies [4.1.3](#).

#### 4.1.1. Keywords identification

For the preliminary search, a search in Google Scholar was conducted with the keyword "LSTM" to explore the available papers on this subject. Subsequently, related variations and additional keywords were identified to refine the search further. In this stage, relevant papers were screened to aid in addressing the SLR. These retrieved papers assisted in identifying more relevant keywords and providing a broad understanding of the search scope. Subsequently, the keywords were analyzed to identify the most suitable combinations that would yield the most relevant papers related to LSTM or "long short-term memory". This combination was done by two authors of this study (S.M. Al-Selwi and A. Muneer). The outcomes of these keyword combinations and the number of papers retrieved from the Google Scholar search are depicted in [Fig. 4](#).

**Table 1**  
Summary of the state-of-the-art RNNs-based review papers.

| Study                            | Gradient Descent-Based Optimization | Adaptive Learning Rate Methods | LSTMs-Specific Optimization Techniques | Weight Initialization Strategies | Metaheuristic Algorithms |
|----------------------------------|-------------------------------------|--------------------------------|----------------------------------------|----------------------------------|--------------------------|
| Vanitha and Jayashree. (2023)    | ✓                                   | X                              | ✓                                      | X                                | X                        |
| Narkhede et al. (2022)           | X                                   | X                              | ✓                                      | ✓                                | ✓                        |
| Fantin and Hadad (2022)          | X                                   | X                              | X                                      | X                                | X                        |
| Alhumoud and Al Wazrah (2022)    | X                                   | X                              | X                                      | X                                | X                        |
| Boulila et al. (2022)            | X                                   | X                              | X                                      | ✓                                | X                        |
| Lalapura et al. (2021)           | ✓                                   | ✓                              | X                                      | X                                | X                        |
| Zeebaree et al. (2021)           | X                                   | X                              | ✓                                      | ✓                                | X                        |
| Weerakody et al. (2021)          | ✓                                   | X                              | ✓                                      | X                                | X                        |
| Abdulrahman et al. (2021)        | X                                   | X                              | ✓                                      | X                                | X                        |
| Torres et al. (2021)             | ✓                                   | ✓                              | ✓                                      | X                                | X                        |
| Gallicchio and Scardapane (2020) | X                                   | X                              | X                                      | ✓                                | X                        |
| Van Houdt et al. (2020)          | ✓                                   | X                              | ✓                                      | X                                | X                        |
| Sezer et al. (2020)              | ✓                                   | ✓                              | ✓                                      | X                                | X                        |
| Surakhi et al. (2020)            | ✓                                   | ✓                              | ✓                                      | X                                | X                        |
| Yu et al. (2019)                 | X                                   | X                              | ✓                                      | X                                | X                        |
| Bianchi et al. (2017)            | ✓                                   | ✓                              | ✓                                      | ✓                                | X                        |
| Salehinejad et al. (2017)        | ✓                                   | ✓                              | ✓                                      | ✓                                | -                        |
| Greff et al. (2017)              | ✓                                   | X                              | X                                      | X                                | X                        |
| Lipton et al. (2015)             | ✓                                   | ✓                              | ✓                                      | X                                | X                        |
| Lukosevicius and Jaeger (2009)   | ✓                                   | X                              | ✓                                      | ✓                                | X                        |
| Prokhorov et al. (2002)          | X                                   | ✓                              | X                                      | X                                | X                        |
| This Study                       | ✓                                   | ✓                              | ✓                                      | ✓                                | ✓                        |



**Fig. 3.** The literature study mapping process.

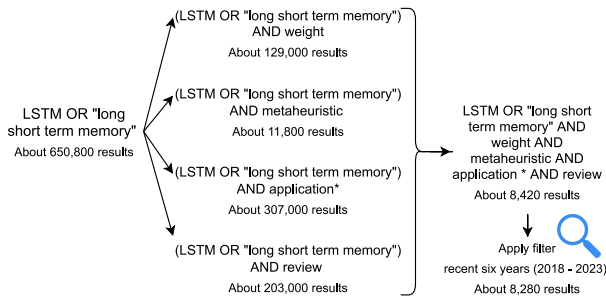


Fig. 4. Google Scholar results for keywords combination.

#### 4.1.2. Research questions formulation

From the previous literature survey, the RQs and their motivations were formulated for this study by four authors (S.M. Al-Selwi, MF. Hassan, SJ Abdulkadir, and A. Muneer) as they worked together to form the following RQs:

- RQ1: What are the current applications and domains into which LSTM networks are used?**  
*Motivation:* To investigate the different usages of RNN-LSTM and how its learning ability impacts several domains and becomes state-of-the-art in numerous fields including among others, energy, speech recognition, manufacturing, natural language processing, and stock market prediction.
- RQ2: What are the current modeling techniques and tools used to build the LSTM models?**  
*Motivation:* To explore the current modeling techniques and tools available for RNN-LSTM development including the programming languages, frameworks, libraries, and evaluation metrics.
- RQ3: What are the weights initialization techniques used in LSTM networks?**  
*Motivation:* Weights initialization directly affects the network convergence. Thus, it is a crucial step before the training of LSTM models. During training, the weights of a network are initialized and subsequently changed many times. This procedure is repeated until the loss converges to a minimum and an optimal weight matrix is achieved. Hence, there is a need to explore the different techniques used to initialize the weights in RNN-LSTM networks
- RQ4: What are the weights optimization techniques used in LSTM networks?**  
*Motivation:* To gain a comprehensive understanding of the most effective weight optimization techniques for RNN-LSTM networks, such as gradient descent-based algorithms, adaptive learning rate algorithms, and metaheuristic algorithms.
- RQ5: What is the intensity of publications related to LSTM?**  
*Motivation:* To investigate the frequency and volume of publications and the prevailing research trends concerning RNN-LSTM and suggest potential future directions.

#### 4.1.3. Search criteria and strategies

Subsequently, it becomes crucial to focus the search on specific domains by applying appropriate search criteria. Hence, five scientific databases and venues for the literature search were selected: Scopus, IEEE Xplore, Web of Science, ScienceDirect, and ACM Digital Library. A search based on titles, abstracts, and keywords has been done to retrieve related LSTM studies. The selected publication types include journals, conference proceedings, and book chapters published from 2018 to 2023, representing the recent six-year period.

To ensure that the search results contained relevant information to address the SLR mentioned in the previous point 4.1.2, the selected search keywords were combined using appropriate advanced search

**Table 2**  
Search criteria and strategies.

| Domain focus | Keywords                               | Search scheme                 | Generalized search string                       |
|--------------|----------------------------------------|-------------------------------|-------------------------------------------------|
| LSTM         | LSTM, "long short term memory", weight | Title, abstract, and keywords | ("LSTM" OR "long short term memory") AND weight |

**Table 3**  
Inclusion and exclusion criteria for search screening.

| Criteria          | Inclusion Criteria                                        | Exclusion Criteria                                                |
|-------------------|-----------------------------------------------------------|-------------------------------------------------------------------|
| Publication year  | 2018–2023                                                 | Not between 2018–2023                                             |
| Article Type      | Journal papers, conference proceedings, and book chapters | Review papers, tutorials, seminars, interviews, letters, or blogs |
| Language          | English                                                   | Non-English                                                       |
| Availability      | Studies available in full text                            | Studies not available in full text (less than 5 pages)            |
| Relevance         | Relevant to the RQs                                       | Not relevant to the RQs                                           |
| Area              | Studies in Computer Science and Engineering areas only    | Publications not in Computer Science and Engineering areas        |
| Publication Stage | Final (Published)                                         | Not published yet (Articles in press)                             |
| Access Type       | Open access                                               | Not open access                                                   |

logical operators (OR and AND) and wildcards. This refined search was conducted across the five chosen scientific databases and venues. However, this initial search yielded a substantial number of publications. To further narrow down the search, a generalized search string was formulated and applied to the titles, abstracts, and keywords of the literature. The domain focus, keywords, search scheme, and generalized search string employed for the SLR search are presented in Table 2.

After executing the search criteria and strategies (Table 2) on the five scientific databases and venues, a total of 1360 studies were identified. The latest search was conducted on 06/04/2023. Following the PRISMA standards (Page et al., 2021), relevant publications were identified. Subsequently, the removal of duplicated studies was carried out using EndNote 20.5 and Microsoft Excel, resulting in 794 unique studies. The procedure flow across the different stages of this SLR study is illustrated in the PRISMA flowchart (Fig. 5) generated by the PRISMA flowchart tool by Haddaway et al. (2022).

The Appendix includes the written search string in each scientific database and venue, along with easy direct hyperlinks to explore the search results from each source. Additionally, it includes the PRISMA 2020 Checklist used in this study.

#### 4.2. Step 2: Screening process

Following the database search, which utilized the identified keywords and the generalized search string, all collected unique 794 papers have gone through three rounds of screening: the inclusion and exclusion round, as well as two rounds of ratings. This process was assessed by three authors of this study (S.M. Al-Selwi, A. Muneer, and E. Sumiea) and they worked together to finalize the screening.

##### 4.2.1. Inclusion and exclusion round

During the screening process, all collected papers were carefully assessed based on the inclusion and exclusion criteria outlined in Table 3. These criteria served as the guidelines for filtering and selecting papers that were directly related to addressing the RQs. Papers failing to meet the specified criteria were excluded.

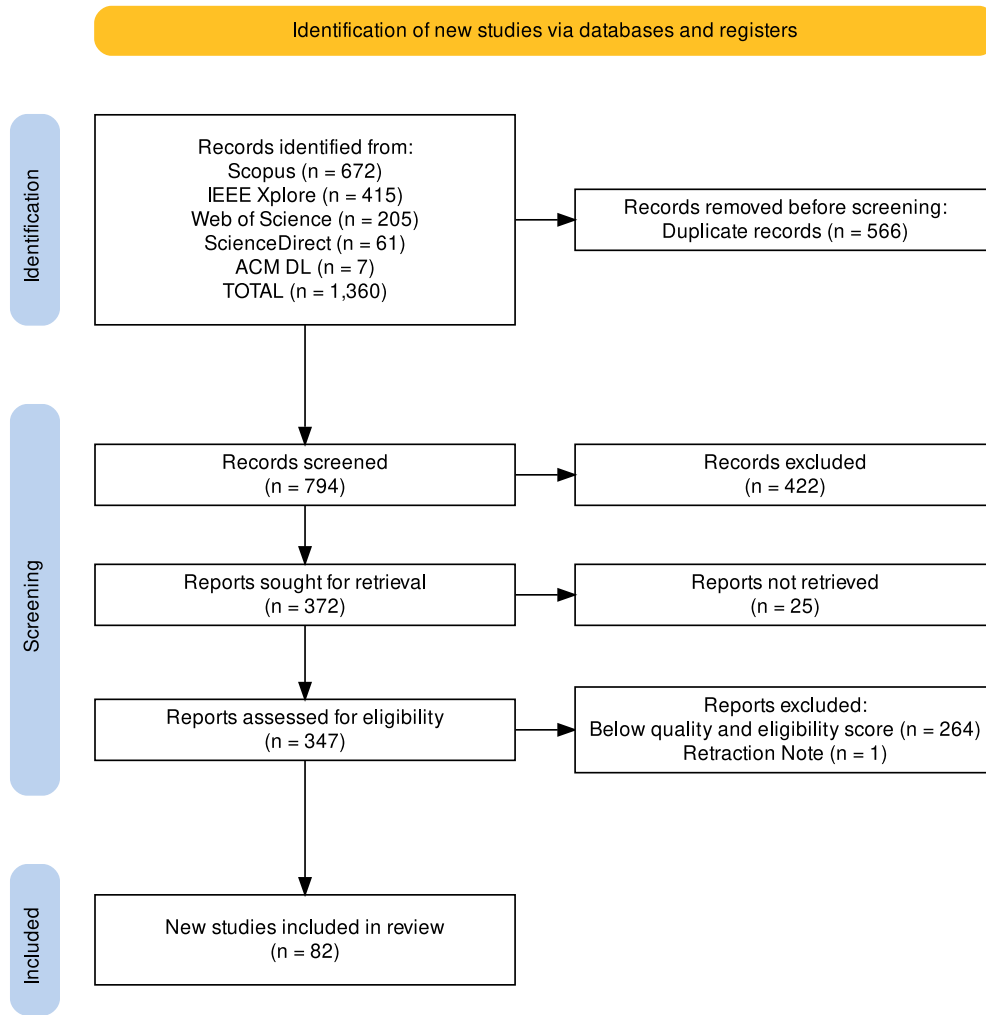


Fig. 5. PRISMA flowchart of the systematic literature process.

#### 4.2.2. First rating round

In the first round of rating, a formal data analysis was conducted to assess the relevancy of each paper based on its title, abstract, and conclusion. Each paper went through a rigorous examination to determine its relevancy to the topic of LSTM weights initialization and optimization techniques, ensuring no important studies were overlooked. However, at the end of this round, it was found that the used weights initialization and/or optimization techniques were not explicitly mentioned in the titles, abstracts, and conclusions of the screened papers. Consequently, only 127 papers were retained at the end of this round, which was deemed insufficient to fulfill all the required objectives. As a result, a second round of screening was necessary to enhance the findings.

#### 4.2.3. Second rating round

In the second round of rating, the advanced search feature in Adobe Acrobat Reader (Version 2023.001.20174) and Foxit PDF Reader (Version 12.1.2.15332), was utilized to search for specific weights initialization and/or optimization techniques within all the downloaded PDF papers files. As a result of this thorough search, a total of 347 papers were identified and included for further analysis.

#### 4.3. Step 3: Eligibility and quality assessment

Once the screening process was completed, the remaining 347 papers were subjected to an eligibility and quality assessment (E&QA)

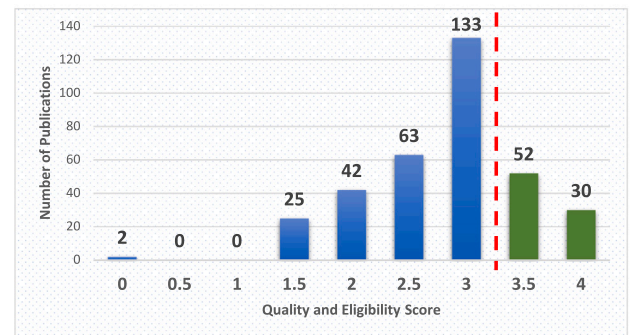


Fig. 6. Quality score distribution of the publications.

before being added to the final list of included papers. As detailed in Table 4, the evaluation was done by applying specific criteria with corresponding scoring values: 1 for “agree” 0.5 for “partly agree”, and 0 for “disagree”. The purpose of this evaluation was to ensure that the final selection included only papers with well-defined objectives, well-explained methodologies, well-acknowledged limitations, and clear findings.

The E&QA steps utilized the scoring criteria outlined in Table 4. The E&QA score ranges from 0 to 4, with 4 representing the highest



**Table 4**  
Criteria for scoring the eligibility and quality assessment.

| Criteria                    | Score | Description                                                                         |
|-----------------------------|-------|-------------------------------------------------------------------------------------|
| Well-defined objectives?    | 1     | Indeed, the paper clearly outlines its objectives and goals.                        |
|                             | 0.5   | The paper outlines its objectives, though the goals are not explicitly defined.     |
|                             | 0     | No, the paper does not have well-defined objectives.                                |
| Well-explained methodology? | 1     | Yes, the paper showcases a comprehensive, and well-documented methodology.          |
|                             | 0.5   | Partially documented methodology that needs more details.                           |
|                             | 0     | No, the paper fails to provide a well-explained methodology.                        |
| Declared limitations?       | 1     | Yes, the limitations are mentioned within the paper.                                |
|                             | 0.5   | The limitations are not in full detail.                                             |
|                             | 0     | No, no declared limitations in this paper.                                          |
| Clear research findings?    | 1     | Indeed, the study provides clear findings with appropriate visualizations.          |
|                             | 0.5   | The study does present its findings, but not in clear statements or visualizations. |
|                             | 0     | The study's research findings are unclear, and lacking elaboration.                 |

**Table 5**  
Studies retrieved in each step of the systematic literature process.

| Source         | Identified | Unique | Screened | Pass E&QA | Included |
|----------------|------------|--------|----------|-----------|----------|
| IEEE Xplore    | 415        | 394    | 194      | 37        | 37       |
| Scopus         | 672        | 307    | 100      | 32        | 32       |
| Science Direct | 61         | 57     | 40       | 11        | 11       |
| Web of Science | 205        | 29     | 9        | 1         | 1        |
| ACM DL         | 7          | 7      | 4        | 1         | 1        |
| Total          | 1360       | 794    | 347      | 82        | 82       |

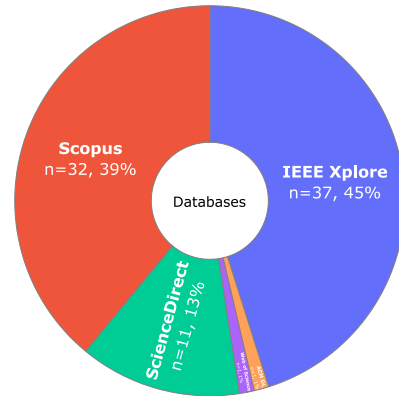
score for excellent-quality papers. To ensure the inclusion of only high-quality studies in the final list of included papers, a minimum score of 3.5 was considered for the selected studies, as illustrated in Fig. 6. As a result, 82 out of 347 studies were deemed eligible and included in the qualitative synthesis of this investigation, while the rest were marked ineligible and excluded. The decision to exclude studies that did not meet the scoring criteria was assessed by two authors of this study (S.M. Al-Selwi and A. Muneer) and it was not made carelessly as it followed a comprehensive evaluation of their eligibility and quality.

#### 4.4. Step 4: Data extraction and compilation of included studies

After completing the E&QA, the final selected studies along with their associated metadata were downloaded from the scientific sources and sorted into a spreadsheet (Which can be downloaded from the Data Availability section). The metadata collected from these studies were done by two authors of this study (S.M. Al-Selwi and E. Sumiea) and it includes title, abstract, author, volume, number, pages, publisher, secondary title, place published, uniform resource locator (URL), digital object identifier (DOI), keywords, reference type, publication year, name of database, application, domain, architecture, objectives, motivation, advantages, limitations, datasets, evaluation metrics, and experiment setup.

Finally, 82 studies were included and thoroughly analyzed to answer the RQs of this SLR. The most valuable information extracted from these studies encompassed information regarding existing LSTM architectures, their domains and applications, algorithms used for weights initialization or optimization, datasets utilized, evaluation metrics, and the tools and techniques employed for experimentation. Table 5 presents the number of studies retrieved from the scientific databases and venues at each step of the SLR process.

This SLR only included studies with explicit objectives, clarified methodologies, declared limitations, and stated research results. Therefore, the information in Table 5 leads us to the conclusion that high-quality and relevant studies can mostly be found in IEEE Xplore (37 studies, 45%), followed by Scopus (32 studies, 39%), ScienceDirect (11 studies, 13%), and finally Web of Science and ACM DL with (1 study, 1%) each. Fig. 7 illustrates the distribution of included studies by scientific databases and venues.



**Fig. 7.** Distribution of included studies by scientific databases and venues.

The next section (Section 5) is dedicated to the data synthesis and analysis of the included studies.

## 5. Synthesis of data and analysis

In this section, the goal is to consolidate and condense the information extracted from the included studies into visual aids and provide answers to the previously posed RQs (Section 4.1.2). The synthesized data aims to offer a comprehensive understanding of the present state of LSTM research, recent applications, and the challenges involved.

### 5.1. RQ1: What are the current applications and domains into which LSTM networks are used?

Based on the included 82 studies, the current applications and domains of LSTM networks can be categorized into various domains as shown in Fig. 8. The domains in ascending order of the most used are Healthcare and Medical, Energy, Natural Language Processing, Prognostics and Health Management, Computer Vision, Communication and Networking, Stock Market and Financial Prediction, Speech Recognition and Synthesis, Safety, Environment, and Manufacturing. LSTM networks have proven to be versatile and valuable in tackling challenges across these diverse domains. It is worth mentioning that the categorization of studies into these domains is based on the authors' best-fit decision. That means, a study may fit into multiple domains, so the authors have chosen the best-fit domain for it based on their intensive reading and systematic categorization of the papers.

#### 5.1.1. Healthcare and medical domain

LSTM holds immense potential in the Health and Medical domain offering solutions to a wide range of challenges and applications. It can automate ECG-based biometric systems for subject classification,

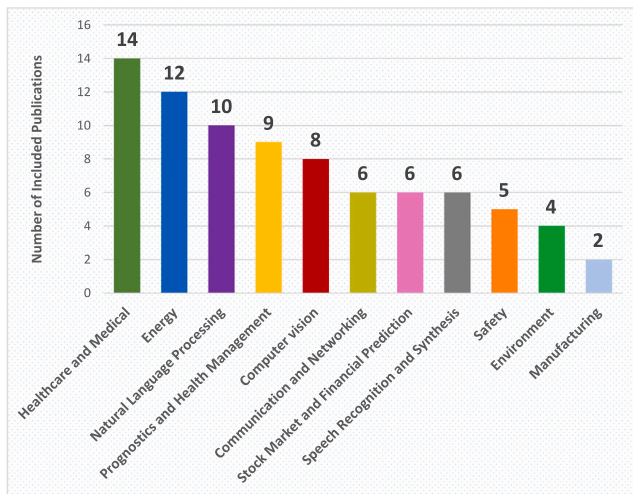


Fig. 8. Distribution of included studies by domain.

diagnose and classify arrhythmia, predict diseases, reconstruct images, and model emotional signals using EEG data. It also aids in genetic disorder prediction, RNA localization, and skin disease diagnosis.

Jyotishi and Dandapat (2022) introduced an ECG biometric system that utilizes a hierarchical LSTM with an AM to effectively capture multiscale information and improve diagnostic performance. Notably, it overcomes the limitation of requiring a large duration of enrollment data, making it more feasible for real-time applications.

The study by Kim et al. (2022) proposed an automatic classification system for an arrhythmia that combines ResNet with a BiLSTM network in ECG signals. This approach proves to be effective in detecting and classifying various cardiac arrhythmias. However, a lightweight model is needed due to computational complexity and training time limitations in practical clinical settings.

Lin et al. (2020) introduced a multimodal interlocutor-modulated attentional BiLSTM model designed for classifying autism subgroups during clinical interviews. By using multimodal data from speech and motion modalities, this framework provides insights into understanding autism spectrum disorders (ASD). Nevertheless, the dataset used only includes 60 ASD subjects, which may not be representative of the entire ASD population.

COVID-19 studies also have been using LSTM networks. Research by Masum et al. (2020) focused on time series forecasting for COVID-19 confirmed cases and introduced the Reproducible-LSTM (r-LSTM) framework. The r-LSTM model generates more precise forecasts compared to traditional ARIMA models. Dey et al. (2022) proposed a fuzzy ensemble model, CovidConvLSTM, for the detection of COVID-19 from chest X-ray images. The model combines ConvLSTM and SE blocks to enhance feature representation and weight allocation, resulting in improved diagnostic accuracy.

Sarvari and Sridevi (2023) presented an optimized EBRSA-BiLSTM model for reconstructing high-quality CT images from limited and under-sampled data. The proposed approach achieves significant improvements in image reconstruction quality. However, The model is computationally expensive and requires a large dataset to be trained.

The study by Chen et al. (2022) focused on cardiovascular disease prediction and introduced an R-Lookahead-LSTM. The proposed architecture outperforms traditional LSTM models by effectively capturing short- and long-term temporal dependencies in time series data.

An LSTM model-based brain-computer interface (BCI) for gait decoding from EEG signals was proposed by Tortora et al. (2020). The model successfully decodes gait patterns from EEG data, offering new possibilities for motor function restoration and rehabilitation.

A hybrid model by Nandhini and Tamilpavai (2022) combined CNN, LSTM, and MWHHO techniques for the prediction of genome sequences in genetic disorders. By integrating the strengths of both CNN and LSTM architectures, the model effectively captures spatial and temporal dependencies in genetic sequences, contributing to advancements in genetic disorder prediction.

Stacked LSTM architecture used by Wang et al. (2018) focused on protein self-interactions prediction. By leveraging protein sequence information, the proposed model effectively resists data skew and achieves good accuracy. However, it requires a large amount of training data.

Asim et al. (2022) proposed EL-RMLocNet, an explainable LSTM network, for RNA-associated multi-compartment localization prediction. By incorporating GeneticSeq2Vec representation learning and AM, the model enhances predictive performance and interpretability.

The Arrhythmia segmentation and classification model using BiLSTM was proposed by Ratnaparkhi et al. (2021). By imposing the bidirectional nature of the LSTM architecture, the model achieves a high classification accuracy of 99.54%, outperforming traditional approaches.

A hybrid model, HSBOS-LSTM, by Elashiri et al. (2022) utilized a modified LSTM architecture and HSBOS algorithm. The proposed model surpasses conventional methods in skin disease classification, achieving higher accuracy by imposing an ensemble of DL models for feature extraction.

Lastly, using surface electromyography (sEMG), Bao et al. (2021) presented a hybrid model that combined CNN and LSTM for the estimation of wrist kinematics. By combining the strengths of CNN and LSTM, the proposed model enables a more accurate estimation of wrist kinematics. However, the model has performance degradation in the presence of noisy or low-quality sEMG signals.

Table 6 summarizes the current LSTM studies categorized into Healthcare and Medical domains.

#### 5.1.2. Energy domain

LSTM networks have proven to be valuable tools in the Energy domain, offering various applications such as forecasting or predicting electricity load, solar power, wind power, and energy price. By accurately predicting future electricity demand, LSTM networks help in optimizing resource management. Similarly, forecasting the output of solar and wind power plants assists businesses and individuals in planning their energy consumption and aids utilities in integrating renewable energy into the grid. Additionally, LSTM networks can forecast energy commodity prices, empowering businesses and individuals to make well-informed decisions regarding their energy purchases.

First, a study by Bian et al. (2021) developed the PSO-AM-LSTM model for abnormal detection of electricity consumption. The model used LSTM's time series fitting ability, incorporates an attention mechanism to enhance relevant information, suppresses irrelevant information, and optimizes the hyperparameters using PSO. Yet, their model performs poorly on "burr" points due to noise and randomness and lacks consideration of practical application issues.

A hybrid model, ISCOA-LSTM, by Somu et al. (2020) combines Improved Sine Cosine Optimization and LSTM networks for accurate building energy consumption forecasting. This model surpasses existing forecast models by delivering stable and precise results, enabling effective energy management and resource allocation. However, it suffers from (i) A lack of analysis on the impact of attributes on power consumption, (ii) Neglecting preprocessing techniques for real-time data noise, glitches, and aging factors in sensor components, and (iii) Failure to determine the appropriate time slot for hyperparameter re-tuning based on real-time data characteristics.

In a case study of building cooling, Dong et al. (2022) introduces a Short-term building cooling load prediction model based on the DwdAdam-ILSTM algorithm. The model utilizes a decoupled weight

**Table 6**

LSTM current studies categorized in the Healthcare and Medical domain.

| Publication                    | Application                                                           | Architecture                            | Dataset                                                                                            | Evaluation metrics                                                      |
|--------------------------------|-----------------------------------------------------------------------|-----------------------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Jyotishi and Dandapat (2022)   | An ECG Bio-metric System                                              | Hierarchical LSTM                       | PTB ECG, ECG-ID, CYBHI, UofTDB                                                                     | EER, Accuracy, F1-score, Kappa score, AUC                               |
| Kim et al. (2022)              | Automatic Cardiac Arrhythmia Classification                           | ResNet, the SE block, and the BiLSTM    | MITDB, AFDB, CinC DB                                                                               | Precision, Recall, F1-score, Specificity, G mean                        |
| Lin et al. (2020)              | Classifying Autism Subgroups During Clinical Interviews               | Multimodal IM-aBiLSTM                   | ADOS Audio-Video Database                                                                          | UAR                                                                     |
| Masum et al. (2020)            | Forecasting For Covid-19 Confirmed Cases                              | Reproducible-LSTM (r-LSTM)              | Daily cumulative confirmed cases of COVID-19 in the US                                             | MAPE, RMSE, MAE                                                         |
| Dey et al. (2022)              | Covid-19 Detection From Chest X-rays                                  | ConvLSTM                                | Three public datasets: Kaggle pneumonia+IEEE covid dataset, COVID-19 Radiography, and COVIDx CXR-3 | Accuracy, Precision, Recall, F1-score                                   |
| Sarvari and Sridevi (2023)     | CT Image Reconstruction                                               | EBRSA-BiLSTM                            | COVID-CT, SARS-CoV-2 CT                                                                            | RMSE, EPI, AMBE, SI, PSNR, SSIM, JND, DE                                |
| Chen et al. (2022)             | Disease Prediction                                                    | R-Lookahead-LSTM                        | cardiovascular disease data set from Kaggle                                                        | Accuracy, Precision, Recall, F1-score, Specificity, MCC Value           |
| Tortora et al. (2020)          | Gait Decoding From EEG                                                | LSTM                                    | EEG recordings from 11 healthy subjects walking on a treadmill                                     | AUC, Accuracy, TPR, TNR, F-Measure                                      |
| Nandhini and Tamilpavai (2022) | Prediction Of Genome Sequences For Genetic Disorders                  | CNN-LSTM based MWHHO                    | UBE3A gene, SARS-COV-2data (NCBI), ENCPP                                                           | Accuracy, Specificity, Precision, Recall, F1-score                      |
| Wang et al. (2018)             | Prediction Of Protein Self-Interactions                               | ZM-SLSTM (Zernike moments-Stacked LSTM) | S.erevisiae and human SIPs datasets                                                                | Accuracy, TPR, PPV, specificity, MCC                                    |
| Asim et al. (2022)             | Ribonucleic Acid-Associated Multi-Compartment Localization Prediction | EL-RMLocNet (Explainable LSTM network)  | miRNA, mRNA, snoRNA, lncRNA, miRNA, mRNA, snoRNA, lncRNA                                           | Accuracy, Precision, Coverage, Ranking Loss, One Error                  |
| Ratnaparkhi et al. (2021)      | Segmentation And Classification Of Arrhythmia                         | BiLSTM                                  | MIT BIH Arrhythmia                                                                                 | Accuracy, Recall, Precision, Specificity, F1-score                      |
| Elashiri et al. (2022)         | Skin Disease Classification                                           | HSBSO-LSTM                              | PH2, Skin Cancer MNIST: HAM10000                                                                   | FPR, Specificity, Recall, F1-score, FRR, NPV, MCC Value, FDR, Precision |
| Bao et al. (2021)              | Wrist Kinematics Estimation                                           | CNN-LSTM                                | Recruitment of six healthy participants                                                            | $R^2$                                                                   |

decay Adam optimization approach, to accurately predict the cooling load of the building in the short term.

Micro-grid load forecasting using an improved LSTM network is proposed by Huang et al. (2022b). The LSTM architecture captures temporal dependencies and accurately predicts future load demands for micro-grid systems, contributing to effective load management in the Energy domain. However, the study relies on a local dataset, which may limit its generalizability.

Precise load forecasting plays a critical role in ensuring the secure and dependable operation of power systems, optimizing power generation, and preventing the unnecessary waste of power resources. Hence, numerous studies have been proposed. A study by Wang et al. (2019) combines AM to highlight important variables, rolling update (RU) for real-time updates, and BiLSTM for training. Still, the RU technique can introduce errors if the data is not updated frequently enough. Another study by Dai et al. (2022) improved the BiLSTM model by combining it with XGBoost and AM. The strengths of three different machine learning models result in improved prediction accuracy and enable efficient load forecasting for energy planning and management.

Photovoltaic (PV) power forecasting is also a common application in the Energy domain. Newly built PV power plants often lack historical data, making it difficult to train an accurate forecasting model. Hence, Zhou et al. (2020) employed transfer learning in LSTM networks. The model, SOL-LSTM, leverages shared optimized layers to

improve the accuracy of power prediction. Wang et al. (2020) proposed an OVMD-IPSO-LSTM model combining Variational Mode Decomposition (VMD), improved PSO, and LSTM to address the challenges of accuracy for PV power, which are intermittent and fluctuating. The method outperforms other traditional methods in terms of prediction accuracy, and optimizing LSTM parameters for reliable PV power forecasting.

Wind farms play a crucial role in the Energy domain and accurate wind speed prediction facilitates better energy dispatch and grid stability. Altan et al. (2021) proposed a hybrid model, ICEEMDAN-LSTM-GWO, combining LSTM, ICEEMDAN decomposition methods, and the GWO algorithm. Liang et al. (2021) combined BiLSTM, MOOFADA optimization algorithm, and transfer learning. The model achieves higher prediction accuracy, strong adaptability, and reduced training time. Liang et al. (2022) proposed a multi-variable Capsnet-BiLSTM-MOHHO model. The model considers multiple meteorological factors and incorporates CapsNet, the MOHHO optimization algorithm, and transfer learning. However, the research solely examines the impacts of two standard climatic conditions on the prediction of wind speed. Finally, Zhang et al. (2019b) proposed SWLSTM-GPR by sharing the weights of LSTM networks and utilizing Gaussian Process Regression (GPR). The model retains the functionality of LSTM while reducing training time through weight sharing. Additionally, the use of GPR for prediction allows the model to acquire a dependable probability distribution function.

**Table 7**  
LSTM current studies categorized in the Energy domain.

| Publication          | Application                                   | Architecture                          | Dataset                                                                   | Evaluation metrics                                                                               |
|----------------------|-----------------------------------------------|---------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Bian et al. (2021)   | Abnormal Detection of Electricity Consumption | PSO-AM-LSTM                           | The electricity dataset of the UMass                                      | RMSE, MAE, MAPE, AE, M, PR, FPR                                                                  |
| Somu et al. (2020)   | Energy Consumption Forecasting                | ISCOA-LSTM                            | KReSIT, IIT Bombay                                                        | MAE, MAPE, MSE, RMSE, Theil's U1, Theil's U2                                                     |
| Dong et al. (2022)   | Short-Term Building Cooling Load Prediction   | DwdAdam-ILSTM                         | Hourly cooling load data from a large commercial building in Xi'an, China | MAE, MAPE, MSE, variation coefficient of root mean square error CV-RMSE, and relative errors E-R |
| Huang et al. (2022b) | Micro-grid Load Forecasting                   | LSTM                                  | Local micro-grid in Zhejiang, China                                       | MAPE, RMSE                                                                                       |
| Wang et al. (2019)   | Short-Term Load Forecasting                   | BiLSTM based on AM and RU             | New South Wales (NSW) and the Victoria (VIC) load data in Australia.      | MAPE, RMSE                                                                                       |
| Dai et al. (2022)    | Short-Term Power Load Prediction              | AM-BiLSTM + XGBoost                   | Two datasets of power load data from Singapore and Norway                 | MAPE, MAE, RMSE                                                                                  |
| Zhou et al. (2020)   | Photovoltaic Power Forecasting                | SOL-LSTM (Share-optimized-layer LSTM) | 6 months data from PV power system in southwest China                     | APE, MAPE, RMSE                                                                                  |
| Wang et al. (2020)   | Short-Term PV Power Prediction                | OVMD-IPSO-LSTM                        | The PV power data from the Changzhou power networks in China              | $R^2$ , MRE, MAPE                                                                                |
| Altan et al. (2021)  | Wind Speed Forecasting                        | LSTM, ICEEMDAN decomposition, and GWO | Five wind farms in the Marmara region, Turkey                             | MAE, MAPE, RMSE                                                                                  |
| Liang et al. (2021)  | Wind Speed Prediction                         | MOOFADA-BiLSTM                        | Historical data from four wind farms in Hebei, China                      | MAE, RMSE, SSE, MAPE                                                                             |
| Liang et al. (2022)  | Wind Speed Prediction                         | multi-variable Capsnet-BiLSTM-MOHHO   | Data obtained from WPCCC in Hebei, China                                  | SSE, MAE, RMSE, IA, PCC, Theil's U1                                                              |
| Zhang et al. (2019b) | Wind Speed Prediction                         | SWLSTM-GPR                            | Four different datasets recorded from wind farms in Mongolia, China       | RMSE, $R^2$ , CP, MWP, CRPS, PIT                                                                 |

Table 7 summarizes the current LSTM studies categorized in the Energy domain.

### 5.1.3. Natural Language Processing (NLP) domain

LSTM networks have revolutionized the domain of NLP. Their abilities to capture long-term dependencies and effectively model sequential data have made them indispensable tools for a wide range of NLP applications. For instance, LSTMs can be used to learn the sentiment of a text by considering the entire sequence of words in the text. Also, to classify text into different categories, such as news articles, and movie reviews. In addition, LSTMs are mostly used in question-answering systems, named entity recognition, sarcasm identification, and script identification in natural scene image and video frames. With LSTM's ability to understand and analyze the contextual information in texts, they have become a fundamental component in the advancement of NLP techniques.

In sentiment analysis applications, Sangeetha and Kumaran (2023) proposed a hybrid optimization algorithm using BiLSTM structure. The THHO-BiLSTM model significantly improves sentiment analysis accuracy by 96.93%. Huang et al. (2022a) introduced the AEC-LSTM model which utilizes emotion intelligence to improve the feature learning ability of the LSTM network. Lastly, Shuang et al. (2019) introduced AE-DLSTMs and AELA-DLSTMs, two novel models for aspect-level sentiment classification. These models incorporate AM and location awareness into the double LSTM architecture, resulting in significant improvements over baseline models.

In text classification applications, Singh et al. (2022) proposed the MOGA-ELSTM model, an evolving LSTM network-based approach that achieves a significant 4.23% accuracy boost compared to traditional LSTM networks.

In named entity recognition (NER) applications, Wei et al. (2019) proposed an AM-BiLSTM-CRF model in biomedical texts. It enhances

the ability to extract significant information without relying on complex feature engineering. Yet, the proposed model does not consider knowledge-based, character-level, and part-of-speech features. Another study by An et al. (2022) addressed Chinese clinical NER by integrating multi-head self-attention with a BiLSTM-CRF architecture. Their MUSA-BiLSTM-CRF model effectively captures diverse semantic features and the interdependence among characters in clinical text. Finally, Gao et al. (2020) CNN-BiLSTM-CRF model focused on HAZOP (Hazard and Operability) text NER. It can automatically learn character-level representations and capture sentence-level features without relying on manual feature engineering. However, the model still struggles with identifying entities that have ambiguous or unclear boundaries.

Bi et al. (2021) proposed the BiLSTM-SRLP model, which combines a BiLSTM with a semantic positional AM for question-answering systems. By emphasizing sentence components and position information, the model effectively bridges lexical gaps and enhances performance in extracting answers from textual data.

Onan and Toçoğlu (2021) introduced a term-weighted model and a three-layer stacked BiLSTM framework for sarcasm identification. The proposed framework demonstrates its potential usefulness in real-world applications where detecting sarcasm plays a crucial role.

Finally, a script identification application by Bhunia et al. (2019) in natural scene images and videos combined AM-CNN-LSTM. The proposed method can effectively handle challenges such as complex backgrounds, distortion, and low image resolution.

Table 8 summarizes the current LSTM studies categorized in the Natural Language Processing domain.

### 5.1.4. Prognostics and Health Management (PHM) domain

LSTMs have emerged as powerful networks in the PHM domain's applications. PHM aims to predict and monitor the health and performance of various systems and components to enable proactive maintenance and reduce downtime. LSTM's ability to capture temporal



**Table 8**

LSTM current studies categorized in the Natural Language Processing domain.

| Publication                  | Application                                                   | Architecture               | Dataset                                                                                 | Evaluation metrics                                 |
|------------------------------|---------------------------------------------------------------|----------------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------|
| Sangeetha and Kumaran (2023) | Sentiment Analysis                                            | THHO-BiLSTM                | Reviews from Amazon's products and Taboada corpus datasets                              | Accuracy, Precision, Recall, F1-score, specificity |
| Huang et al. (2022a)         | Sentiment Analysis                                            | AEC-LSTM                   | IMDB, Yelp201, JDReview, SinaWeibo                                                      | Accuracy                                           |
| Shuang et al. (2019)         | Aspect-Level Sentiment Classification                         | AELA-DLSTMs                | SemEval-2014 Task 4 English, Weibo Chinese dataset                                      | Accuracy                                           |
| Singh et al. (2022)          | Text Classification                                           | MOGA-ELSTM                 | Factory reports dataset                                                                 | Accuracy                                           |
| Wei et al. (2019)            | Biomedical Named Entity Recognition                           | AM-BiLSTM-CRF              | JNLPBA corpus                                                                           | Precision, Recall, F1-score                        |
| An et al. (2022)             | Chinese Clinical Named Entity Recognition                     | MUSA-BiLSTM-CRF            | CCKS2017-CNER, CCKS2018-CNER                                                            | Precision, Recalls, F1-score                       |
| Gao et al. (2020)            | HAZOP Text Named Entity Recognition                           | CNN-Bilstm-CRF             | HAZOP analysis report                                                                   | Accuracy, Recall, F1-score                         |
| Bi et al. (2021)             | Question Answering System                                     | BLSTM-SRLP                 | Food Safety Question and Answer                                                         | MRR, MAP                                           |
| Onan and Toçoğlu (2021)      | Sarcasm Identification                                        | Three-layer stacked BiLSTM | Sarcasm Corpus 1, Sarcasm Corpus 2, and The News headline dataset for Sarcasm detection | Accuracy                                           |
| Bhunja et al. (2019)         | Script Identification In Natural Scene Image And Video Frames | AM-CNN-LSTM                | Four public script identification datasets: ICDAR-17, CVSI2015, MLe2e, and SIW-13       | Accuracy                                           |

dependencies and handle sequential data makes it well-suited for a wide range of PHM applications. Whether it is predicting aircraft engine vibrations, forecasting degradation trends of units, prognosticating lithium-ion battery performance, or estimating the remaining useful life of engines. LSTM offers a powerful and effective solution for accurate and timely prognostics.

ElSaid et al. (2018) utilized the ACO algorithm to automatically optimize the LSTM networks to enhance the predictive capability of turbine engine vibration. By doing so, the performance of the LSTM networks is significantly improved, and the training time is reduced.

In the study Shi et al. (2021), the authors proposed a constrained LSTM network for predicting flight trajectories. This architecture utilizes a 4-D spatial-temporal trajectory set, enabling efficient prediction of aircraft trajectories. However, there could be limitations in handling highly complex or unpredictable flight scenarios.

RUL estimation of lithium-ion batteries is another common application of PHM. Liu et al. (2019b) applied multiple LSTM models through Bayesian Model Averaging (BMA). Additionally, an online iterated training strategy for the BMA algorithm is introduced, allowing for continuous improvement and adaptation to changing battery conditions. Another study by Zhang et al. (2021a) combined an adaptive levy flight-optimized particle filter (ALF-PF) with the LSTM network. This approach outperforms some currently popular particle filter-based algorithms. However, the proposed approach increases computational costs.

RUL of aero-engines is also another common application of PHM. Chen et al. (2021) proposed a model to quantify the uncertainties in RUL prediction using many techniques including BiLSTM. This consolidated model provides more reasonable and more trustworthy maintenance decisions based on the predicted RUL outputs. Cheng et al. (2021) proposed the ensemble LSTM model that uses LSTMNNs trained on different historical data sets. Bayesian inference is used to combine the LSTMNN predictions, resulting in optimal RUL estimation. Yet, only the data sets under single working conditions are validated.

To extract nonlinear dynamic latent information from both process variables and quality variables, a supervised BiLSTM architecture Lui et al. (2022) for data-driven dynamic soft sensor modeling is proposed.

This architecture is highly effective in modeling complex systems, resulting in improved accuracy in soft sensor modeling applications.

Chia et al. (2021) applied their work for predictive maintenance by proposing a two-phase switching optimization using SDAGD with momentum to optimize and accelerate the convergence of the LSTM model. This optimization reduces both testing and training error rates.

Lastly, Ma et al. (2021) built their architecture, IDRSN-PSO-LSTM, by utilizing MOPSO and random walk strategy to improve the gas prediction accuracy in transformer oil.

Table 9 summarizes the current LSTM studies categorized in the Prognostics and Health Management domain.

#### 5.1.5. Computer Vision domain

LSTMs are also used in the Computer Vision domain for actions and gesture recognition and classification. This domain includes numerous applications, such as human pose identification, anomalous behaviors detection in crowds, hyperspectral image classification, image captioning, and generating text descriptions of videos. Usually, CNNs and LSTMs networks are combined together in these applications. CNN for feature extraction which is then fed into LSTM to generate the results.

Human poses, gestures, and action recognition applications in the Computer Vision domain are supported by LSTM capabilities to remember long-term dependencies. For instance, Yang et al. (2019) utilized a weighted convolutional autoencoder in combination with LSTM to effectively capture anomalous patterns in crowded scenes. Rehman et al. (2022) focused their study on hand gesture recognition. A 3D-CNN with LSTM network architecture is built for feature extraction and gesture recognition. This proposed architecture is a lightweight architecture, with about 3.7 million training parameters, with high accuracy results. Lastly, Tan et al. (2022) used the temporal dense sampling to partition videos of human actions into partitions and then let the BiLSTMs, with adaptive fusion, encode the long-term spatial and temporal dependencies in both forward and backward directions. This model achieved an accuracy of 94.78% and 70.72% respectively on the used datasets.

Xue et al. (2019) proposed an ST-HConvLSTM network for action recognition in videos. The spatial-temporal attention module can determine which video segmentation is more informative for action

**Table 9**

LSTM current studies categorized in the Prognostics and Health Management domain.

| Publication          | Application                                       | Architecture                             | Dataset                                        | Evaluation metrics       |
|----------------------|---------------------------------------------------|------------------------------------------|------------------------------------------------|--------------------------|
| ElSaid et al. (2018) | Aircraft Engine Vibrations Prediction             | ACO-LSTM                                 | Data from aircraft flight data recorder        | MSE, MAE                 |
| Shi et al. (2021)    | Flight Trajectory Prediction                      | cLSTM (constrained LSTM)                 | Round trip data obtained by ADS-B              | MAE, MRE, RMSE, DTW      |
| Liu et al. (2019b)   | Lithium-Ion Battery Prognostics                   | BMA-LSTM (Bayesian model averaging-LSTM) | CALCE datasets from the University of Maryland | ERUL                     |
| Zhang et al. (2021a) | Lithium-Ion Batteries RUL Prediction              | ALF-PF-LSTM                              | PCoE, HUST                                     | RMSE, MAPE, $R^2$ , ERUL |
| Chen et al. (2021)   | Prediction Interval Estimation of Aero-engine RUL | BiLSTM                                   | C-MAPSS                                        | PICP, PINAW, CWC         |
| Cheng et al. (2021)  | RUL Prognosis                                     | ELSTMNN (Ensemble LSTM)                  | C-MAPSS                                        | RMSE, Mscore             |
| Lui et al. (2022)    | Data-Driven Dynamic Soft Sensor Modeling          | SBiLSTM (Supervised BiLSTM)              | WWTP                                           | RMSE, $R^2$              |
| Chia et al. (2021)   | Predictive Maintenance                            | SDAGD-LSTM                               | Machinery Fault Database                       | MSE                      |
| Ma et al. (2021)     | Transformer Fault Prediction                      | IDRSN-PSO-LSTM                           | IEC TC10                                       | MAPE, RMSE               |

recognition, while the HConvLSTM module can represent the spatial and temporal structures of actions.

Xue et al. (2019) introduced an ST-HConvLSTM model designed for videos action recognition. The model incorporates a spatial-temporal attention module to identify which video segments carry more informative cues. Additionally, the HConvLSTM module is used to represent the spatial and temporal structures of actions, enhancing the model's ability to recognize and understand actions in the video.

Hyperspectral image classification is another application in the Computer Vision domain. Mei et al. (2022) proposed a spatial-spectral AM with a BiLSTM model that outperforms existing RNN-based classifiers in terms of classification performance. Yet, it may be computationally expensive for large-scale HSIs.

Dai et al. (2020) proposed a hidden-layer LSTM and employed grow-and-prune training to repeatedly adjust the hidden layers of the architecture to achieve higher accuracy with fewer external stacked layers. This compact design makes it suitable for resource-constrained applications, such as image captioning, and speech recognition.

Shi et al. (2020) presented the GOA improved LSTM architecture for decision-making in self-driving vehicles. Surrounding vehicle information were extracted by three parallel LSTM, classification tasks by SVM, and network optimization by GOA.

Lastly, in moving vehicle classification, Mohine et al. (2022) introduced 1D CNN-BiLSTM. The 1D CNN is used to extract features from the acoustic signals, and the BiLSTM is used to classify the vehicles. This approach outperforms conventional classifiers (SVM, ANN, CNN, and CNN-LSTM) with higher accuracy 92% and a lower misclassification rate (0.08).

Table 10 summarizes the current LSTM studies categorized in the Computer Vision domain.

#### 5.1.6. Communication and Networking domain

In recent years, the use of LSTM models has gained significant attention in the field of Communication and Networking. LSTM offers powerful capabilities in capturing and analyzing sequential data, making it well-suited for various applications in this domain. From channel estimation and fault detection to positioning systems and intrusion detection, LSTM has been successfully applied by researchers to tackle diverse challenges. These applications leverage LSTM's ability to handle temporal dependencies and learn patterns from complex data, ultimately leading to improved performance, reliability, and security in communication and networking systems.

For instance, a study by Dash and Thampy (2023) focuses on utilizing LSTM in channel estimation for MIMO communications in

5G networks. The proposed RNN-LSTM network improves channel estimation accuracy and reduces computational complexity compared to conventional methods. Their motivation was that channel estimation is one of the major challenges in MIMO systems, and existing methods have limitations in terms of accuracy and complexity.

Indoor positioning by Zhang et al. (2020) is another application where LSTM demonstrates its powerful capabilities. By leveraging the Channel State Information (CSI) and incorporating features optimization mechanisms, LSTM-based methods offer accurate indoor positioning, improving location-based services and navigation systems. However, this is only tested in a laboratory environment and may not be suitable for all indoor environments.

In the field of intrusion detection, deep learning IDS by Sun et al. (2020b) presents a hybrid model of CNN-LSTM to automatically extract spatial and temporal features from network traffic for improved intrusion detection. The proposed model achieves an overall accuracy of 98.67% and an accuracy above 99.50% for each attack type. However, the authors acknowledged that it may have lower detection accuracy for Heartbleed and SSH-Patator attacks due to limited data availability, which can be mitigated by incorporating GANs and traditional traffic features.

Time series prediction is yet another area where LSTM shines. Fu et al. (2022) have proposed a temporal self-attention-based Conv-LSTM network to predict multivariate time series accurately compared with six baseline models. In contrast, (1) Full parallelization of the model is not achievable due to its heavy reliance on LSTM units. (2) The model's capacity to extract spatial correlation decreases as the spatial span between sequences increases.

Addressing the challenge of occasional large time delays in Ultra-Reliable Low-Latency Communications (URLLC), Liu and Sheng (2022) propose an approach based on unbalanced regression algorithms and LSTM. This model predicts and mitigates the impact of large time delays, ensuring the required reliability and low latency for critical communications.

Lastly, LSTM-based wireless intrusion detection systems have been optimized using stacked auto-encoder networks by Karthic and Kumar (2022). By improving the performance of LSTM, these systems enhance the detection of intrusions and contribute to maintaining the security of wireless networks. Still, a limited set of attack types is considered, which may not be capable of detecting all possible types of attacks that can target a wireless network.

Overall, LSTM models have gained attention in the Communication and Networking domain, excelling in channel estimation, fault detection, positioning, and intrusion detection. Their ability to handle temporal dependencies and learn patterns improves system performance, reliability, and security.

**Table 10**

LSTM current studies categorized in the Computer Vision domain.

| Publication          | Application                                                                       | Architecture                                        | Dataset                                                                 | Evaluation metrics                                                                |
|----------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Yang et al. (2019)   | Anomalous Behaviors Detection In Moving Crowds                                    | WCAE-LSTM (Weighted convolutional autoencoder-LSTM) | CUHK Avenue and UCSD pedestrian                                         | AUC, EER                                                                          |
| Rehman et al. (2022) | Hand Gesture Recognition                                                          | 3D-CNN with LSTM                                    | 20BN-jester                                                             | Accuracy, Precision, Recall                                                       |
| Tan et al. (2022)    | Human Action Recognition                                                          | TDS-BiLSTM                                          | UCF101 and HMDB51                                                       | Accuracy                                                                          |
| Xue et al. (2019)    | Action Recognition In Videos                                                      | ST-HConvLSTM                                        | UCF101, HMDB51 and Kinetics datasets                                    | Accuracy                                                                          |
| Mei et al. (2022)    | Hyper-spectral Image Classification                                               | BiLSTM                                              | Three HSI datasets: Salinas Valley, Pavia Centre, and Pavia University. | per-class Accuracy, Overall Accuracy (OA), average Accuracy (AA), and Kappa score |
| Dai et al. (2020)    | Speech Recognition, Image Captioning, and Neural Machine Translation Applications | H-LSTM (Hidden-layer LSTM)                          | MSCOCO, AN4, and IWSLT 2014 German-English dataset                      | CIDEr-D score, WER (word error rate), and BLEU (Bilingual Evaluation Understudy)  |
| Shi et al. (2020)    | Decision-Making For Self-Driving Vehicles                                         | GOA-LSTM                                            | NGSIM (Next Generation SIMulation)                                      | Accuracy                                                                          |
| Mohine et al. (2022) | Moving Vehicle Classification                                                     | 1D CNN-BiLSTM                                       | SITEX02                                                                 | Accuracy, Precision, Recall, F1-score, AUC                                        |

**Table 11**

LSTM current studies categorized in the Communication and Networking domain.

| Publication              | Application                                              | Architecture                                         | Dataset                                                                         | Evaluation metrics                                 |
|--------------------------|----------------------------------------------------------|------------------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------|
| Dash and Thamby (2023)   | Channel Estimation For MIMO Communications In 5G Network | PSO-Adam optimizer based RNN-LSTM                    | Historical channel responses of pilot block                                     | BER, MSE                                           |
| Zhang et al. (2020)      | Indoor positioning                                       | LSTM                                                 | Raw CSI data collected from multiple access points in a laboratory environment. | MSE, MAE, hinge                                    |
| Sun et al. (2020b)       | Intrusion Detection System (IDS)                         | CNN-LSTM                                             | CICIDS2017                                                                      | Accuracy, F1-score, FPR, TPR                       |
| Fu et al. (2022)         | Multivariate Time Series Prediction                      | Temporal Self-Attention-Based DA-Conv-Lstm           | SML2010, NASDAQ100, and one private satellite dataset                           | MAE, RMSE, MAPE                                    |
| Liu and Sheng (2022)     | URLLC Occasional Large Time Delay Prediction             | Unbalanced regression algorithms and LSTM            | Simulated time delay data                                                       | RMSE, MAE, Precision, Recall                       |
| Karthic and Kumar (2022) | Wireless Intrusion Detection                             | OLSTM-SAE (Optimized LSTM with Stacked Auto Encoder) | NSL-KDD and UNSW-NB15                                                           | Accuracy, Precision, Recall, F1-score, G mean, FPR |

Table 11 summarizes the current LSTM studies categorized in the Communication and Networking domain.

#### 5.1.7. Stock Market and Financial Prediction domain

Another field in which LSTMs have proven to be especially dominant is the Stock Market and Financial Prediction where LSTM can be trained with historical data to forecast future prices.

Ravi (2020) proposed FuzzyCSA-based deep LSTM for predicting bitcoin prices. The FuzzyCSA optimizes the LSTM model by determining the optimal weights of the model resulting in a minimal MAE, and RMSE of 0.4811, and 0.3905, respectively. This fusion provides a robust framework for investors and traders in the cryptocurrency market.

An IPSO algorithm applied to the LSTM model via Shao et al. (2019) for nickel price forecasting. Compared with conventional LSTM and ARIMA models, the IPSO algorithm has higher reliability and prediction accuracy. The prediction error is reduced by 9% and 13%, respectively.

The study by Eom and Kim (2023) proposed a method for predicting customers' credit card turnover by combining application logs and credit card payment data. A term weighting and a 2-layer LSTM were used to make the prediction. The proposed model demonstrates superior performance than those using only payment data or time-weighted logs.

In stock market prediction applications, a new model for predicting the Dow Jones index was proposed by Ying et al. (2021). The proposed

model, LSTM-AdaBoost, iteratively updates the weights and integrates multiple weak classifiers. It shows a boost in prediction accuracy when compared to conventional models, showcasing an average rise of 43.76% in the  $R^2$  metric. Ji et al. (2021) proposed a hybrid model, IPSO-LSTM, with a nonlinear method to enhance the inertia weight of the PSO algorithm. Subsequently, the IPSO was applied to optimize the hyperparameters of the LSTM model. The proposed approach surpassed other baseline models, including LSTM, and PSO-LSTM. Finally, another hybrid model, APSO-LSTM, by Kumar et al. (2022) is proposed. The APSO algorithm was employed to evolve the initial weights in various layers of the LSTM network. The performance of the proposed model was better than other models, such as a GA-LSTM, an Elman neural network, and a standard LSTM, in terms of their forecasting capabilities.

Table 12 summarizes the current LSTM studies categorized in the Stock Market and Financial Prediction domain.

#### 5.1.8. Speech Recognition and Synthesis domain

Speech Recognition, referred to as converting spoken words into written text, is the capacity of a machine or computer program to recognize and transcribe speech into a readable format. voice search features, voice dialing, and speech-to-text processing are all applications in the Speech Recognition domain. Also, LSTM networks have been used for

**Table 12**

LSTM current studies categorized in the Stock Market and Financial Prediction domain.

| Publication         | Application                              | Architecture                                        | Dataset                                                                       | Evaluation metrics                  |
|---------------------|------------------------------------------|-----------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------|
| Ravi (2020)         | Bitcoin Prediction                       | FuzzyCSA-based Deep LSTM                            | Two publicly available datasets: Bitcoin dataset, and CM Coin-metrics dataset | RMSE, MAE                           |
| Shao et al. (2019)  | Nickel Price Forecast                    | IPSO-LSTM                                           | London Metal Exchange                                                         | RMSE, MAE, MAPE                     |
| Eom and Kim (2023)  | Predicting Credit Card Customer Turnover | Time-Weighted Cumulative LSTM Method Using Log Data | Three months of data from credit card companies                               | Precision, Recall, F1-score         |
| Ying et al. (2021)  | Prediction Model Of Dow Jones Index      | LSTM-AdaBoost                                       | Historical data of the Dow Jones index                                        | MSE, RMSE, $R^2$ , MAE              |
| Ji et al. (2021)    | Stock Indices Forecast                   | IPSO-LSTM                                           | S&P/ASX200                                                                    | RMSE, MAE, MAPE and $R^2$           |
| Kumar et al. (2022) | Stock Price Time Series Forecasting      | APSO-LSTM                                           | Yahoo Finance (S&P 500, Sensex, and Nifty 50)                                 | MSE, MAAPE, RMSE, SMAPE, Theil's U1 |

emotion recognition, hate speech prediction, and speech intelligibility prediction.

Passricha and Aggarwal (2020) proposed a hybrid architecture, CNN-BiLSTM, combining deep CNN and BiLSTM for automatic speech recognition. This architecture achieves significant improvements over CNN and DNN architecture, with a relative improvement of 5.8% and 10%, respectively.

Hwang et al. (2021) presented a study for emotion recognition using BiLSTM optimized by ACO. The proposed model extracts valid features from both the central nervous system and peripheral nervous system signals, incorporating past and future bio-signal information. This approach improves emotion recognition by finding optimal combinations of features and effectively enhances the temporal dynamics of the input signals.

To take precautions in online social networks, Fazil et al. (2023) proposed a method for predicting hate speech and fake news by using an attentional multi-channel CNN-BiLSTM. This model utilizes multiple channels with different filters and kernel sizes.

Zhang et al. (2021b) developed an acoustic multi-layer model for speech emotion recognition. The model, SCBAMM, has these layers in order: dense, convolutional, skip, mask, BiLSTM, attention, and pooling layer. Since the model can handle spatiotemporal information and identify emotion-related features, it achieves an accuracy rate of 94.58% and 72.50% compared with baseline models.

To analyze netizens' public opinions, a new speech emotion recognition algorithm by Yang et al. (2021) is proposed. The model, PSO-GA-CDBN (PGCDBN) along with BiLSTM, achieved the best results compared with popular classifiers, such as SVM, ResNet, LSTM, and DBN with an accuracy of 98.67%.

Another acoustic model was proposed by Gallardo-Antolín and Montero (2021) for speech intelligibility level classification. The proposed architecture utilizes LSTM to exploit the complementary information provided by acoustic and modulation spectrograms.

Table 13 summarizes the current LSTM studies categorized in Speech Recognition and Synthesis domain.

### 5.1.9. Safety domain

LSTM has been successfully applied in various Safety applications, such as in pharmacovigilance, explosive substance identification, fire detection systems, coal mine safety, and traffic management. These applications demonstrate LSTM's effectiveness in enhancing safety measures.

First of all, Wang et al. (2023) developed a quantum BiLSTM with AM, to accurately detect adverse drug reactions (ADR) from social media big data. This hybridization simplified the structure by removing biases and utilizing weights and active values qubits for weight updates. Results proved the proposed model's potential usage in early ADR identification which is crucial for medication safety. However, the

study did not provide a multi-stage classifier to handle diverse types of data.

Another study by Torres-Tello et al. (2020) aimed to develop a fast, accurate, and lightweight deep learning model for detecting explosive substances using a MOX chemical sensors array. By combining TNT or gunpowder with numerous substances, 140 samples were collected for classification. Also, five machine learning models were evaluated, and the proposed model, LeNet-5-LSTM, achieved 100% accurate compound classification within a very short time.

Another tangible work in the Safety domain is the fire detection application by Xu et al. (2021). The authors proposed a novel method using deep LSTM and VAE and they used simulations and real-world fire datasets for evaluation. Performance is superior compared with standard LSTM, CUSUM, and EWMA.

In short-term traffic flow prediction applications, Zhao et al. (2020) introduced an EnLSTM-WPEO. The authors integrated LSTM ensemble learning, NNCT weight integration, and the PEO algorithm. The evaluation process on six datasets demonstrated the superiority of the proposed model over other popular traffic flow forecasting models. Finally, Wu et al. (2020) proposed a framework with two modules, a preprocessing module, and a prediction module. The missing data were repaired in the preprocessing module, and the prediction results were made by an AM-LSTM. The inclusion of the AM has a significant impact on the performance of the LSTM model, leading to a reduction in error by as much as 12.4%.

Table 14 summarizes the current LSTM studies categorized in the Safety domain.

### 5.1.10. Environment domain

LSTM finds also applications in the Environment domain, such as forecasting PM2.5 concentration levels, estimating the stress-strain behavior of concrete under high temperatures, and predicting effluent Chemical Oxygen Demand (COD) in wastewater.

PM2.5, which is a type of air pollutant, concentration prediction applications play an important role in air pollution prevention and control. Ma et al. (2019) addressed the limitations of existing studies by integrating deep learning to predict air pollution in areas without monitoring stations. Their proposed model is composed of BiLSTM and the IDW technique for spatiotemporal predictions of air pollutants. The BiLSTM captures long-term temporal mechanisms while the IDW layer considers spatial correlation to interpolate air pollution distribution.

Dong et al. (2020) built a new LSTM-based framework for cost index prediction in construction projects. It outperforms other methods, such as SVM, capturing long-distance dependencies and providing accurate short-term predictions.

The authors, Tanhadoust et al. (2023), applied LSTM to accurately predict the stress-strain relationship for lightweight aggregate and normal-weight aggregate concrete after high-temperature exposures. LSTM accurately predicted stress-strain in LWAC and NWAC for strength, elasticity, and failure at high temperatures.



**Table 13**

LSTM current studies categorized in the Speech Recognition and Synthesis domain.

| Publication                         | Application                                 | Architecture                                     | Dataset                                                                                       | Evaluation metrics                      |
|-------------------------------------|---------------------------------------------|--------------------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------|
| Passricha and Aggarwal (2020)       | Automatic Speech Recognition                | CNN-BiLSTM                                       | Medium-sized corpus containing 200,000 Hindi spoken utterances.                               | WER                                     |
| Hwang et al. (2021)                 | Emotion Recognition                         | ACO-BiLSTM                                       | MAHNOB-HCI, the DEAP, and the MERTI-Apps datasets                                             | RMSE                                    |
| Fazil et al. (2023)                 | Hate Speech Prediction                      | Multi-channel attention-aware CNN-BiLSTM         | Three twitter-related datasets, DS-1, DS-2, DS-3                                              | Precision, Recall, F1-score, Accuracy   |
| Zhang et al. (2021b)                | Speech Emotion Recognition                  | SCBAMM (attention-based skip convolution BiLSTM) | EMO-DB and CASIA                                                                              | Accuracy, Precision, UAR, WAR, F1-score |
| Yang et al. (2021)                  | Speech Emotion Recognition                  | PSO-GA-CDBN (PGCDBN)                             | Two datasets: the CASIA Chinese emotional corpus and a self-collected Chinese speech dataset. | FPR, FRR, Accuracy, F1-Score            |
| Gallardo-Antolín and Montero (2021) | Speech Intelligibility Level Classification | LSTM                                             | UA-Speech database which contains dysarthric speech                                           | Accuracy                                |

**Table 14**

LSTM current studies categorized in the Safety domain.

| Publication                | Application                                             | Architecture                  | Dataset                                            | Evaluation metrics                                                               |
|----------------------------|---------------------------------------------------------|-------------------------------|----------------------------------------------------|----------------------------------------------------------------------------------|
| Wang et al. (2023)         | Adverse Drug Reaction Detection From Social Media       | Quantum BiLSTM With Attention | Twimed, and TwitterADR                             | Precision, Recall, F1-score                                                      |
| Torres-Tello et al. (2020) | Detection of Explosives In A MOX Chemical Sensors Array | LeNet5-LSTM                   | A collection of 140 samples                        | Accuracy                                                                         |
| Xu et al. (2021)           | Fire Detection                                          | LSTM-VAE                      | 69 real-world datasets provided by NIST            | Fire Alarm Time Lag, Missed Detection Rate, False Alarm Rate, F1-score, Accuracy |
| Zhao et al. (2020)         | Short-Term Traffic Flow Prediction                      | EnLSTM-WPEO                   | Six real-world traffic flow datasets from DRIVENET | MAE, and RMSE                                                                    |
| Wu et al. (2020)           | Short-Term Traffic Speed Prediction                     | AM-LSTM                       | 30-day traffic speed dataset in Nanshan, China     | MAE, RMSE                                                                        |

**Table 15**

LSTM current studies categorized in the Environment domain.

| Publication              | Application                                                                    | Architecture   | Dataset                                                   | Evaluation metrics     |
|--------------------------|--------------------------------------------------------------------------------|----------------|-----------------------------------------------------------|------------------------|
| Ma et al. (2019)         | PM2.5 Concentration Prediction                                                 | IDW-BiLSTM     | Historical air pollution data from Guangdong, China.      | RMSE, MAE, MAPE        |
| Dong et al. (2020)       | Cost Index Predictions for Construction Engineering                            | LSTM           | Data taken from four domestic datasets in Shenzhen, China | MSE, MAE, MAPE         |
| Tanhadoust et al. (2023) | Predicting Stress-Strain Behavior                                              | LSTM           | Derived from 120 individual experimental results          | $R^2$ , RMSE, MAE      |
| Liu et al. (2021)        | Predicting the Effluent Chemical Oxygen Demand in a Wastewater Treatment Plant | AHMPSO-LSTM-AM | 396 data points collected from a WWTP in China            | RMSE, MAE, MAPE, $R^2$ |

Finally, Liu et al. (2021) proposed a hybrid model, AHMPSO-LSTM-AM, to enhance wastewater treatment plant monitoring capabilities using an LSTM model optimized by AHMPSO with an AM to mine local water quality features to improve the Chemical Oxygen Demand (COD) prediction accuracy. The approach aims to overcome the limitations of traditional monitoring methods in terms of time and cost, enabling more efficient and accurate planning for controlling water pollution.

Table 15 summarizes the current LSTM studies categorized in the Environment domain.

#### 5.1.11. Manufacturing domain

LSTM models also play an important role in Manufacturing domain applications. One notable application in the Manufacturing domain

is medical equipment demand forecasting to optimize the operations of production and supply chain and meet the requirements during healthcare emergencies. By analyzing historical data on factors such as population demographics, healthcare trends, and previous outbreak patterns. LSTM models can provide accurate predictions of these demands. Koç and Türkoğlu (2022) built a deep model using a multilayer LSTM network which includes normalization, deep LSTM architecture, and dropout-dense regression layers. Experimental results demonstrate its potential for estimating future case numbers and equipment demand related to COVID-19.

Additionally, LSTM models have been successfully utilized in predicting the flatness of tandem cold-rolled strips, a critical quality parameter in steel manufacturing. A study by Chen et al. (2023) proposed

**Table 16**  
LSTM current studies categorized in the Manufacturing domain.

| Publication             | Application                                                    | Architecture     | Dataset                                             | Evaluation metrics    |
|-------------------------|----------------------------------------------------------------|------------------|-----------------------------------------------------|-----------------------|
| Koç and Türkoğlu (2022) | Forecasting Of Medical Equipment Demand And Outbreak Spreading | Multi layer LSTM | COVID-19 data, Turkey                               | MAPE, $R^2$           |
| Chen et al. (2023)      | Prediction Of Tandem Cold-Rolled Strip Flatness                | AM-LSTM          | 85,478 sets of data collected from Tandem cold mill | MSE, RMSE, MAE, $R^2$ |

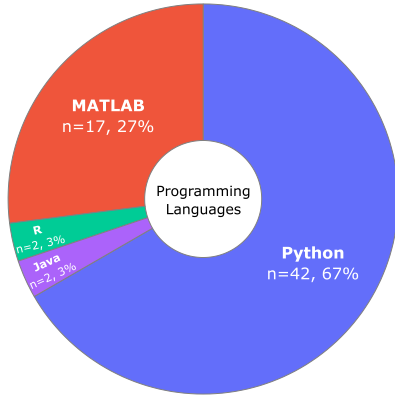


Fig. 9. Preferred programming languages for LSTM development.

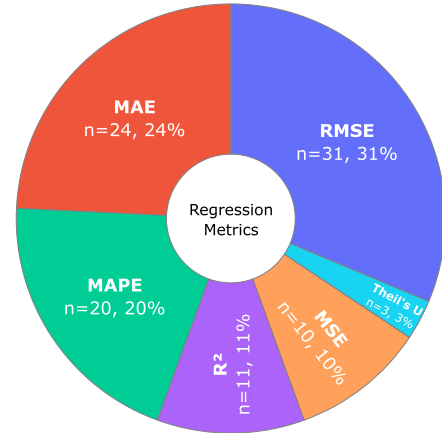


Fig. 10. Most used evaluation metrics in regression problems.

an AM with LSTM architecture that allows manufacturers to implement corrective actions and maintain high-quality standards while manufacturing these items.

Table 16 summarizes the current LSTM studies categorized in the Manufacturing domain.

### 5.2. RQ2: What are the current modeling techniques and tools used to build the LSTM models?

This question aims to find out the current techniques and tools available for LSTM development. Its answer includes two main subsections: (1) The preferred programming languages, frameworks, and libraries (5.2.1), and (2) The most used evaluation metrics in regression and classification problems (5.2.2).

#### 5.2.1. Preferred programming languages and tools in LSTM development

After analyzing the 82 included studies, it was clear that the preferred and most used programming languages for building robust LSTM models are Python (42 studies, 67%) and MATLAB (17 studies, 27%), followed by Java (2 studies, 3%) and R (2 studies, 3%). Fig. 9 illustrates the preferred and most used programming languages in LSTM development.

There are numerous reasons behind Python's prominent and dominant role in machine learning development, especially for LSTM network development. Some of these reasons are:

- 1. Powerful libraries and frameworks:** Python has specialized libraries and frameworks, such as TensorFlow (22 studies), Keras (27 studies), PyTorch (5 studies), and Theano (4 studies), dedicated to machine learning and data science applications. These tools provide researchers and practitioners with powerful and pre-built functions for tasks ranging from data preprocessing to model development and evaluation.
- 2. Strong visualization capabilities:** Python offers a variety of libraries, such as Matplotlib, Seaborn, and Plotly, that allow researchers to easily convert their data into understandable and good-looking visualizations.

### 3. Other reasons: Extensive community support, cross-platform availability, and syntax simplicity.

Additionally, MATLAB also has a significant role in machine learning application development, despite being primarily recognized for numerical computing. Moreover, some researchers used both Python and MATLAB in the same research. While Python was used to build the proposed models, MATLAB was used for features extraction by Hwang et al. (2021), or for data visualization by Bian et al. (2021) in the same experiment. Finally, here are some worth mentioning techniques used less than 5 times in the included papers: Anaconda, CUDA, CUDNN, OpenCV, Caffe, MXNet, and Kaldi. Comprehensive numerical results for all of these techniques can be downloaded from the SLR Excel file in the Data Availability section.

#### 5.2.2. Most used evaluation metrics in LSTM development

Evaluation metrics are used to measure how good the performance of the proposed model is in order to enhance it more or to compare it with other baseline models. Those evaluation metrics can be categorized into one or more categories depending on the associated tasks and what the practitioners need to evaluate in their developed models. From the 82 included studies, 67 distinct evaluation metrics have been identified. However, for the sake of clarity and simplicity, evaluation metrics with 3 or more occurrences in the included papers were only considered. Also, while some metrics can be applicable to multiple groups, they have been categorized according to two main problems in machine learning, regression and classification problems.

##### 1. Regression problems evaluation metrics

Through this SLR, it is clear that RMSE is the most commonly used evaluation metric in regression problems. RMSE has been used in 31 studies, which is about 31% of the total studies, followed by MAE (24 studies, 24%), MAPE (20 studies, 20%),  $R^2$  (11 studies, 11%), MSE (10 studies, 10%), and Theil's U (3 studies, 3%). Fig. 10 illustrates the most used evaluation metrics in regression problems.

The definitions and the equations of these evaluation metrics are as follows:

- (a) **RMSE (Root Mean Square Error):** It measures the distance between the data points and the regression line, and it is represented as:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (8)$$

- (b) **MAE (Mean Absolute Error):** It represents the mean absolute difference between the actual and predicted values in the dataset, and it is denoted as:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (9)$$

- (c) **MAPE (Mean Absolute Percentage Error):** It represents the average percentage error across all data points, and it is denoted as:

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\% \quad (10)$$

- (d)  **$R^2$  (Coefficient of Determination):** It represents the proportion of variance in the dependent variable that can be explained by the independent variable.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (11)$$

- (e) **MSE (Mean Square Error):** It represents the average of the squared difference between the actual and predicted values in the dataset, and it is denoted as:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (12)$$

- (f) **Theil's  $U_1$ :** It compares the RMSE of the model's predictions to the RMSE of a naive model that predicts the mean value.

$$U_1 = \frac{\sqrt{\frac{1}{n} \sum_{i=1}^n \left( \frac{y_i - \hat{y}_i}{y_i} \right)^2}}{\sqrt{\frac{1}{n} \sum_{i=1}^n \left( \frac{y_i - \bar{y}}{y_i} \right)^2}} \quad (13)$$

Where in all these equations:  $n$  represents the number of data points,  $y_i$  denotes the actual value of the  $i$ th data point,  $\hat{y}_i$  represents the predicted value of the  $i$ th data point, and  $\bar{y}$  represents the mean of the actual values.

## 2. Classification problems evaluation metrics

From this SLR, the most commonly used evaluation metric in classification problems is the Accuracy metric. Accuracy has been used in 29 studies, about 29% of the total studies, followed by F1-score (20 studies, 20%), Recall (18 studies, 18%), Precision (18 studies, 18%), Specificity (7 studies, 7%), FPR (5 studies, 5%), and AUC (4 studies, 4%). Fig. 11 illustrates the most used evaluation metrics in classification problems.

The definitions and the equations of these evaluation metrics are as follows, where TP, FP, TN, and FN denote true positive, false positive, true negative, and false negative respectively:

- (a) **Accuracy:** It is the proportion of the predictions that were correct.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (14)$$

- (b) **F1-score:** It is a measure of precision and recall.

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times TP}{2 \times TP + FP + FN} \quad (15)$$

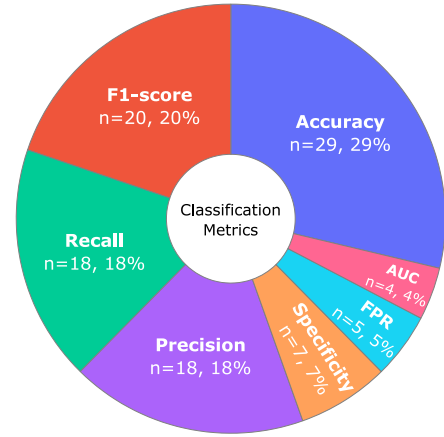


Fig. 11. Most used evaluation metrics in classification problems.

- (c) **Recall:** It is the proportion of actual positives that were correctly predicted as positive.

$$Recall = \frac{TP}{TP + FN} \quad (16)$$

- (d) **Precision:** It is the proportion of positive predictions that were actually positive.

$$Precision = \frac{TP}{TP + FP} \quad (17)$$

- (e) **Specificity:** It is the proportion of actual positives that were correctly predicted as positive.

$$Specificity = \frac{TN}{FP + TN} \quad (18)$$

- (f) **FPR (False Positive Rate):** It is the proportion of negative instances that are incorrectly classified as positive.

$$FPR = \frac{FP}{TN + FP} = 1 - Specificity \quad (19)$$

- (g) **AUC (Area Under the Curve):** It is a measure of the overall performance of a classifier. It is calculated by taking the area under the ROC curve.

## 5.3. RQ3: What are the weights initialization techniques used in LSTM networks?

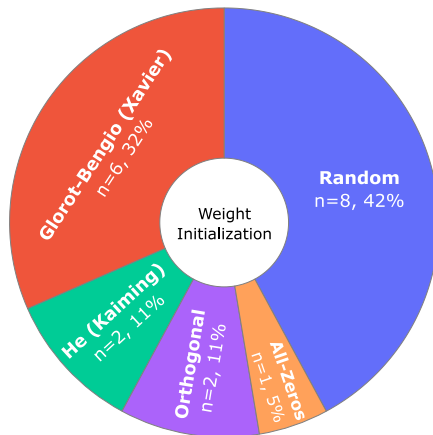
This question aimed to find out what the most used weight initialization techniques in LSTM networks are because weight initialization is a crucial step applied before training DL models. The network's weights are initialized and then optimized repeatedly while training the developed model. This is an extensive and repeated process until the model's loss converges to a minimum loss value and an ideal matrix of weights is obtained. Weights initialization directly affects the convergence of a network (Narkhede et al., 2022; Boulila et al., 2022). Hence, choosing an appropriate weight initialization technique becomes crucial for end-to-end training to avoid both vanishing and exploding gradients problems and accelerate the performance (Narkhede et al., 2022).

There are numerous weights initialization techniques used while building DL applications. Despite the fact that networks' weights initialization is very important and crucial for network convergence and stability, only 19 explicit declarations of weights initialization techniques were reported in the 82 included studies, which were finally categorized into 5 main techniques. The rest of the unreported papers

**Table 17**

Comparison between LSTM weights initialization techniques.

| Technique                                       | Characteristics                                                                                                                                                                                                | Advantages                                                                                                                                          | Disadvantages                                                                                                                                                                    | Research papers                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Random</i>                                   | ❖ Weights are randomly initialized with Uniform or Gaussian distributions.                                                                                                                                     | ❖ Enables exploration of a wide range of weight values, helping to escape local optima.                                                             | ❖ Lack of control over initial weight values can lead to slow convergence or vanishing/exploding gradients.<br><br>❖ Causes the network to become stuck at local points.         | <a href="#">Somu et al. (2020)</a> , <a href="#">Chia et al. (2021)</a> , <a href="#">Wu et al. (2020)</a> , <a href="#">Huang et al. (2022a)</a> , <a href="#">Wang et al. (2019)</a> , <a href="#">Tan et al. (2022)</a> , <a href="#">Dong et al. (2020)</a> , <a href="#">Dai et al. (2020)</a> |
| <i>Xavier/ Glorot (Glorot and Bengio, 2010)</i> | ❖ Designed to better handle vanishing/exploding gradients by controlling the variance of weight initialization based on the input and output dimensions of the layer ( <a href="#">Boulila et al., 2022</a> ). | ❖ Improves the training speed and convergence of deep networks, especially those with differentiable activation functions such as tanh and sigmoid. | ❖ May not be suitable for very deep neural networks when the activation function is non-differentiable.<br>❖ Can still suffer from vanishing/exploding gradients in some cases.  | <a href="#">Passricha and Aggarwal (2020)</a> , <a href="#">Yang et al. (2019)</a> , <a href="#">Gao et al. (2020)</a> , <a href="#">Masum et al. (2020)</a> , <a href="#">Kumar et al. (2022)</a> , <a href="#">Bhunia et al. (2019)</a>                                                           |
| <i>He/ Kaiming (He et al., 2015)</i>            | ❖ Similar to Xavier initialization but optimized for non-differentiable activation functions such as ReLU.                                                                                                     | ❖ Suitable for deeper networks using ReLU activations prevents vanishing gradient problem.                                                          | ❖ May not be suitable for very deep neural networks when the activation function is differentiable. <a href="#">Boulila et al. (2022)</a><br>❖ It solves dying neurons problems. | <a href="#">Gao et al. (2020)</a> , <a href="#">Chen et al. (2023)</a>                                                                                                                                                                                                                              |
| <i>Orthogonal</i>                               | ❖ Initializes weights as an orthogonal matrix.                                                                                                                                                                 | ❖ Helps stabilize the training process and improves gradient flow in recurrent layers.                                                              | ❖ Computationally expensive for large networks.                                                                                                                                  | <a href="#">Masum et al. (2020)</a> , <a href="#">Kumar et al. (2022)</a>                                                                                                                                                                                                                           |
| <i>All-zeros/ Constants</i>                     | ❖ Weights are initialized to be all zeros or constant values.                                                                                                                                                  | ❖ Very simple and easy to use.                                                                                                                      | ❖ Can lead to symmetric weight updates and network symmetry issues.<br>❖ Can lead to slow network convergence.                                                                   | <a href="#">Ying et al. (2021)</a>                                                                                                                                                                                                                                                                  |

**Fig. 12.** Weights initialization techniques in LSTM.

may have used any one of these techniques without explicitly reporting it in their literature.

Among the numerous techniques analyzed, *Random* weights initialization was the most frequently employed technique in the included literature with (N=8, 42%), followed by *Glorot/Xavier* Initialization ([Glorot and Bengio, 2010](#)) with (N=6, 32%). Then, *He/Kaiming* initialization ([He et al., 2015](#)), and *Orthogonal Matrix* initialization both with (N=2, 11%). Finally, *All-zeros* initialization (N=1, 5%) was comparatively less prevalent and not frequently used. [Fig. 12](#) illustrates the most used weights initialization techniques in LSTM models.

[Table 17](#) shows a comparison between the most used weights initialization techniques in LSTM, according to their main characteristics, advantages, disadvantages, and research papers.

#### 5.4. RQ4: What are the weights optimization techniques used in LSTM networks?

Network optimization is one of the core elements in neural network applications ([Sun et al., 2020a](#)). It refers to the continuous procedure of seeking the optimal values for the parameters of the network to minimize its loss and maximize its results ([Mirjalili, 2016b](#)). This question aimed to find out what the most used weight optimization techniques in LSTM networks are. After analyzing the 82 included studies, a total of 37 different optimization algorithms with their variants have been identified. These algorithms are used for network hyper-parameters and parameters optimization including the weights of networks. These algorithms are further categorized into five primary groups. Adaptive learning rate algorithms, gradient descent-based algorithms, metaheuristic algorithms, boosting algorithms, and other algorithms. These five groups will be discussed in detail in Sections 5.4.1, 5.4.2, 5.4.3, 5.4.4, and 5.4.5 respectively.

[Fig. 13](#) reveals interesting insights regarding the most commonly used techniques in LSTM weights optimization as it shows the intensity of these five groups. The obtained results indicate that adaptive learning rate techniques were the most prevalent, with about 59% in the included studies. These algorithms dynamically and adaptively adjust the learning rate during the training process, allowing for more efficient convergence and better performance. Among them, *Adam* algorithm was the dominant one because it is considered the default weight optimization algorithm in most of the DL applications due to its remarkable performance.

Next, metaheuristic techniques, which employ higher-level strategies inspired by natural processes to search for optimal solutions, were utilized in about 21% of the included studies. Their adaptive and explorative nature makes them particularly useful for LSTM network applications. Following closely behind, gradient descent-based techniques were utilized in about 15% of the included studies, emphasizing their significance in optimizing LSTM model weights through iterative updates based on the gradient of the loss function. After that, a small number of studies (2 instances, 2%) explored boosting



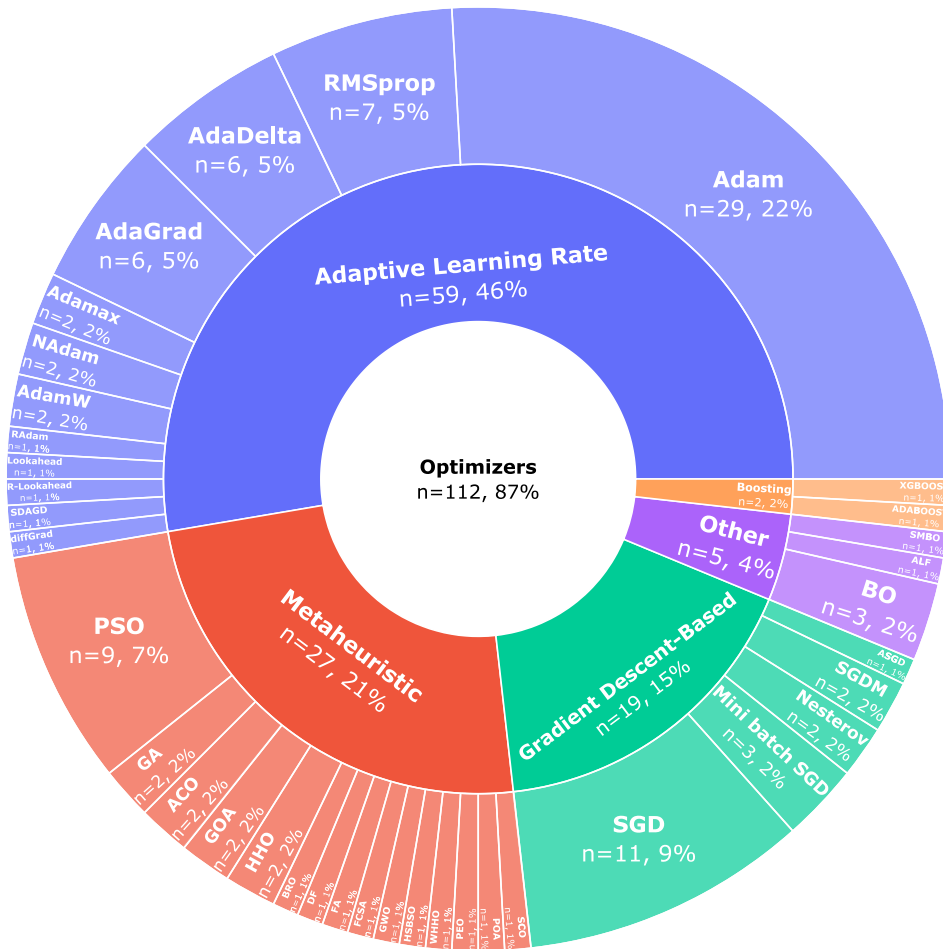


Fig. 13. Weights optimization techniques in LSTM.

algorithms approaches, which might include lesser-known optimization techniques. Finally, additional optimization algorithms that are not directly classified within the preceding four primary categories were utilized in about 4% of the included studies.

Overall, if the *Adam* algorithm were kept aside, this SLR data underscores the prominence of metaheuristic algorithms in the domain of LSTM weights optimization. At least 15 metaheuristic algorithms along with their variants used in numerous LSTM applications have been identified. This reflects the current trend of optimization techniques applied in recently proposed applications. Also, it is worth mentioning that, one or more optimization techniques can be employed in the literature experiments and that is why the count of these techniques in Fig. 13 is greater than the 82 included studies in this SLR.

#### 5.4.1. Adaptive learning rate algorithms

Adaptive learning rate algorithms are commonly used in ML and DL applications. These algorithms aim to overcome some of the limitations of traditional fixed learning rate approaches, such as SGD algorithms, by adaptively adjusting the learning rate during the training process (Chia et al., 2021). Among the numerous techniques analyzed (Fig. 13), the *Adam* algorithm was the most frequently utilized algorithm in the included literature with (N=29), followed by *RMSprop* (N=7), *AdaGrad* (N=6), *AdaDelta* (N=6). *AdamW*, *NAdam*, *Adamax*, *RAdam*, *Lookahead*, *R-Lookahead*, *SDAGD*, and *diffGrad* were also observed, but in smaller occurrences.

Table 18 shows a comparison between the most used adaptive learning rate weights optimization algorithms in LSTM, according to their main characteristics, advantages, disadvantages, and referenced papers from the included studies.

#### 5.4.2. Gradient descent-based algorithms

Gradient-based algorithms are iterative techniques as they rely on gradient information from the objective function during each iteration. While these methods are effective for finding optimal solutions, they are susceptible to getting stuck in poor local minima. Additionally, gradient-based algorithms often face challenges in achieving high accuracy while suffering from slow training speeds (Chia et al., 2021). Among the numerous techniques analyzed (Fig. 13), *Stochastic Gradient Descent* (SGD) was the most prevalent algorithm in this category (N=11), followed by its variant, *Mini-batch SGD* (N=3). Next, *Nesterov Accelerated Gradient* (NAG), and *Gradient Descent with Momentum* (SGDM) were utilized in 2 studies. Finally, *Asynchronous SGD* was used in only 1 study.

Table 19 shows a comparison between the most used gradient descent-based weights optimization algorithms in LSTM, according to their main characteristics, advantages, disadvantages, and referenced papers from the included studies.

#### 5.4.3. Metaheuristic algorithms

Metaheuristic algorithms have special advantages over traditional optimization methods, such as their flexibility, simplicity, and gradient-free nature (Dhiman and Kumar, 2019; Ragab et al., 2020). These algorithms do not involve computing gradients but focus on evaluating the objective function for feasible solutions, thereby reducing computational complexity significantly. Furthermore, their adaptable nature allows easy customization for diverse problems by modifying the objective function, making them widely applicable across various fields (Bahreininejad, 2019; Sumiea et al., 2023).

Table 18

Comparison between adaptive learning rate algorithms used in LSTM models.

| Algorithm        | Proposed by                  | Characteristics                                                                                                   | Advantages                                                                                                                                                                                          | Disadvantages                                                                                                                                                                                                        | Research papers                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------|------------------------------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Adam</i>      | Kingma and Ba (2014)         | ❖ Combines the strength of RMSprop and the SGD with momentum (Dash and Thampy, 2023).                             | ❖ Relatively stable gradient descent procedures in most cases (Tan et al., 2022).<br>❖ Useful in non-convex optimization problems.                                                                  | ❖ May not converge in some cases.<br>❖ Easy to fall into the local optimum (Chen et al., 2022).                                                                                                                      | Dash and Thampy (2023), Huang et al. (2022b), Chen et al. (2022), Shi et al. (2021), Bao et al. (2021), Wu et al. (2020), Ratnaparkhi et al. (2021), Lui et al. (2022), Xu et al. (2021), Fazil et al. (2023), Huang et al. (2022a), Kim et al. (2022), Tan et al. (2022), An et al. (2022), Tortora et al. (2020), Rehman et al. (2022), Koç and Türkoğlu (2022), Dai et al. (2020), Gao et al. (2020), Mei et al. (2022), Wei et al. (2019), Gallardo-Antolín and Montero (2021), Chen et al. (2021, 2023), Masum et al. (2020), Bhunia et al. (2019), Fu et al. (2022), Eom and Kim (2023), Zhang et al. (2019b) |
| <i>AdaGrad</i>   | Duchi et al. (2011)          | ❖ The learning rate is dynamically adjusted based on the sum of squares of all past gradients.                    | ❖ Well-suited for addressing sparse gradient problems.<br>❖ During the initial training phase, as the cumulative gradient remains smaller, the learning rate increases, leading to faster learning. | ❖ Prolonged training increases the cumulative gradient, reducing the learning rate and hindering parameter updates.<br>❖ Manual set of global learning rate is necessary.<br>❖ Not suitable for non-convex problems. | Shuang et al. (2019), Yang et al. (2019), Fazil et al. (2023), Hwang et al. (2021), Wei et al. (2019), Zhang et al. (2019b)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <i>RMSprop</i>   | Tieleman and Hinton (2012)   | ❖ An extension of AdaGrad that uses a moving average of the squared gradient to normalize the learning rate.      | ❖ Addresses the late stages learning inefficiency issue in AdaGrad.<br>❖ Effective for optimizing non-convex problems.                                                                              | ❖ During the later phases of training, the update steps might repeatedly occur around the local minimum.                                                                                                             | Ratnaparkhi et al. (2021), Zhang et al. (2021b), Wang et al. (2019), Dey et al. (2022), Sun et al. (2020b), Chen et al. (2022), Zhang et al. (2019b)                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <i>AdaDelta</i>  | Zeiler (2012)                | ❖ A modification of AdaGrad that adapts the learning rate based on a moving window of the past gradients updates. | ❖ Addresses the late stages learning inefficiency issue in AdaGrad.<br>❖ Effective for optimizing non-convex problems.<br>❖ Uses less memory since it stores only a window of past gradients.       | ❖ During the later phases of training, the update steps might repeatedly occur around the local minimum.                                                                                                             | Ratnaparkhi et al. (2021), Fazil et al. (2023), Bi et al. (2021), Rehman et al. (2022), Torres-Tello et al. (2020), Zhang et al. (2019b)                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <i>AdamW</i>     | Loshchilov and Hutter (2018) | ❖ An improvement to Adam optimizer that decouples weight decay from the gradients update.                         | ❖ Better training loss and more generalized performance than Adam.                                                                                                                                  | ❖ Hyperparameters Sensitivity.                                                                                                                                                                                       | Asim et al. (2022), Dong et al. (2022)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <i>Adamax</i>    | Kingma and Ba (2014)         | ❖ Variant of Adam with infinity norm (max).                                                                       | ❖ Uses the maximum value of the past gradients rather than their sum of squares.                                                                                                                    | ❖ May not converge in some cases.                                                                                                                                                                                    | Lin et al. (2020), Tanhadoust et al. (2023)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <i>RAdam</i>     | Liu et al. (2019a)           | ❖ Introduced a new variant to Adam optimizer to rectify the variance of the adaptive learning rate.               | ❖ More stable especially during the initial training.<br>❖ Accelerated convergence.<br>❖ Can dynamically toggle the adaptive momentum on and off.                                                   | ❖ Needs to manually adjust the number of preheating processes.                                                                                                                                                       | Chen et al. (2022)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <i>NAdam</i>     | Dozat (2016)                 | ❖ Combines NAG and Adam algorithms.                                                                               | ❖ Generally used for noisy and unstable gradients. Ratnaparkhi et al. (2021)                                                                                                                        | ❖ Extra computational cost compared to Adam optimizer.                                                                                                                                                               | Ratnaparkhi et al. (2021), Wang et al. (2018)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>Lookahead</i> | Zhang et al. (2019a)         | ❖ Maintains two sets of weights (fast and slow) to achieve faster convergence.                                    | ❖ Enables the fast set of weights to explore ahead while leaving the slower set of weights behind, leading to improved long-term stability.                                                         | ❖ Computationally expensive.                                                                                                                                                                                         | Huang et al. (2022b)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

(continued on next page)

Table 18 (continued).

| Algorithm          | Proposed by          | Characteristics                                                                                                | Advantages                                                                                                    | Disadvantages                                                                                     | Research papers    |
|--------------------|----------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------|
| <i>R-Lookahead</i> | Zhang et al. (2019a) | ❖ Same as Lookahead optimizer, but it uses RAdam algorithm to optimize the fast weights part of the algorithm. | ❖ Prevents the occurrence of violent oscillations due to the limited number of samples during early training. | ❖ Computationally expensive.                                                                      | Chen et al. (2022) |
| <i>diffGrad</i>    | Dubey et al. (2020)  | ❖ An optimizer based on the difference between the present and the immediate past gradient.                    | ❖ outperforms other SGD optimizers.<br>❖ performs uniformly well for training CNN.                            | ❖ Needs to manually adjust the diffGrad friction coefficient (DFC).                               | Mei et al. (2022)  |
| <i>SDAGDm</i>      | Tan and Lim (2022)   | ❖ Combines the power of the Hessian-based method and adaptive learning rate methods.                           | ❖ Faster and better convergence as compared to SGD and Adam.                                                  | ❖ High memory consumption and computationally expensive.<br><br>❖ Fluctuation issues in momentum. | Chia et al. (2021) |

Table 19

Comparison between gradient descent-based algorithms used in LSTM models.

| Algorithm               | Proposed by              | Characteristics                                                                                                                | Advantages                                                                                                     | Disadvantages                                                                                                                                                                                    | Research papers                                                                                                                                                                                                                                          |
|-------------------------|--------------------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>SGD</i>              | Robbins and Monro (1951) | ❖ Updating the gradient per iteration using a single random sample, rather than computing the precise gradient value directly. | ❖ Reduce the computational complexity and accelerate convergence.<br>❖ Memory efficient.                       | ❖ The gradient oscillates due to added noise from random selection, and the search process becomes blind in the solution space.<br><br>❖ May fall into local minima.                             | Mohine et al. (2022), Wang et al. (2023), Jyotishi and Dandapat (2022), Zhang et al. (2020), Xue et al. (2019), Liu et al. (2019b), Rehman et al. (2022), Dai et al. (2020), Wei et al. (2019), Gallardo-Antolín and Montero (2021), Cheng et al. (2021) |
| <i>NAG</i>              | Nesterov (1983)          | ❖ Accumulates the previous gradient as momentum to enhance the current gradient descent process during the update step.        | ❖ Has further improvements over the traditional SDG.<br>❖ Momentum helps to escape the local optima solutions. | ❖ Slightly more complex than standard SGD.<br>❖ Not easy to choose an appropriate learning rate.                                                                                                 | Zhang et al. (2019b), Dai et al. (2020)                                                                                                                                                                                                                  |
| <i>SGDM</i>             | Qian (1999)              | ❖ Uses a moving average of past gradients to update the current gradients.                                                     | ❖ Enhances convergence speed when handling high curvature, small yet consistent gradients, or noisy gradients. | ❖ If the current gradient is opposite to the previous update, the old value will have a deceleration effect on the search.                                                                       | Ratnaparkhi et al. (2021), Dong et al. (2020)                                                                                                                                                                                                            |
| <i>Mini-Batch SGD</i>   | Hinton et al. (2010)     | ❖ Updating the parameters using randomly sampled mini-batches.                                                                 | ❖ Reduces the variance of the gradients and makes the convergence more stable and faster than SGD.             | ❖ Deterministic gradient in batch gradient descent may lead to a local minimum in multi-modal problems.<br>❖ An improper batch size (too small or too large) directly affects model convergence. | Lui et al. (2022), Tan et al. (2022), Passricha and Aggarwal (2020), Ma et al. (2019)                                                                                                                                                                    |
| <i>Asynchronous SGD</i> | Dean et al. (2012)       | ❖ Used to parallelize computing on multi-GPU.                                                                                  | ❖ Works well on fully distributed datasets and large mini-batches.                                             | ❖ Smaller mini-batches can weaken the performance.                                                                                                                                               | Passricha and Aggarwal (2020)                                                                                                                                                                                                                            |

Fig. 13 reveals interesting synthesized findings of this study regarding the metaheuristic algorithms used in the included literature. The original *Particle Swarm Optimization (PSO)* and some of its variants (the improved PSO, the adaptive hybrid mutation PSO, the adaptive PSO, and the multi-objective PSO) emerged as the most prevalent algorithms in the metaheuristic category for LSTM weights optimization, appearing in 9 instances across the 82 included papers. Next, the *Genetic Algorithm (GA)*, *HHO (Harris Hawks Optimization)*, *Grey Wolf Optimizer (GWO)*, and *Ant Colony Optimization (ACO)* were each utilized twice, showcasing their potentials in the metaheuristic domain. Lastly, several metaheuristic algorithms were found to be employed once in the included papers.

Table 20 shows a comparison between the most used Metaheuristic weights optimization algorithms in LSTM, according to their main characteristics, advantages, disadvantages, and referenced papers from the included studies.

#### 5.4.4. Boosting algorithms

Boosting algorithms, a family of ensemble learning techniques, utilize an iterative approach for generating a strong classifier (Zhang and Ma, 2012). The boosting algorithms were the least utilized optimization algorithms in the included studies. Only two boosting algorithms have been identified, namely *Adaptive Boosting (AdaBoost)* and *Extreme Gradient Boosting (XGBoost)*, and each one was used only once in the included papers.

Table 21 shows a comparison between the most used Boosting weights optimization algorithms in LSTM, according to their main characteristics, advantages, disadvantages, and referenced papers from the included studies.

#### 5.4.5. Other optimization algorithms

This section encompasses additional optimization algorithms that are not directly classified within the preceding primary categories

Table 20

Comparison between metaheuristic algorithms used in LSTM models.

| Algorithm                                 | Proposed by                 | Characteristics                                                                                                                                                                                                                                               | Advantages                                                                                                                                                                                                                                                                              | Disadvantages                                                                                                                                                                                                               | Research papers                                                                                                                                                                    |
|-------------------------------------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Particle Swarm Optimization</i>        | Kennedy and Eberhart (1995) | ❖ Each particle has a unique position, velocity, and objective, aiming to achieve the best objective value and global position (Shami et al., 2022).                                                                                                          | ❖ Simple to implement and code.<br>❖ Has only three controlling parameters (inertia weight, cognitive ratio, and social ratio).<br>❖ Flexible to hybridize with other optimization algorithms.                                                                                          | ❖ Becoming trapped in local optima.<br>❖ Lack of Diversity.<br>❖ Sensitive to parameters change.                                                                                                                            | Bian et al. (2021), Dash and Thampy (2023), Yang et al. (2021), Liu et al. (2021), Ji et al. (2021), Shao et al. (2019), Wang et al. (2020), Kumar et al. (2022), Ma et al. (2021) |
| <i>Genetic Algorithm</i>                  | Grefenstette (1993)         | ❖ Uses natural evolution to select the best or near-optimal solution from randomly initialized options.                                                                                                                                                       | ❖ Tackles any optimization challenge that can be expressed through chromosomal representation.                                                                                                                                                                                          | ❖ Parameters like mutation, reproduction, and crossover must be properly tuned.<br>❖ The discovery of a global optimum is not always guaranteed.                                                                            | Singh et al. (2022), Yang et al. (2021)                                                                                                                                            |
| <i>Ant Colony Optimization</i>            | Dorigo et al. (2006)        | ❖ Uses a population of ants to search the solution space and find good solutions efficiently.                                                                                                                                                                 | ❖ Possesses the strength to address intricate, non-uniform, and non-linear challenges effectively.<br>❖ Rapidly reach convergence.                                                                                                                                                      | ❖ The computation is influenced when dealing with explicit or implicit stochastic problems.<br>❖ Requires adjustment of its parameters, such as evaporation, relative pheromone trail, and heuristic information.           | Hwang et al. (2021), ElSaid et al. (2018)                                                                                                                                          |
| <i>Grasshopper Optimization Algorithm</i> | Saremi et al. (2017)        | ❖ Mimics the behavior of grasshopper swarms in nature to search for an optimal global solution to a problem (Qin et al., 2021).                                                                                                                               | ❖ Efficient in solving global unconstrained and constrained optimization problems (Luo et al., 2018).<br>❖ Can be integrated with local search techniques for improved performance (El-Shorbagy and Ayoub, 2021).                                                                       | ❖ Slow convergence speed (Alirezapour et al., 2024).<br>❖ Prone to falling into local optima (Alirezapour et al., 2024).<br>❖ May have some drawbacks compared to other optimization algorithms (Saremi et al., 2017).      | Shi et al. (2020), Liu and Sheng (2022)                                                                                                                                            |
| <i>Harris Hawks Optimization</i>          | Heidari et al. (2019)       | ❖ Mimics the chasing styles of Harris' hawks in nature called "surprise pounce".<br>❖ Has several active and time-varying phases of exploration and exploitation.                                                                                             | ❖ Can be used for non-convex function optimization and design optimization problems (Kang et al., 2023).<br>❖ Can conduct a smooth transition between the exploratory and exploitive patterns.                                                                                          | ❖ Can easily fall into a local optimum (Kang et al., 2023) (Qu et al., 2020).<br>❖ Slow convergence speed and low optimization (Liang et al., 2022).                                                                        | Sangeetha and Kumaran (2023), Liang et al. (2022)                                                                                                                                  |
| <i>Battle Royale Optimization</i>         | Rahkar Farshi (2021)        | ❖ Inspired by the Battle Royale game genre, where players compete to be the last one standing. ❖ Used in continuous problem spaces (Rahkar Farshi, 2021). ❖ Binary algorithm that uses stochastic methods to find an optimal solution (Akan et al., 2022).    | ❖ Relatively new algorithm that has shown promising results in solving optimization problems (Akan et al., 2022).<br>❖ Suitable for problems involving binary variables (Akan et al., 2022).<br>❖ Appropriate for complex evaluations or extensive search spaces (Rahkar Farshi, 2021). | ❖ Has not been extensively tested and validated (Akan et al., 2022). ❖ May not be suitable for problems with continuous variables (Akan et al., 2022). ❖ Multiple runs might be needed for convergence (Akan et al., 2022). | Sarvari and Sridevi (2023)                                                                                                                                                         |
| <i>Dragonfly Algorithm</i>                | Mirjalili (2016a)           | ❖ Consists of two main phases: exploitation and exploration.<br>❖ Uses a dynamic swarm in the exploitation phase and a static swarm in the exploration phase.<br>❖ Utilizes group intelligence for global search and local development (Rahman et al., 2019). | ❖ Effective in optimizing different real-world problems (Rahman et al., 2019).<br>❖ Provides a balance between exploration and exploitation in metaheuristic optimization.                                                                                                              | ❖ Some limitations and challenges in understanding and applying the algorithm (Alshinwan et al., 2021)                                                                                                                      | Liang et al. (2021)                                                                                                                                                                |

(continued on next page)



Table 20 (continued).

| Algorithm                                            | Proposed by                                                                   | Characteristics                                                                                                                                                                                                                                                                                                                                                                           | Advantages                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Disadvantages                                                                                                                                                                                                                                                                                                                                                                                                                    | Research papers                          |
|------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <i>Firefly Algorithm</i>                             | <a href="#">Yang (2010)</a>                                                   | <ul style="list-style-type: none"> <li>❖ Inspired by the flashing behavior of fireflies and their attraction to each other.</li> <li>❖ Uses randomization to search for a set of solutions.</li> </ul>                                                                                                                                                                                    | <ul style="list-style-type: none"> <li>❖ Efficient and easy to implement.</li> <li>❖ Suitable for parallel implementation.</li> <li>❖ Simultaneously and effectively discovering both global and local solutions.</li> <li>❖ Allows numerous fireflies to operate independently and in parallel.</li> </ul>                                                                                                                                                                                       | <ul style="list-style-type: none"> <li>❖ Tuning of the attractiveness, randomization parameter, and the absorption coefficient is necessary.</li> </ul>                                                                                                                                                                                                                                                                          | <a href="#">Liang et al. (2021)</a>      |
| <i>Fuzzy Crow Search</i>                             | <a href="#">Askarzadeh (2016)</a>                                             | <ul style="list-style-type: none"> <li>❖ Uses fuzzy logic to balance exploration and exploitation.</li> <li>❖ Dynamically adjusts control parameters based on the search state.</li> <li>❖ Scalable for optimization problems of varying sizes and complexities.</li> </ul>                                                                                                               | <ul style="list-style-type: none"> <li>❖ Simple implementation.</li> <li>❖ Flexible with few parameters.</li> <li>❖ Handles various input types, including vague data.</li> <li>❖ Solves complex problems and mimics human decision-making.</li> <li>❖ Manages engineering uncertainties.</li> </ul>                                                                                                                                                                                              | <ul style="list-style-type: none"> <li>❖ Not always accurate; may include noise or faults.</li> <li>❖ Needs improvements, such as better Fuzzy Decision Trees.</li> </ul>                                                                                                                                                                                                                                                        | <a href="#">Ravi (2020)</a>              |
| <i>Grey Wolf Optimization</i>                        | <a href="#">Mirjalili et al. (2014)</a>                                       | <ul style="list-style-type: none"> <li>❖ Population-based algorithm that has a leader-follower hierarchy.</li> <li>❖ Integration of a two-phase mutation mechanism.</li> </ul>                                                                                                                                                                                                            | <ul style="list-style-type: none"> <li>❖ Replicates grey wolves' hunting to break complex problems into subsets for optimal solutions.</li> <li>❖ Comparing diverse solutions accelerates iteration, leading to quicker convergence.</li> </ul>                                                                                                                                                                                                                                                   | <ul style="list-style-type: none"> <li>❖ Bad local searching ability.</li> <li>❖ Provides near-optimal solutions (not always the best solution).</li> </ul>                                                                                                                                                                                                                                                                      | <a href="#">Altan et al. (2021)</a>      |
| <i>Pigeon Optimization Algorithm</i>                 | <a href="#">Goel (2014)</a>                                                   | <ul style="list-style-type: none"> <li>❖ Population-based algorithm using candidate solutions to iteratively improve (<a href="#">Duan and Qiu, 2019</a>; <a href="#">Pan et al., 2021</a>).</li> <li>❖ Aims to find the optimal solution for a given problem (<a href="#">Pan et al., 2021</a>).</li> </ul>                                                                              | <ul style="list-style-type: none"> <li>❖ Simple and easy to implement with minimal parameter tuning (<a href="#">Zhun et al., 2021</a>).</li> <li>❖ Explores a large search space to find the global optimum (<a href="#">Duan and Qiu, 2019</a>; <a href="#">Zhun et al., 2021</a>).</li> <li>❖ Suitable for both continuous and discrete optimization problems (<a href="#">Zhun et al., 2021</a>).</li> </ul>                                                                                  | <ul style="list-style-type: none"> <li>❖ Suffers from premature convergence (<a href="#">Duan and Qiu, 2019</a>).</li> <li>❖ May require many iterations to reach the optimal solution.</li> <li>❖ Not suitable for large-scale problems due to its population-based nature (<a href="#">Zhun et al., 2021</a>).</li> </ul>                                                                                                      | <a href="#">Karthic and Kumar (2022)</a> |
| <i>Hybrid Squirrel Butterfly Search Optimization</i> | <a href="#">Jain et al. (2019)</a> and <a href="#">Arora and Singh (2019)</a> | <ul style="list-style-type: none"> <li>❖ Combines the exploration ability of squirrel search and butterfly search.</li> <li>❖ Adjusts the population size based on the progress of the optimization process.</li> </ul>                                                                                                                                                                   | <ul style="list-style-type: none"> <li>❖ Improved optimization techniques in various optimization problems, such as continuous optimization, and discrete optimization.</li> <li>❖ Fast convergence rate compared to other optimization techniques.</li> <li>❖ Adapt to different optimization problems and dynamic environments.</li> </ul>                                                                                                                                                      | <ul style="list-style-type: none"> <li>❖ Can be computationally expensive, especially for large-scale optimization problems.</li> <li>❖ Has several parameters that need to be tuned, which can be time-consuming and require expert knowledge.</li> <li>❖ Complex and may not be suitable for very large optimization problems.</li> </ul>                                                                                      | <a href="#">Elashiri et al. (2022)</a>   |
| <i>Sine Cosine Optimization</i>                      | <a href="#">Mirjalili (2016b)</a>                                             | <ul style="list-style-type: none"> <li>❖ Position updates of search agents are controlled by sine and cosine functions. <a href="#">Gabis et al. (2021)</a>.</li> <li>❖ Population-based algorithm (<a href="#">Bansal et al., 2023</a>).</li> <li>❖ It is a meta-heuristic algorithm that is used to solve complex optimization problems (<a href="#">Zhou et al., 2021</a>).</li> </ul> | <ul style="list-style-type: none"> <li>❖ More efficient and cost-effective than other methods for solving complex problems (<a href="#">Zhou et al., 2021</a>).</li> <li>❖ Good balance between exploration and exploitation, which allows it to find the global optimum without getting stuck in local optima (<a href="#">Gabis et al., 2021</a>).</li> <li>❖ Used effectively in feature selection, image processing, and robot path planning. <a href="#">Gabis et al. (2021)</a>.</li> </ul> | <ul style="list-style-type: none"> <li>❖ May suffer from premature convergence (near-optimal solutions) (<a href="#">Gabis et al., 2021</a>).</li> <li>❖ Numerous function evaluations are required to converge to the optimal solution (<a href="#">Zhou et al., 2021</a>).</li> <li>❖ Sensitive to the choice of parameters, such as the population size and the mutation rate (<a href="#">Zhou et al., 2021</a>).</li> </ul> | <a href="#">Somu et al. (2020)</a>       |

(continued on next page)

Table 20 (continued).

| Algorithm                               | Proposed by                 | Characteristics                                                                                                                                                                                                                                                                     | Advantages                                                                                                                                                                                                                                                                                                                                                                | Disadvantages                                                                                                                                                                                                                                                                | Research papers                |
|-----------------------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| <i>Population Extremal Optimization</i> | Boettcher and Percus (2002) | <ul style="list-style-type: none"> <li>❖ Inspired by self-organized criticality and extremal dynamics.</li> <li>❖ Guided by a single mutation operation.</li> <li>❖ Can handle multi-objective optimization problems.</li> </ul>                                                    | <ul style="list-style-type: none"> <li>❖ Ability to exploit the search space effectively.</li> <li>❖ Suitable for handling many-objective optimization problems.</li> <li>❖ Can find optimal solutions in complex and large search spaces.</li> </ul>                                                                                                                     | <ul style="list-style-type: none"> <li>❖ Insufficient ability to exploit the search space compared to other algorithms.</li> <li>❖ Require tuning of parameters for optimal performance.</li> <li>❖ Slower convergence compared to other optimization algorithms.</li> </ul> | Zhao et al. (2020)             |
| <i>Wild Horse Herd Optimization</i>     | Naruei and Keynia (2022)    | <ul style="list-style-type: none"> <li>❖ Inspired by the social life behavior of wild horses (Naruei and Keynia, 2022).</li> <li>❖ The Wild Horse Optimizer involves five steps: initializing groups, mating, leadership, leader exchange, and saving the best solution.</li> </ul> | <ul style="list-style-type: none"> <li>❖ Capable of solving complex problems in which the solution space failed to depict.</li> <li>❖ A population-based algorithm, so local optimal avoidance is inherently very high (Naruei and Keynia, 2022).</li> <li>❖ Achieves population diversity by computing random movements for each horse across all dimensions.</li> </ul> | Recommended to evaluate the proposed algorithm further as it is a newly proposed algorithm.                                                                                                                                                                                  | Nandhini and Tamilpavai (2022) |

Table 21

Comparison between boosting algorithms used in LSTM models.

| Algorithm       | Proposed by                | Characteristics                                                            | Advantages                                        | Disadvantages                                                                                  | Research papers    |
|-----------------|----------------------------|----------------------------------------------------------------------------|---------------------------------------------------|------------------------------------------------------------------------------------------------|--------------------|
| <i>AdaBoost</i> | Freund and Schapire (1996) | Utilized to integrate multiple-weak classifiers to form a more strong one. | Easier to use than SVM. Not prone to overfitting. | Extremely sensitive to noisy data and outliers. Slower than XGBoost.                           | Ying et al. (2021) |
| <i>XGBoost</i>  | Chen and Guestrin (2016)   | Parallelized and optimized version of the gradient boosting algorithm.     | Performs very well on structured datasets.        | Not suitable for sparse and unstructured data. Extremely sensitive to noisy data and outliers. | Dai et al. (2022)  |

of optimization methods. The *Bayesian Optimization* is utilized in 3 studies, while *Adaptive Levy Flight*, and *Sequential Model-based Global Optimization* were each used in only one study.

Table 22 shows a comparison between other weights optimization algorithms in LSTM, according to their main characteristics, advantages, disadvantages, and referenced papers from the included studies.

##### 5.5. RQ5: What is the intensity of publications related to LSTM?

The objective of this question is to investigate the prevalence of publications and current research trends concerning LSTM weight initialization and optimization. This analysis aims to offer valuable insights and support researchers in identifying potential future directions. The answer to this question encompasses four key distributions of the included studies, categorized by the year of publication (5.5.1), publication type (5.5.2), publisher (5.5.3), and journal (5.5.4).

###### 5.5.1. Distribution of included studies by year

To start with the yearly publication's trend. There has been a consistent upward trend in the number of studies related to LSTM weight initialization and optimization over the past six years (2018–2023). However, in 2023, fewer studies were identified and included compared to 2022 due to ongoing publications. However, the number of studies is expected to rise and surpass the trend line by year-end, indicating a current upward trend in this area.

This trend is evident in Fig. 14(A) which displays the overall trend in the number of identified studies, whereas Fig. 14(B) displays the publication trend of the included studies within the domain area.

###### 5.5.2. Distribution of included studies by publication type

Next and as stated earlier, this SLR focused on including studies with well-defined objectives, clear methodology, stated limitations, and clear well-presented findings. Based on the final 82 included studies in

Table 5, high-quality studies relevant to LSTM weights were published in journals (77 studies, 94%), and then only (5 studies, 6%) were published as conference proceedings. Fig. 15 depicts the distribution of the included studies by publication type, while the previously mentioned Fig. 7 depicts the distribution of the included studies by the scientific databases and venues.

###### 5.5.3. Distribution of included studies by publisher

Fig. 16 illustrates the distribution of the included publications by the academic publisher. It highlights the intensity of each publisher's contribution. Among the publishers, *IEEE* had the highest intensity with 37 papers, followed by *Elsevier* with 25 papers, *Hindawi* with 7 papers, *Springer* with 3 papers, and then *Tech Science Press*, and *Wiley* with 2 papers each. Finally, *ACM*, *BioMed Central*, *De Gruyter*, *IGI Global*, *IOP Publishing Ltd*, and the *University of Bahrain* each published 1 paper. These publisher-specific data indicate the distribution and intensity of the selected 82 papers, showcasing the diverse contributions from those publishers in the LSTM publications.

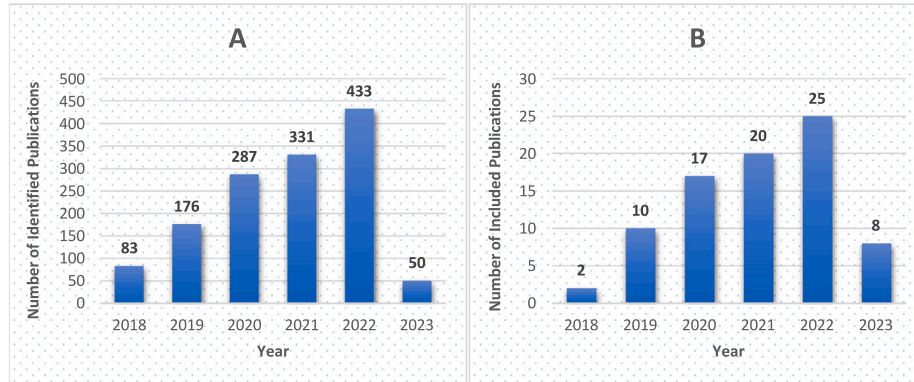
###### 5.5.4. Distribution of included studies by journal

Fig. 17 depicts the intensity of those publishers' journals with more than one publication. This SLR found that the *IEEE's* journals, being the prominent publisher, had a significant impact with 15 papers in *IEEE Access*, 6 papers in *IEEE Transactions On Instrumentation And Measurement*, 3 papers in *IEEE Sensors Journal*, and lastly 2 papers in *IEEE Transactions On Intelligent Transportation Systems*. Then for the *Elsevier's* journals, there are 3 papers in *Biomedical Signal Processing And Control* and *Neurocomputing*, followed by 2 papers in each of the following journals: *Applied Energy*, *Applied Soft Computing*, *Energy*, and *Expert Systems With Applications*. Finally, for the *Hindawi's* journals, there are 2 papers in *Computational Intelligence And Neuroscience*.

**Table 22**

Comparison between other optimization algorithms used in LSTM models.

| Algorithm                                         | Proposed by                          | Characteristics                                                                                                                                                                                                                                                                                                                                                                   | Advantages                                                                                                                                                                                                                                                                                                                                                                                                                                            | Disadvantages                                                                                                                                                                                                                                                                                       | Research papers                                                                                                     |
|---------------------------------------------------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <i>Bayesian Optimization</i>                      | <a href="#">Moćkus (1975)</a>        | <ul style="list-style-type: none"> <li>❖ Excels in low dimensions (typically <math>\leq 10</math>) and for computationally expensive objectives.</li> <li>❖ While not highly accurate, it provides global solutions without requiring multiple initial points.</li> </ul>                                                                                                         | <ul style="list-style-type: none"> <li>❖ Effective for optimizing complex, noisy, and/or expensive objective functions.</li> <li>❖ Efficient, as it only evaluates the points that are most likely to be informative.</li> <li>❖ Robust, as it is not sensitive to the initial conditions.</li> </ul>                                                                                                                                                 | <ul style="list-style-type: none"> <li>❖ Can be computationally expensive, especially for high-dimensional problems.</li> <li>❖ Can be difficult to tune the hyperparameters of the algorithm.</li> <li>❖ Not guaranteed to find the global optimum, especially for non-convex problems.</li> </ul> | <a href="#">Onan and Toçoğlu (2021)</a> , <a href="#">Huang et al. (2022a)</a> , <a href="#">Zhou et al. (2020)</a> |
| <i>Adaptive Levy Flight</i>                       | <a href="#">Lévy (1954)</a>          | <ul style="list-style-type: none"> <li>❖ A random search algorithm obeying Levy distribution.</li> <li>❖ A non-Gaussian process that combines short-distance local searches with occasional long-distance jumps to enhance search space and global search.</li> </ul>                                                                                                             | <ul style="list-style-type: none"> <li>❖ Better performance than several well-known metaheuristics, such as PSO, GA, Differential Evolution, and Elephant Herding Optimization. <a href="#">Zhang et al. (2021a)</a></li> <li>❖ Increased population diversity and expanded search area.</li> </ul>                                                                                                                                                   |                                                                                                                                                                                                                                                                                                     | <a href="#">Zhang et al. (2021a)</a>                                                                                |
| <i>Sequential Model-based Global Optimization</i> | <a href="#">Hutter et al. (2011)</a> | <ul style="list-style-type: none"> <li>❖ Adapts an enhanced Bayesian optimization approach, employing the tree-structured Parzen estimator.</li> <li>❖ Focus on deterministic target algorithms (<a href="#">Hutter et al., 2011</a>; <a href="#">Li et al., 2022</a>).</li> <li>❖ Reliance on computationally expensive models (<a href="#">Hutter et al., 2011</a>).</li> </ul> | <ul style="list-style-type: none"> <li>❖ Smart hyperparameter tuning approach (<a href="#">Li et al., 2022</a>). ❖ Capable of locating global optimum despite multiple local optima.</li> <li>❖ Uses statistical models to guide the search</li> <li>❖ Adjusts search based on optimization data, aiding quicker convergence to the optimal solution.</li> <li>❖ Easily parallelized, making it suitable for solving large-scale problems.</li> </ul> | <ul style="list-style-type: none"> <li>❖ Computationally expensive models (<a href="#">Li et al., 2022</a>).</li> <li>❖ Costly initial experimental designs (<a href="#">Hutter et al., 2011</a>).</li> <li>❖ Requires accurate statistical models to guide the search process.</li> </ul>          | <a href="#">Zhou et al. (2020)</a>                                                                                  |

**Fig. 14.** Publications trend in the recent 6 years for the: (A) Number of identified studies by publication year and (B) Number of included studies by publication year.

## 6. Conclusion

This SLR is concluded based on its theoretical implications, limitations, and closing remarks in the Sections 6.1, 6.2, and 6.3 respectively.

### 6.1. Theoretical implications

Efficient weight initialization strategies and optimization methods hold paramount importance in the training of LSTM models as they improve their overall performance. The success of the LSTM models is intricately linked to the identification of appropriate weight initialization techniques and the selection of optimal optimization algorithms. Therefore, this paper emphasizes the significance of exploring these critical aspects to achieve superior results and overcome challenges encountered during LSTM model training.

First of all, this study is among the first SLRs on the current state-of-the-art weights initialization and optimization techniques in RNN-LSTM models. Although LSTM literature reviews are available, none of them have addressed this narrowed-down area appropriately. This SLR focuses on presenting extensive research on techniques used for RNN-LSTM model training, domains, applications, datasets, environments, and the evaluation metrics used for each LSTM model.

Second, through an exhaustive search, 15 relevant reviews, SLR, and survey studies published within the last six years, and 6 studies published in the past, so a total of 21 relevant reviews have been identified. The study aims to provide the most up-to-date information on recent advancements by including literature published between 2018 and 2023. This comparative analysis reveals that several studies encompass a diverse range of optimization techniques, such as adaptive learning rate methods, gradient descent-based optimization, specific LSTM optimization techniques, and weight initialization strategies.

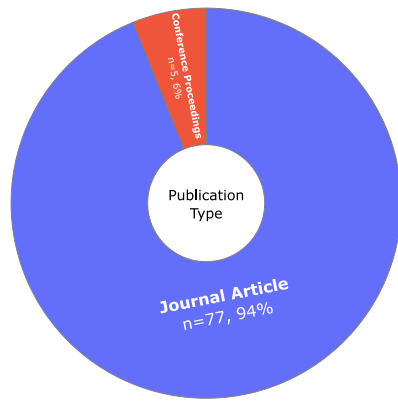


Fig. 15. Distribution of included studies by publication type.

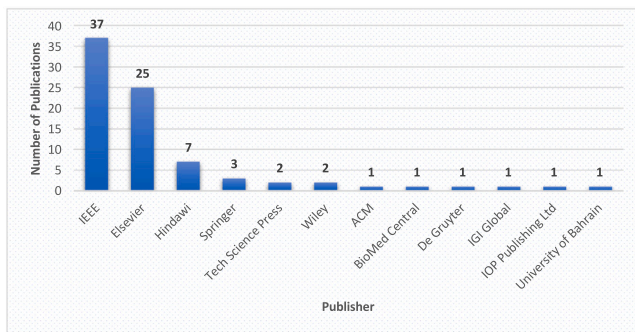


Fig. 16. Distribution of included studies by publisher.

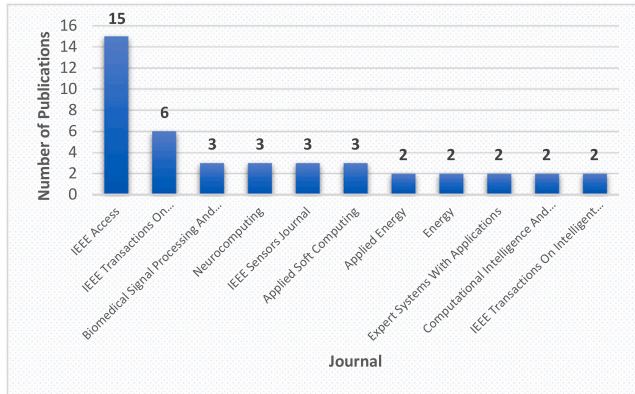


Fig. 17. Distribution of included studies by journal.

However, it is noteworthy that none of these previous studies specifically focus on metaheuristic algorithms, except for a slight discussion in one study. This identified research gap serves as the impetus for this review, as the aim is to comprehensively address all the aspects of weight initialization strategies and optimization methods, including the incorporation of metaheuristic algorithms, within the context of LSTM. By synthesizing and analyzing the most recent research findings, valuable insights into advancements, challenges, and potential future directions for optimizing LSTMs and their variations in diverse real-world applications have been provided (Section 5.1).

## 6.2. Limitations

This research focuses specifically on weight initialization and optimization techniques employed for the RNN-LSTM models. As such,

this SLR encompasses studies published between 2018 and 2013, potentially excluding algorithms proposed before 2018. However, the aim is to provide an up-to-date assessment of algorithms in this context.

Another limitation of this SLR is its exclusive focus on reviewed papers that have explored LSTM weights initialization and/or optimization techniques across 11 different domains, and the strict inclusion and exclusion criteria for search screening (Table 3), resulting in the inclusion of 82 selected papers that met the selection criteria, out of 1360 identified paper. While this approach ensures rigor and quality, it may not cover all relevant sources. It is recognized that the review could have been further enriched by benchmarking the performance of the 37 identified optimization algorithms for LSTM training on the same task.

Furthermore, the SLR includes only papers written in English and related exclusively to the areas of Computer Science and Engineering, leading to the acknowledgment that some potentially useful papers may have been excluded during the search screening process.

Nevertheless, despite these limitations, this SLR contributes valuable insights into the latest advancements in RNN-LSTM optimization, aiding researchers in the field in making informed decisions for efficient model training.

## 6.3. Closing remarks

In this SLR, 1360 studies have been collected (Table 5). Then, after a four-step study mapping process based on the PRISMA methodology to systematically analyze studies related to LSTM weights initialization and optimization, only 82 studies have been included based on the inclusion and exclusion criteria for search screening (Table 3), and the eligibility and quality assessment (Section 4.3, and Fig. 6). These studies have been reviewed and analyzed from different perspectives to answer the formulated five research questions (RQ1, RQ2, RQ3, RQ4, and RQ5).

1. **For RQ1**, after reviewing and analyzing the 82 included studies, it was observed that LSTMs were widely used in 11 domains (Fig. 8) namely healthcare and medical, energy, natural language processing, prognostics and health management, computer vision, communication and networking, stock market and financial prediction, speech recognition and synthesis, safety, environment, and manufacturing. Therefore, the studies for each corresponding domain have been analyzed, and then the specific LSTM model applications, architectures, datasets, and evaluation metrics have been identified. The answer to this question opens the doors for more future works in the neglected domains, especially more future research that needs to be done in the manufacturing, environment, and safety domains.
2. **For RQ2**, this question has been answered in two different directions (i) The preferred and most used programming languages, frameworks, and libraries (5.2.1), and (ii) The most used evaluation metrics in regression and classification problems (5.2.2). After analyzing the 82 studies, it was clear that the preferred and most used programming languages for building robust LSTM models are Python, and MATLAB, followed by Java and R. Also, 67 distinct evaluation metrics for regression and classification problems have been identified. Those metrics are widely used to evaluate the LSTM model's performance in the literature. Python is recommended to be utilized in future studies as it has specialized libraries and frameworks, especially for ML and DL applications. Also, the most frequently used evaluation metrics are recommended in future work experiments as they provide baseline comparisons with other researchers' works.



Table 23  
PRISMA 2020 checklist.

| Topic             | Item  | Location                          |
|-------------------|-------|-----------------------------------|
| Title             | 1     | Title                             |
| Abstract          | 2     | Abstract                          |
| Introduction      | 3     | Section 1                         |
|                   | 4     | Section 4.1.2                     |
|                   | 5     | Section 4.3                       |
|                   | 6     | Section 4.1.3, Table 5            |
|                   | 7     | Sections 4.1.3, 4.2.1             |
| Methods           | 8     | Section 4.1.3                     |
|                   | 9     | Section 4.4                       |
|                   | 10–15 | Not applicable                    |
|                   | 16a   | Flowchart 5                       |
| Results           | 16b   | Not applicable                    |
|                   | 17    | Section 5.1                       |
|                   | 18–22 | Not applicable                    |
| Other Information | 24    | Not applicable                    |
|                   | 25    | Acknowledgment                    |
|                   | 26    | Declaration of competing interest |
|                   | 27    | Data availability                 |

Table 24  
Abbreviations.

| Abbreviation | Definition                                                      |
|--------------|-----------------------------------------------------------------|
| ACO          | Ant Colony Optimization                                         |
| AdaDelta     | Adaptive Delta Algorithm                                        |
| AdaGrad      | Adaptive Gradient Algorithm                                     |
| ADAM         | Adaptive Moment Estimation                                      |
| ADAMAX       | Adaptive Moment Estimation with Maximum                         |
| ADAMW        | Adaptive Moment Estimation with Weight Decay                    |
| AEC-LSTM     | Attention-Emotion-Enhanced LSTM                                 |
| AELA         | Attention-Enabled and Location-Aware                            |
| AGA          | Adaptive Genetic Algorithm                                      |
| AHMPPO       | Adaptive Hybrid Mutation Particle Swarm Optimization            |
| ALF          | Adaptive Levy Flight                                            |
| AM           | Attention Mechanism                                             |
| APSO         | Adaptive Particle Swarm Optimization                            |
| ASGD         | Asynchronous Stochastic Gradient Descent                        |
| BMA          | Bayesian Model Averaging                                        |
| BO           | Bayesian Optimization                                           |
| BRO          | Battle Royal Optimization                                       |
| CapsNet      | Capsule Neural Networks                                         |
| CE           | Copula entropy                                                  |
| C-MAPSS      | Commercial Modular Aero-Propulsion System Simulation            |
| CRF          | Conditional Random Field                                        |
| CWC          | Coverage Width-Based Criterion                                  |
| DF           | Dragonfly Algorithm                                             |
| DL           | Deep Learning                                                   |
| DLSTMs       | Double LSTMs                                                    |
| EBRSA        | Enhanced Battle Royale Self-Attention                           |
| EEMD         | Ensemble Empirical Mode Decomposition                           |
| FA           | Firefly Algorithm                                               |
| FCSA         | Fuzzy Crow Search Algorithm                                     |
| GA           | Genetic Algorithm                                               |
| GOA          | Grasshopper Optimization Algorithm                              |
| GPR          | Gaussian Process Regression                                     |
| GWO          | Grey Wolf Optimization                                          |
| HHO          | Harris Hawks Optimization                                       |
| HSBSO        | Hybrid Squirrel Butterfly Search Optimization                   |
| IDRSN        | Improved sub-channel threshold Depth Residual Shrinkage Network |
| IDW          | Inverse Distance Weighting                                      |
| IPSO         | Improved Particle Swarm Optimization                            |
| LSTM         | Long-short Term Memory                                          |
| ML           | Machine Learning                                                |
| MBSGD        | Mini-Batch Stochastic Gradient Descent                          |
| MOGA-ELSTM   | Multiobjective Genetic Algorithm and Evolving LSTM              |

(continued on next page)

Table 24 (continued).

| Abbreviation | Definition                                                       |
|--------------|------------------------------------------------------------------|
| MOHHO        | Multi-Objective Harris Hawk Optimization                         |
| MOPSO        | Multi-Objective Particle Swarm Optimization                      |
| MUSA-BiLSTM  | Multi-head self-attention based BiLSTM                           |
| MWHHO        | Modified Wild Horse Herd Optimization                            |
| NAdam        | Nesterov-accelerated Adaptive Moment Estimation                  |
| NAG          | Nesterov Accelerated Gradient                                    |
| NNCT         | No Negative Constraint Theory                                    |
| PEO          | Population Extremal Optimization                                 |
| PF           | Particle Filter                                                  |
| PICP         | PI Coverage Probability                                          |
| PINAW        | PI Normalized Averaged Width                                     |
| POA          | Pigeon Optimization Algorithm                                    |
| PSO          | Particle Swarm Optimization                                      |
| RQs          | Research Questions                                               |
| RADAM        | Rectified Adam Optimizer                                         |
| ResNet       | residual network                                                 |
| RUL          | Remaining Useful Life                                            |
| SCO          | Sine Cosine Optimization                                         |
| SDAGDm       | Momentum-based Stochastic Diagonal Approximate Greatest Descent  |
| SGD          | Stochastic Gradient Descent                                      |
| SGDM         | Stochastic Gradient Descent with Momentum                        |
| SMBO         | Sequential Model-Based Global Optimization                       |
| SRLP         | Semantic Role Labeling Position                                  |
| ST-HConvLSTM | Attention-based spatial-temporal hierarchical convolutional LSTM |
| SWLSTM       | Shared Weight Long Short-Term Memory Network                     |
| TDS          | Temporal Dense Sampling BiLSTM                                   |
| THHO         | Taylor-Harris Hawks Optimization                                 |
| VAE          | Variational Auto-Encoder                                         |
| WCAE-LSTM    | Weighted convolutional autoencoder-LSTM                          |
| XGBOOST      | Extreme Gradient Boosting                                        |

3. For RQ3, the most frequently used weights initialization techniques in LSTM networks have been identified, as this step plays a crucial role in the training of DL models. The process involves initializing and repeatedly optimizing the network’s weights during training until convergence to an ideal matrix of weights and minimum loss value is achieved. Proper weight initialization is essential to avoid vanishing and exploding gradient problems and accelerate model performance. In this part, only 19 studies explicitly declared weights initialization techniques, which were categorized into five main techniques: *Random* weights initialization, *Glorot/Xavier* Initialization, *He/Kaiming* Initialization, *Orthogonal matrix* initialization and *All-zeros* initialization. Table 17 shows the comparison of these techniques based on their characteristics, advantages, disadvantages, and referenced papers. The recommendation for future studies is to stop utilizing the All-zeros initialization technique and to use an appropriate weights initialization, such as *Glorot/Xavier* or *He/Kaiming* initializers.

4. For RQ4, the most prevalent weights optimization techniques employed in LSTM models have been investigated. Upon analyzing the 82 included studies, a total of about 37 optimization algorithms with their variants have been identified. These algorithms were utilized for optimizing the hyperparameters and parameters, including the network weights of the proposed models. These algorithms were subsequently grouped into five main categories, namely *adaptive learning rate* algorithms, *gradient descent-based* algorithms, *metaheuristic* algorithms, *boosting* algorithms, and other algorithms. A comprehensive discussion of each group is presented in the Sections 5.4.1, 5.4.2, 5.4.3, 5.4.4, and 5.4.5 respectively.

PSO, in particular, appears to be the preferred choice for various optimization applications. However, the presence of a wide range of other metaheuristic algorithms suggests ongoing research and exploration in that field. The recommendation for

**Table 25**

Search string of each scientific database and venue.

| Source              | URL                                                                                       | Search string                                                                                                                                                                                                                                                                                                                         |
|---------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scopus              | <a href="https://bit.ly/lstm_slr_scopus">https://bit.ly/lstm_slr_scopus</a>               | (TITLE(*LSTM* OR "long short term memory") AND TITLE-ABS-KEY(weight)) AND PUBYEAR > 2017 AND PUBYEAR < 2024 AND (LIMIT-TO (DOCTYPE,"ar") OR LIMIT-TO (DOCTYPE,"cp")) OR LIMIT-TO (DOCTYPE,"ch")) AND (LIMIT-TO (LANGUAGE,"English")) AND (LIMIT-TO (SUBJAREA,"COMP") OR LIMIT-TO (SUBJAREA,"ENGI")) AND (LIMIT-TO (PUBSTAGE,"final")) |
| IEEE Xplore         | <a href="https://bit.ly/lstm_slr_ieeexplore">https://bit.ly/lstm_slr_ieeexplore</a>       | ("Document Title":*LSTM* OR "Document Title":"long short term memory") AND ("All Metadata":weight), Filters Applied (Type: Conferences or Journals, Years: 2018 – 2023)                                                                                                                                                               |
| Web of Science      | <a href="https://bit.ly/lstm_slr_wos">https://bit.ly/lstm_slr_wos</a>                     | *LSTM* OR "long short term memory" (Title) and weight (Topic), Refined by (Document Type: Article and Proceeding Paper, Language: English, Research Area: Computer Science or Engineering, Open Access)                                                                                                                               |
| Science Direct      | <a href="https://bit.ly/lstm_slr_sciencedirect">https://bit.ly/lstm_slr_sciencedirect</a> | Title: LSTM OR "long short term memory", Title, abstract, keywords: weight, Filters Applied (Type: Research articles and book chapters, Years: 2018 – 2023, Subject area: Computer Science and Engineering)                                                                                                                           |
| ACM Digital Library | <a href="https://bit.ly/lstm_slr_acmdl">https://bit.ly/lstm_slr_acmdl</a>                 | [[Title: *LSTM*] OR [Title: "long short term memory"]] AND [Abstract: weight] AND [E-Publication Date: (01/01/2018 TO 12/31/2023)]                                                                                                                                                                                                    |

future studies is to focus more on the nature of their experiments and needs and to choose the appropriate algorithms that will give them outstanding results.

- For RQ5, the prevalence of publications and current research trends concerning LSTM weights initialization and optimization have been examined. The investigation conducted seeks to offer valuable insights that can guide researchers in their future directions and give them a better understanding of the publications related to LSTM optimization.

## Abbreviations

Abbreviations used in this SLR are in Table 24.

## CRedit authorship contribution statement

**Safwan Mahmood Al-Selwi:** Writing – review & editing, Writing – original draft, Visualization, Methodology, Investigation, Formal analysis, Conceptualization. **Mohd Fadzil Hassan:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Conceptualization. **Said Jadid Abdulkadir:** Writing – review & editing, Resources, Conceptualization. **Amgad Muneer:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Conceptualization. **Ebrahim Hamid Sumiea:** Writing – review & editing, Formal analysis. **Alawi Alqushaibi:** Writing – review & editing, Formal analysis, Conceptualization. **Mohammed Gamal Ragab:** Writing – review & editing, Formal analysis.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

The data used to conduct this SLR is available via this link: <https://github.com/SafwanAlselwi/RNN-LSTM-SLR>.

## Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used different AI tools to improve the language and readability. After using these tools/services, the authors reviewed and edited the content as needed and took full responsibility for the content of the publication.

## Acknowledgment

Universiti Teknologi PETRONAS fully supports this research under the Yayasan Universiti Teknologi PETRONAS Fundamental Research Grant (YUTP-FRG 1/2021 (015LC0-370)).

## Appendix A

The PRISMA 2020 Checklist is sourced from <https://www.prisma-statement.org/prisma-2020-checklist> (accessed 15/05/2024). Table 23 shows the checklist used in this SLR, with section, number, and location in the paper.

## Appendix B

Various databases were used, each with a unique search scheme. To maintain consistency, two approaches were employed: creating advanced search queries using operators and wildcards, and using the advanced search form on each database's website. Table 25 shows the search strings and provides direct URLs to explore the results for each database.

## References

- Abdullah, M.H.A., Aziz, N., Abdulkadir, S.J., Alhussian, H.S.A., Talpur, N., 2023. Systematic literature review of information extraction from textual data: Recent methods, applications, trends, and challenges. *IEEE Access* <http://dx.doi.org/10.1109/ACCESS.2023.3240898>.
- Abdulrahman, M.L., Ibrahim, K.M., Gital, A.Y., Zambuk, F.U., Ja'afaru, B., Yakubu, Z.I., Ibrahim, A., 2021. A review on deep learning with focus on deep recurrent neural network for electricity forecasting in residential building. *Procedia Comput. Sci.* 193, 141–154. <http://dx.doi.org/10.1016/j.procs.2021.10.014>.
- Akan, T., Agahian, S., Dehkharghani, R., 2022. Binbro: Binary battle royale optimizer algorithm. *Expert Syst. Appl.* 195, 116599. <http://dx.doi.org/10.1016/j.eswa.2022.116599>.
- Al-Qubaydhi, N., Alenezi, A., Alanazi, T., Senyor, A., Alanezi, N., Alotaibi, B., Alotaibi, M., Razaque, A., Hariri, S., 2024. Deep learning for unmanned aerial vehicles detection: A review. *Comp. Sci. Rev.* 51, 100614. <http://dx.doi.org/10.1016/j.cosrev.2023.100614>.
- Al-Selwi, S.M., Hassan, M.F., Abdulkadir, S.J., Muneer, A., 2023. LSTM inefficiency in long-term dependencies regression problems. *J. Adv. Res. Appl. Sci. Eng. Technol.* 30, 1631. <http://dx.doi.org/10.37934/araset.30.3.1631>.
- Alhumoud, S.O., Al Wazrah, A.A., 2022. Arabic sentiment analysis using recurrent neural networks: a review. *Artif. Intell. Rev.* 55 (1), 707–748. <http://dx.doi.org/10.1007/s10462-021-09989-9>.
- Alirezapour, H., Mansouri, N., Mohammad Hasani Zade, B., 2024. A comprehensive survey on feature selection with grasshopper optimization algorithm. *Neural Process. Lett.* 56 (1), 28.
- Alshinwan, M., Abualigah, L., Shehab, M., Elaziz, M.A., Khasawneh, A.M., Alabool, H., Hamad, H.A., 2021. Dragonfly algorithm: a comprehensive survey of its results, variants, and applications. *Multimedia Tools Appl.* 80, 14979–15016. <http://dx.doi.org/10.1007/s11042-020-10255-3>.

- Altan, A., Karasu, S., Zio, E., 2021. A new hybrid model for wind speed forecasting combining long short-term memory neural network, decomposition methods and grey wolf optimizer. *Appl. Soft Comput.* 100, 106996. <http://dx.doi.org/10.1016/j.asoc.2020.106996>.
- An, Y., Xia, X., Chen, X., Wu, F.-X., Wang, J., 2022. Chinese clinical named entity recognition via multi-head self-attention based BiLSTM-CRF. *Artif. Intell. Med.* 127, 102282. <http://dx.doi.org/10.1016/j.artmed.2022.102282>.
- Arora, S., Singh, S., 2019. Butterfly optimization algorithm: a novel approach for global optimization. *Soft Comput.* 23, 715–734. <http://dx.doi.org/10.1007/s00500-018-3102-4>.
- Asim, M.N., Ibrahim, M.A., Malik, M.I., Zehe, C., Cloarec, O., Trygg, J., Dengel, A., Ahmed, S., 2022. EL-mlLocNet: An explainable LSTM network for RNA-associated multi-compartment localization prediction. *Comput. Struct. Biotechnol. J.* 20, 3986–4002. <http://dx.doi.org/10.1016/j.csbj.2022.07.031>.
- Askarzadeh, A., 2016. A novel metaheuristic method for solving constrained engineering optimization problems: Crow search algorithm. *Comput. Struct.* 169, 1–12. <http://dx.doi.org/10.1016/j.compstruc.2016.03.001>.
- Bahreininejad, A., 2019. Improving the performance of water cycle algorithm using augmented Lagrangian method. *Adv. Eng. Softw.* 132, 55–64. <http://dx.doi.org/10.1016/j.advengsoft.2019.03.008>.
- Bansal, J.C., Bajpai, P., Rawat, A., Nagar, A.K., 2023. Sine cosine algorithm. In: *Sine Cosine Algorithm for Optimization*. Springer, pp. 15–33. [http://dx.doi.org/10.1007/978-981-19-9722-8\\_3](http://dx.doi.org/10.1007/978-981-19-9722-8_3).
- Bao, T., Zaidi, S.A.R., Xie, S., Yang, P., Zhang, Z.-Q., 2021. A CNN-LSTM hybrid model for wrist kinematics estimation using surface electromyography. *IEEE Trans. Instrum. Meas.* 70, 1–9. <http://dx.doi.org/10.1109/TIM.2020.3036654>.
- Beltozar-Clemente, S., Iparraquirre-Villanueva, O., Pucuhayla-Revatta, F., Zapata-Paulini, J., Cabanillas-Carbonell, M., 2024. Predicting customer abandonment in recurrent neural networks using short-term memory. *J. Open Innov. Technol. Mark. Complex.* 100237.
- Bengio, Y., Simard, P., Frasconi, P., 1994. Learning long-term dependencies with gradient descent is difficult. *IEEE Trans. Neural Netw.* 5 (2), 157–166. <http://dx.doi.org/10.1109/72.279181>.
- Bhunia, A.K., Konwar, A., Bhunia, A.K., Bhowmick, A., Roy, P.P., Pal, U., 2019. Script identification in natural scene image and video frames using an attention based Convolutional-LSTM network. *Pattern Recognit.* 85, 172–184. <http://dx.doi.org/10.1016/j.patcog.2018.07.034>.
- Bi, M., Zhang, Q., Zuo, M., Xu, Z., Jin, Q., 2021. Bi-directional long short-term memory model with semantic positional attention for the question answering system. *ACM Trans. Asian Low-Resour. Lang. Inf. Process.* 20 (5), <http://dx.doi.org/10.1145/3439800>.
- Bian, J., Wang, L., Scherer, R., Woźniak, M., Zhang, P., Wei, W., 2021. Abnormal detection of electricity consumption of user based on particle swarm optimization and long short term memory with the attention mechanism. *IEEE Access* 9, 47252–47265. <http://dx.doi.org/10.1109/ACCESS.2021.3062675>.
- Bianchi, F.M., Maiorino, E., Kampffmeyer, M.C., Rizzi, A., Jenssen, R., 2017. An overview and comparative analysis of recurrent neural networks for short term load forecasting. <http://dx.doi.org/10.48550/arXiv.1705.04378>, arXiv preprint arXiv:1705.04378.
- Bilokon, P., Qiu, Y., 2023. Transformers versus LSTMs for electronic trading. <http://dx.doi.org/10.48550/arXiv.2309.11400>, arXiv preprint arXiv:2309.11400.
- Boettcher, S., Percus, A.G., 2002. Optimization with extremal dynamics. *Complexity* 8 (2), 57–62. <http://dx.doi.org/10.1002/cplx.10072>.
- Boulila, W., Driss, M., Alshanjiti, E., Al-Sarem, M., Saeed, F., Krichen, M., 2022. Weight initialization techniques for deep learning algorithms in remote sensing: Recent trends and future perspectives. In: *Advances on Smart and Soft Computing: Proceedings of ICACIn 2021*. Springer Singapore, pp. 477–484. [http://dx.doi.org/10.1007/978-981-16-5559-3\\_39](http://dx.doi.org/10.1007/978-981-16-5559-3_39).
- Chen, H., Du, M., Zhang, Y., Yang, C., 2022. Research on disease prediction method based on R-lookahead-LSTM. *Comput. Intell. Neurosci.* 2022, 1–13. <http://dx.doi.org/10.1155/2022/8431912>.
- Chen, T., Guestrin, C., 2016. XGBoost: A scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. KDD '16*, Association for Computing Machinery, New York, NY, USA, pp. 785–794. <http://dx.doi.org/10.1145/2939672.2939785>.
- Chen, C., Lu, N., Jiang, B., Xing, Y., Zhu, Z.H., 2021. Prediction interval estimation of aeroengine remaining useful life based on bidirectional long short-term memory network. *IEEE Trans. Instrum. Meas.* 70, 1–13. <http://dx.doi.org/10.1109/TIM.2021.3126006>.
- Chen, Y., Peng, L., Wang, Y., Zhou, Y., Li, C., 2023. Prediction of tandem cold-rolled strip flatness based on Attention-LSTM model. *J. Manuf. Process.* 91, 110–121. <http://dx.doi.org/10.1016/j.jmapro.2023.02.048>.
- Cheng, Y., Wu, J., Zhu, H., Or, S.W., Shao, X., 2021. Remaining useful life prognosis based on ensemble long short-term memory neural network. *IEEE Trans. Instrum. Meas.* 70, 1–12. <http://dx.doi.org/10.1109/TIM.2020.3031113>.
- Chia, Z.C., Hann Lim, K., Lian Tan, T.P., 2021. Two-phase switching optimization strategy in LSTM model for predictive maintenance. In: *2021 International Conference on Green Energy, Computing and Sustainable Technology. GECOST*, pp. 1–6. <http://dx.doi.org/10.1109/GECOST52368.2021.9538639>.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bougares, F., Schwenk, H., Bengio, Y., 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. <http://dx.doi.org/10.48550/arXiv.1406.1078>, CoRR abs/1406.1078 arXiv:1406.1078.
- Dai, X., Yin, H., Jha, N.K., 2020. Grow and prune compact, fast, and accurate LSTMs. *IEEE Trans. Comput.* 69 (3), 441–452. <http://dx.doi.org/10.1109/TC.2019.2954495>.
- Dai, Y., Zhou, Q., Leng, M., Yang, X., Wang, Y., 2022. Improving the Bi-LSTM model with XGBoost and attention mechanism: A combined approach for short-term power load prediction. *Appl. Soft Comput.* 130, 109632. <http://dx.doi.org/10.1016/j.asoc.2022.109632>.
- Dash, L., Thampy, A.S., 2023. Channel estimation using hybrid optimizer based recurrent neural network long short term memory for MIMO communications in 5G network. *SN Appl. Sci.* 5 (2), 60. <http://dx.doi.org/10.1007/s42452-022-05253-z>.
- Dean, J., Corrado, G., Monga, R., Chen, K., Devin, M., Mao, M., Ranzato, M., Senior, A., Tucker, P., Yang, K., Le, Q., Ng, A., 2012. Large scale distributed deep networks. In: *Proceedings of the 25th International Conference on Neural Information Processing Systems-Volume 1*. pp. 1223–1231, URL [http://proceedings.neurips.cc/paper\\_files/paper/2012/file/Gaca97005c68f1206823815f66102863-Paper.pdf](http://proceedings.neurips.cc/paper_files/paper/2012/file/Gaca97005c68f1206823815f66102863-Paper.pdf).
- Dey, S., Bhattacharya, R., Malakar, S., Schwenker, F., Sarkar, R., 2022. CovidConvLSTM: A fuzzy ensemble model for COVID-19 detection from chest X-rays. *Expert Syst. Appl.* 206, 117812. <http://dx.doi.org/10.1016/j.eswa.2022.117812>.
- Dhiman, G., Kumar, V., 2019. Seagull optimization algorithm: Theory and its applications for large-scale industrial engineering problems. *Knowl.-Based Syst.* 165, 169–196. <http://dx.doi.org/10.1016/j.knsys.2018.11.024>.
- Dong, J., Chen, Y., Guan, G., 2020. Cost index predictions for construction engineering based on LSTM neural networks. *Adv. Civ. Eng.* 2020, 6518147. <http://dx.doi.org/10.1155/2020/6518147>.
- Dong, F., Yu, J., Quan, W., Xiang, Y., Li, X., Sun, F., 2022. Short-term building cooling load prediction model based on DwdAdam-ILSTM algorithm: A case study of a commercial building. *Energy Build.* 272, 112337. <http://dx.doi.org/10.1016/j.enbuild.2022.112337>.
- Dorigo, M., Birattari, M., Stutzle, T., 2006. Ant colony optimization. *IEEE Comput. Intell. Mag.* 1 (4), 28–39. <http://dx.doi.org/10.1109/MCI.2006.329691>.
- Dozat, T., 2016. Incorporating nesterov momentum into adam. In: *Proceedings of 4th International Conference on Learning Representations (ICLR), Workshop Track*. pp. 1–4.
- Duan, H., Qiu, H., 2019. Advancements in pigeon-inspired optimization and its variants. *Sci. China Inf. Sci.* 62 (7), 70201. <http://dx.doi.org/10.1007/s11042-020-10255-3>.
- Dubey, S.R., Chakraborty, S., Roy, S.K., Mukherjee, S., Singh, S.K., Chaudhuri, B.B., 2020. diffGrad: An optimization method for convolutional neural networks. *IEEE Trans. Neural Netw. Learn. Syst.* 31 (11), 4500–4511. <http://dx.doi.org/10.1109/TNNLS.2019.2955777>.
- Duchi, J., Hazan, E., Singer, Y., 2011. Adaptive subgradient methods for online learning and stochastic optimization. *J. Mach. Learn. Res.* 12 (7).
- El-Shorbagy, M.A., Ayoub, A., 2021. Integrating grasshopper optimization algorithm with local search for solving data clustering problems. *Int. J. Comput. Intell. Syst.* 14 (1), 783–793. <http://dx.doi.org/10.2991/ijcis.d.210203.008>.
- Elashiri, M.A., Rajesh, A., Nath Pandey, S., Kumar Shukla, S., Urooj, S., Lay-Ekuakille, A., 2022. Ensemble of weighted deep concatenated features for the skin disease classification model using modified long short term memory. *Biomed. Signal Process. Control* 76, 103729. <http://dx.doi.org/10.1016/j.bspc.2022.103729>.
- Elman, J.L., 1990. Finding structure in time. *Cogn. Sci.* 14 (2), 179–211. <http://dx.doi.org/10.1207/s15516709cog1402.1>.
- ElSaid, A., El Jamiy, F., Higgins, J., Wild, B., Desell, T., 2018. Optimizing long short-term memory recurrent neural networks using ant colony optimization to predict turbine engine vibration. *Appl. Soft Comput.* 73, 969–991. <http://dx.doi.org/10.1016/j.asoc.2018.09.013>.
- Eom, J., Kim, J., 2023. Time-weighted cumulative LSTM method using log data for predicting credit card customer turnover. *IEEE Access* 11, 18245–18251. <http://dx.doi.org/10.1109/ACCESS.2023.3247446>.
- Ezen-Can, A., 2020. A comparison of LSTM and BERT for small corpus. *CoRR abs/2009.0545* arXiv:2009.05451.
- Fantini, C.O., Hadad, E., 2022. Stock price forecasting with artificial neural networks long short-term memory: A bibliometric analysis and systematic literature review. *J. Comput. Commun.* 10 (12), 29–50. <http://dx.doi.org/10.4236/jcc.2022.1012003>.
- Fazil, M., Khan, S., Albahhal, B.M., Alotaibi, R.M., Siddiqui, T., Shah, M.A., 2023. Attentional multi-channel convolution with bidirectional LSTM cell toward hate speech prediction. *IEEE Access* 11, 16801–16811. <http://dx.doi.org/10.1109/ACCESS.2023.3246388>.
- Freund, Y., Schapire, R.E., 1996. Experiments with a new boosting algorithm. In: *International Conference on Machine Learning*, Vol. 96. Citeseer, pp. 148–156, URL <https://api.semanticscholar.org/CorpusID:1836349>.
- Fu, E., Zhang, Y., Yang, F., Wang, S., 2022. Temporal self-attention-based Conv-LSTM network for multivariate time series prediction. *Neurocomputing* 501, 162–173. <http://dx.doi.org/10.1016/j.neucom.2022.06.014>.
- Gabis, A.B., Meraihi, Y., Mirjalili, S., Ramdane-Cherif, A., 2021. A comprehensive survey of sine cosine algorithm: variants and applications. *Artif. Intell. Rev.* 54 (7), 5469–5540. <http://dx.doi.org/10.1007/s10462-021-10026-y>.



- Gallardo-Antolín, A., Montero, J.M., 2021. On combining acoustic and modulation spectrograms in an attention LSTM-based system for speech intelligibility level classification. *Neurocomputing* 456, 49–60. <http://dx.doi.org/10.1016/j.neucom.2021.05.065>.
- Gallicchio, C., Scardapane, S., 2020. Deep randomized neural networks. In: *Recent Trends in Learning from Data: Tutorials from the INNS Big Data and Deep Learning Conference. INNSBDDL2019*, Springer International Publishing, Cham, pp. 43–68. [http://dx.doi.org/10.1007/978-3-030-43883-8\\_3](http://dx.doi.org/10.1007/978-3-030-43883-8_3).
- Gao, D., Peng, L., Bai, Y., 2020. HAZOP text named entity recognition using CNN-bilstm-CRF model. In: *2020 Chinese Automation Congress. CAC*, pp. 6159–6164. <http://dx.doi.org/10.1109/CAC51589.2020.9327702>.
- Glorot, X., Bengio, Y., 2010. Understanding the difficulty of training deep feedforward neural networks. In: Teh, Y.W., Titterton, M. (Eds.), *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. In: *Proceedings of Machine Learning Research*, vol. 9, PMLR, Chia Laguna Resort, Sardinia, Italy, pp. 249–256. URL <https://proceedings.mlr.press/v9/glorot10a.html>.
- Goel, S., 2014. Pigeon optimization algorithm: A novel approach for solving optimization problems. In: *2014 International Conference on Data Mining and Intelligent Computing. ICDMIC, IEEE*, pp. 1–5. <http://dx.doi.org/10.1109/ICDMIC.2014.6954259>.
- Grefenstette, J.J., 1993. Genetic algorithms and machine learning. In: *Proceedings of the Sixth Annual Conference on Computational Learning Theory*, pp. 3–4.
- Greff, K., Srivastava, R.K., Koutnik, J., Steunebrink, B.R., Schmidhuber, J., 2017. LSTM: A search space odyssey. *IEEE Trans. Neural Netw. Learn. Syst.* 28 (10), 2222–2232. <http://dx.doi.org/10.1109/TNNLS.2016.2582924>.
- Haddaway, N.R., Page, M.J., Pritchard, C.C., McGuinness, L.A., 2022. PRISMA2020: An r package and shiny app for producing PRISMA 2020-compliant flow diagrams, with interactivity for optimised digital transparency and open synthesis. *Campbell Syst. Rev.* 18 (2), e1230. <http://dx.doi.org/10.1002/cl2.1230>.
- He, K., Zhang, X., Ren, S., Sun, J., 2015. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In: *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1026–1034. [arXiv:1502.01852](http://arxiv.org/abs/1502.01852).
- Heidari, A.A., Mirjalili, S., Faris, H., Aljarah, I., Mafarja, M., Chen, H., 2019. Harris hawks optimization: Algorithm and applications. *Future Gener. Comput. Syst.* 97, 849–872. <http://dx.doi.org/10.1016/j.future.2019.02.028>.
- Hinton, G., Srivastava, N., Swersky, K., 2010. Neural networks for machine learning lecture 6a overview of mini-batch gradient descent. In: *Lecture Notes Distributed in CSC321 of University of Toronto*, vol. 14, (8), p. 2.
- Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. *Neural Comput.* 9 (8), 1735–1780. <http://dx.doi.org/10.1162/neco.1997.9.8.1735>.
- Huang, F., Li, X., Yuan, C., Zhang, S., Zhang, J., Qiao, S., 2022a. Attention-emotion-enhanced convolutional LSTM for sentiment analysis. *IEEE Trans. Neural Netw. Learn. Syst.* 33 (9), 4332–4345. <http://dx.doi.org/10.1109/TNNLS.2021.3056664>.
- Huang, Q., Zheng, Y., Xu, Y., 2022b. Microgrid load forecasting based on improved long short-term memory network. *J. Electr. Comput. Eng.* 2022, 4017708. <http://dx.doi.org/10.1155/2022/4017708>.
- Hutter, F., Hoos, H.H., Leyton-Brown, K., 2011. Sequential model-based optimization for general algorithm configuration. In: *Learning and Intelligent Optimization: 5th International Conference, LION 5, Rome, Italy, January 17–21, 2011. Selected Papers 5*. Springer, pp. 507–523. [http://dx.doi.org/10.1007/978-3-642-25566-3\\_40](http://dx.doi.org/10.1007/978-3-642-25566-3_40).
- Hwang, W.-H., Kang, D.-H., Kim, D.-H., 2021. Brain lateralisation feature extraction and ant colony optimisation-bidirectional Lstm network model for emotion recognition. *IET Signal Process.* 16, 45–61. <http://dx.doi.org/10.1049/sil2.12076>.
- Jain, M., Singh, V., Rani, A., 2019. A novel nature-inspired algorithm for optimization: Squirrel search algorithm. *Swarm Evol. Comput.* 44, 148–175. <http://dx.doi.org/10.1016/j.swevo.2018.02.013>.
- Jaydip, S., Sidra, M., 2022. Long-and-short-term memory (LSTM) networks architectures and applications in stock price prediction. In: *Emerging Computing Paradigms: Principles, Advances and Applications*. Wiley, pp. 143–160. <http://dx.doi.org/10.1002/9781119813439.ch8>.
- Ji, Y., Liew, A.W.-C., Yang, L., 2021. A novel improved particle swarm optimization with long-short term memory hybrid model for stock indices forecast. *IEEE Access* 9, 23660–23671. <http://dx.doi.org/10.1109/ACCESS.2021.3056713>.
- Jyotishi, D., Dandapat, S., 2022. An ECG biometric system using hierarchical LSTM with attention mechanism. *IEEE Sens. J.* 22 (6), 6052–6061. <http://dx.doi.org/10.1109/JSEN.2021.3139135>.
- Kang, H., Liu, R., Yao, Y., Yu, F., 2023. Improved harris hawks optimization for non-convex function optimization and design optimization problems. *Math. Comput. Simulation* 204, 619–639. <http://dx.doi.org/10.1016/j.matcom.2022.09.010>.
- Karthic, S., Kumar, S.M., 2022. Wireless intrusion detection based on optimized LSTM with stacked auto encoder network. *Intell. Autom. Soft Comput.* 34 (1), 439–453. <http://dx.doi.org/10.32604/iasc.2022.025153>.
- Kennedy, J., Eberhart, R., 1995. Particle swarm optimization. In: *Proceedings of ICNN'95 - International Conference on Neural Networks*, Vol. 4, pp. 1942–1948. <http://dx.doi.org/10.1109/ICNN.1995.488968>.
- Kim, Y.K., Lee, M., Song, H.S., Lee, S.-W., 2022. Automatic cardiac arrhythmia classification using residual network combined with long short-term memory. *IEEE Trans. Instrum. Meas.* 71, 1–17. <http://dx.doi.org/10.1109/TIM.2022.3181276>.
- Kingma, D.P., Ba, J., 2014. Adam: A method for stochastic optimization. <http://dx.doi.org/10.48550/arXiv.1412.6980>, arXiv preprint [arXiv:1412.6980](http://arxiv.org/abs/1412.6980).
- Kitchenham, B., Charters, S., 2007. Guidelines for performing systematic literature reviews in software engineering. Report, Keele University and Durham University Joint Report, URL [https://www.elsevier.com/\\_data/promis\\_misc/525444systematicreviewsguide.pdf](https://www.elsevier.com/_data/promis_misc/525444systematicreviewsguide.pdf).
- Koç, E., Türkoglu, M., 2022. Forecasting of medical equipment demand and outbreak spreading based on deep long short-term memory network: the COVID-19 pandemic in Turkey. *Signal Image Video Process.* 16 (3), 613–621. <http://dx.doi.org/10.1007/s11760-020-01847-5>.
- Kumar, G., Singh, U.P., Jain, S., 2022. An adaptive particle swarm optimization-based hybrid long short-term memory model for stock price time series forecasting. *Soft Comput.* 26 (22), 12115–12135. <http://dx.doi.org/10.1007/s00500-022-07451-8>.
- Lalapura, V.S., Amudha, J., Satheesh, H.S., 2021. Recurrent neural networks for edge intelligence: A survey. *ACM Comput. Surv.* 54 (4), <http://dx.doi.org/10.1145/3448974>.
- Lauriola, I., Lavelli, A., Aiolfi, F., 2022. An introduction to deep learning in natural language processing: Models, techniques, and tools. *Neurocomputing* 470, 443–456. <http://dx.doi.org/10.1016/j.neucom.2021.05.103>.
- Lévy, P., 1954. Théorie de l'addition des variables aléatoires. URL <https://cir.nii.ac.jp/crid/1130000795096849792>.
- Li, Y., Cao, J., Xu, Y., Zhu, L., Dong, Z.Y., 2024. Deep learning based on Transformer architecture for power system short-term voltage stability assessment with class imbalance. *Renew. Sustain. Energy Rev.* 189, 113913. <http://dx.doi.org/10.1016/j.rser.2023.113913>.
- Li, B., Ma, J.Y., Hu, K., Xu, S.H., Jiao, H., Chen, J.M., Liu, W., et al., 2022. A method for parameter identification of distribution network equipment based on sequential model-based optimization. *Int. Trans. Electr. Energy Syst.* 2022, <http://dx.doi.org/10.1155/2022/9880284>.
- Liang, T., Chai, C., Sun, H., Tan, J., 2022. Wind speed prediction based on multi-variable Capsnet-BILSTM-MOHHO for WPCCC. *Energy* 250, 123761. <http://dx.doi.org/10.1016/j.energy.2022.123761>.
- Liang, T., Zhao, Q., Lv, Q., Sun, H., 2021. A novel wind speed prediction strategy based on Bi-LSTM, MOOFADA and transfer learning for centralized control centers. *Energy* 230, 120904. <http://dx.doi.org/10.1016/j.energy.2021.120904>.
- Lin, Y.-S., Gau, S.S.-F., Lee, C.-C., 2020. A multimodal interlocutor-modulated attentional BLSTM for classifying autism subgroups during clinical interviews. *IEEE J. Sel. Top. Sign. Proces.* 14 (2), 299–311. <http://dx.doi.org/10.1109/JSTSP.2020.2970578>.
- Lipton, Z.C., Berkowitz, J., Elkan, C., 2015. A critical review of recurrent neural networks for sequence learning. <http://dx.doi.org/10.48550/arXiv.1506.00019>, arXiv preprint [arXiv:1506.00019](http://arxiv.org/abs/1506.00019).
- Liu, L., Jiang, H., He, P., Chen, W., Liu, X., Gao, J., Han, J., 2019a. On the variance of the adaptive learning rate and beyond. In: *Proceedings of the Eighth International Conference on Learning Representations. ICLR*, p. 8. <http://dx.doi.org/10.48550/arXiv.1908.03265>.
- Liu, Z., Sheng, Z., 2022. URLLC occasional large time delay prediction based on unbalanced regression and LSTM. *Phys. Commun.* 54, 101785. <http://dx.doi.org/10.1016/j.phycom.2022.101785>.
- Liu, X., Shi, Q., Liu, Z., Yuan, J., 2021. Using LSTM neural network based on improved PSO and attention mechanism for predicting the effluent COD in a wastewater treatment plant. *IEEE Access* 9, 146082–146096. <http://dx.doi.org/10.1109/ACCESS.2021.3123225>.
- Liu, Y., Zhao, G., Peng, X., 2019b. Deep learning prognostics for lithium-ion battery based on ensemble long short-term memory networks. *IEEE Access* 7, 155130–155142. <http://dx.doi.org/10.1109/ACCESS.2019.2937798>.
- Loshchilov, I., Hutter, F., 2018. Fixing weight decay regularization in Adam. URL <https://openreview.net/forum?id=rk6qdGgCZ>.
- Lui, C.F., Liu, Y., Xie, M., 2022. A supervised bidirectional long short-term memory network for data-driven dynamic soft sensor modeling. *IEEE Trans. Instrum. Meas.* 71, 1–13. <http://dx.doi.org/10.1109/TIM.2022.3152856>.
- Lukosevicius, M., Jaeger, H., 2009. Reservoir computing approaches to recurrent neural network training. *Comp. Sci. Rev.* 3 (3), 127–149. <http://dx.doi.org/10.1016/j.cosrev.2009.03.005>.
- Lu, J., Chen, H., Xu, Y., Huang, H., Zhao, X., et al., 2018. An improved grasshopper optimization algorithm with application to financial stress prediction. *Appl. Math. Model.* 64, 654–668. <http://dx.doi.org/10.1016/j.apm.2018.07.044>.
- Ma, J., Ding, Y., Gan, V.J.L., Lin, C., Wan, Z., 2019. Spatiotemporal prediction of PM2.5 concentrations at different time granularities using IDW-BLSTM. *IEEE Access* 7, 107897–107907. <http://dx.doi.org/10.1109/ACCESS.2019.2932445>.
- Ma, X., Hu, H., Shang, Y., 2021. A new method for transformer fault prediction based on multifactor enhancement and refined long short-term memory. *IEEE Trans. Instrum. Meas.* 70, 1–11. <http://dx.doi.org/10.1109/TIM.2021.3098383>.
- Masum, M., Shahriar, H., Haddad, H.M., Alam, M.S., 2020. R-LSTM: Time series forecasting for COVID-19 confirmed cases with lstm-based framework. In: *2020 IEEE International Conference on Big Data. Big Data*, pp. 1374–1379. <http://dx.doi.org/10.1109/BigData50022.2020.9378276>.
- Mei, S., Li, X., Liu, X., Cai, H., Du, Q., 2022. Hyperspectral image classification using attention-based bidirectional long short-term memory network. *IEEE Trans. Geosci. Remote Sens.* 60, 1–12. <http://dx.doi.org/10.1109/TGRS.2021.3102034>.



- Mirjalili, S., 2016a. Dragonfly algorithm: a new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems. *Neural Comput. Appl.* 27 (4), 1053–1073. <http://dx.doi.org/10.1007/s00521-015-1920-1>.
- Mirjalili, S., 2016b. SCA: A Sine cosine algorithm for solving optimization problems. *Knowl.-Based Syst.* 96, 120–133. <http://dx.doi.org/10.1016/j.knsys.2015.12.022>.
- Mirjalili, S., Lewis, A., 2016. The whale optimization algorithm. *Adv. Eng. Softw.* 95, 51–67. <http://dx.doi.org/10.1016/j.advengsoft.2016.01.008>.
- Mirjalili, S., Mirjalili, S.M., Lewis, A., 2014. Grey wolf optimizer. *Adv. Eng. Softw.* 69, 46–61. <http://dx.doi.org/10.1016/j.advengsoft.2013.12.007>.
- Moćkus, J., 1975. On Bayesian methods for seeking the extremum. In: *Optimization Techniques IFIP Technical Conference: Novosibirsk, July 1–7, 1974*. Springer, pp. 400–404. [http://dx.doi.org/10.1007/978-3-662-38527-2\\_55](http://dx.doi.org/10.1007/978-3-662-38527-2_55).
- Mohine, S., Bansod, B.S., Bhalla, R., Basra, A., 2022. Acoustic modality based hybrid deep 1D CNN-BiLSTM algorithm for moving vehicle classification. *IEEE Trans. Intell. Transp. Syst.* 23 (9), 16206–16216. <http://dx.doi.org/10.1109/TITS.2022.3148783>.
- Muneer, A., Ali, R.F., Almaghthawi, A., Taib, S.M., Alghamdi, A., Ghaleb, E.A.A., 2022. Short term residential load forecasting using lstm recurrent neural network. *Int. J. Electric. Comput. Eng. (IJECE)* 12 (5), <http://dx.doi.org/10.11591/ijece.v12i5.pp5589-5599>.
- Nandhini, K., Tamilpavai, G., 2022. Hybrid CNN-LSTM and modified wild horse herd model-based prediction of genome sequences for genetic disorders. *Biomed. Signal Process. Control* 78, 103840. <http://dx.doi.org/10.1016/j.bspc.2022.103840>.
- Narkhede, M.V., Bartakke, P.P., Sutaone, M.S., 2022. A review on weight initialization strategies for neural networks. *Artif. Intell. Rev.* 55 (1), 291–322. <http://dx.doi.org/10.1007/s10462-021-10033-z>.
- Naruei, I., Keynia, F., 2022. Wild horse optimizer: A new meta-heuristic algorithm for solving engineering optimization problems. *Eng. Comput.* 38 (Suppl 4), 3025–3056. <http://dx.doi.org/10.1007/s00366-021-01438-z>.
- Nesterov, Y., 1983. A method for unconstrained convex minimization problem with the rate of convergence  $O(1/k^2)$ . *Dokl. Akad. Nauk. SSSR* 269 (3), 543, URL <https://cir.nii.ac.jp/crid/1370576118744597902>.
- Onan, A., Toçoğlu, M.A., 2021. A term weighted neural language model and stacked bidirectional LSTM based framework for sarcasm identification. *IEEE Access* 9, 7701–7722. <http://dx.doi.org/10.1109/ACCESS.2021.3049734>.
- Page, M.J., McKenzie, J.E., Bossuyt, P.M., Boutron, I., Hoffmann, T.C., Mulrow, C.D., Shamseer, L., Tetzlaff, J.M., Akl, E.A., Brennan, S.E., Chou, R., Glanville, J., Grimshaw, J.M., Hróbjartsson, A., Lalu, M.M., Li, T., Loder, E.W., Mayo-Wilson, E., McDonald, S., McGuinness, L.A., Stewart, L.A., Thomas, J., Tricco, A.C., Welch, V.A., Whiting, P., Moher, D., 2021. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *Int. J. Surg.* 88, 105906. <http://dx.doi.org/10.1016/j.ijsu.2021.105906>.
- Pan, J.-S., Tian, A.-Q., Chu, S.-C., Li, J.-B., 2021. Improved binary pigeon-inspired optimization and its application for feature selection. *Appl. Intell.* 51 (12), 8661–8679. <http://dx.doi.org/10.1007/s10489-021-02302-9>.
- Passricha, V., Aggarwal, R.K., 2020. A hybrid of deep CNN and bidirectional LSTM for automatic speech recognition. *J. Intell. Syst.* 29 (1), 1261–1274. <http://dx.doi.org/10.1515/jisys-2018-0372>.
- Petrozelli, A., Troiano, L., Serra, A., Jordanov, I., Storti, G., Tagliaferri, R., La Rocca, M., 2022. Deep learning for volatility forecasting in asset management. *Soft Comput.* 26 (17), 8553–8574. <http://dx.doi.org/10.1007/s00500-022-07161-1>.
- Prokhorov, D., Feldkarnp, L., Tyukin, I., 2002. Adaptive behavior with fixed weights in RNN: an overview. In: *Proceedings of the 2002 International Joint Conference on Neural Networks. IJCNN'02 (Cat. No.02CH37290)*. 3, pp. 2018–2022 vol.3. <http://dx.doi.org/10.1109/IJCNN.2002.1007449>.
- Qian, N., 1999. On the momentum term in gradient descent learning algorithms. *Neural Netw.* 12 (1), 145–151. [http://dx.doi.org/10.1016/S0893-6080\(98\)00116-6](http://dx.doi.org/10.1016/S0893-6080(98)00116-6).
- Qin, P., Hu, H., Yang, Z., 2021. The improved grasshopper optimization algorithm and its applications. *Sci. Rep.* 11 (1), 23733.
- Qu, C., He, W., Peng, X., Peng, X., 2020. Harris hawks optimization with information exchange. *Appl. Math. Model.* 84, 52–75. <http://dx.doi.org/10.1016/j.apm.2020.03.024>.
- Ragab, M.G., Abdulkadir, S.J., Aziz, N., Al-Tashi, Q., Alyousifi, Y., Alhussian, H., Alqushaibi, A., 2020. A novel one-dimensional cnn with exponential adaptive gradients for air pollution index prediction. *Sustainability* 12 (23), 10090. <http://dx.doi.org/10.3390/su122310090>.
- Ragab, M.G., Abdulkadir, S.J., Muneer, A., Alqushaibi, A., Sumiea, E.H., Qureshi, R., Al-Selwi, S.M., Alhussian, H., 2024. A comprehensive systematic review of YOLO for medical object detection (2018 to 2023). *IEEE Access* 12, 57815–57836. <http://dx.doi.org/10.1109/ACCESS.2024.3386826>.
- Rahkar Farshi, T., 2021. Battle royale optimization algorithm. *Neural Comput. Appl.* 33 (4), 1139–1157. <http://dx.doi.org/10.1007/s00521-020-05004-4>.
- Rahman, C.M., Rashid, T.A., et al., 2019. Dragonfly algorithm and its applications in applied science survey. *Comput. Intell. Neurosci.* 2019, <http://dx.doi.org/10.1155/2019/2923617>.
- Rashid, T., Fattah, P., Awla, D., 2018. Using accuracy measure for improving the training of LSTM with metaheuristic algorithms. *Procedia Comput. Sci.* 140, 324–333. <http://dx.doi.org/10.1016/j.procs.2018.10.307>.
- Ratnaparkhi, A., Deshpande, P., Ghule, G., 2021. A framework for segmentation and classification of arrhythmia using novel bidirectional LSTM network. *Int. J. Comput. Dig. Syst.* 10, <http://dx.doi.org/10.12785/ijcds/100178>.
- Ravi, C., 2020. Fuzzy crow search algorithm-based deep LSTM for bitcoin prediction. *Int. J. Distributed Syst. Technol. (IJ DST)* 11 (4), 53–71. <http://dx.doi.org/10.4018/IJ DST.2020100104>.
- Rehman, M.U., Ahmed, F., Khan, M.A., Tariq, U., Alfouzan, F.A., Alzahrani, N.M., Ahmad, J., 2022. Dynamic hand gesture recognition using 3D-CNN and LSTM networks. *Comput. Mater. Continua* 70 (3), 4675–4690. <http://dx.doi.org/10.32604/cmc.2022.019586>.
- Reza, S., Ferreira, M.C., Machado, J., Tavares, J.M.R., 2022. A multi-head attention-based transformer model for traffic flow forecasting with a comparative analysis to recurrent neural networks. *Expert Syst. Appl.* 202, 117275. <http://dx.doi.org/10.1016/j.eswa.2022.117275>, URL <https://www.sciencedirect.com/science/article/pii/S0957417422006443>.
- Robbins, H., Monro, S., 1951. A stochastic approximation method. *Ann. Math. Stat.* 400–407, URL <https://www.jstor.org/stable/2236626>.
- Rumelhart, D.E., Hinton, G.E., Williams, R.J., 1986. Learning representations by back-propagating errors. *Nature* 323 (6088), 533–536. <http://dx.doi.org/10.1038/323533a0>.
- Salehinejad, H., Sankar, S., Barfett, J., Colak, E., Valaee, S., 2017. Recent advances in recurrent neural networks. <http://dx.doi.org/10.48550/arXiv.1801.01078>, arXiv preprint arXiv:1801.01078.
- Sangeetha, J., Kumaran, U., 2023. A hybrid optimization algorithm using BiLSTM structure for sentiment analysis. *Measur. Sens.* 25, 100619. <http://dx.doi.org/10.1016/j.measen.2022.100619>.
- Saremi, S., Mirjalili, S., Lewis, A., 2017. Grasshopper optimisation algorithm: theory and application. *Adv. Eng. Softw.* 105, 30–47. <http://dx.doi.org/10.1016/j.advengsoft.2017.01.004>.
- Sarvari, A., Sridevi, K., 2023. An optimized EBRSA-Bi LSTM model for highly undersampled rapid CT image reconstruction. *Biomed. Signal Process. Control* 83, 104637. <http://dx.doi.org/10.1016/j.bspc.2023.104637>.
- Sezer, O.B., Gudelek, M.U., Ozbayoglu, A.M., 2020. Financial time series forecasting with deep learning : A systematic literature review: 2005–2019. *Appl. Soft Comput.* 90, 106181. <http://dx.doi.org/10.1016/j.asoc.2020.106181>.
- Shami, T.M., El-Saleh, A.A., Alswaiti, M., Al-Tashi, Q., Summakieh, M.A., Mirjalili, S., 2022. Particle swarm optimization: A comprehensive survey. *IEEE Access* 10, 10031–10061. <http://dx.doi.org/10.1109/ACCESS.2022.3142859>.
- Shao, B., Li, M., Zhao, Y., Bian, G., 2019. Nickel price forecast based on the LSTM neural network optimized by the improved PSO algorithm. *Math. Probl. Eng.* 2019, 1934796. <http://dx.doi.org/10.1155/2019/1934796>.
- Shi, Y., Li, Y., Fan, J., Wang, T., Yin, T., 2020. A novel network architecture of decision-making for self-driving vehicles based on long short-term memory and grasshopper optimization algorithm. *IEEE Access* 8, 155429–155440. <http://dx.doi.org/10.1109/ACCESS.2020.3019048>.
- Shi, Z., Xu, M., Pan, Q., 2021. 4-D flight trajectory prediction with constrained LSTM network. *IEEE Trans. Intell. Transp. Syst.* 22 (11), 7242–7255. <http://dx.doi.org/10.1109/TITS.2020.3004807>.
- Shuang, K., Ren, X., Yang, Q., Li, R., Loo, J., 2019. AELA-DLSTMs: Attention-enabled and location-aware double LSTMs for aspect-level sentiment classification. *Neurocomputing* 334, 25–34. <http://dx.doi.org/10.1016/j.neucom.2018.11.084>.
- Singh, A., Dargar, S.K., Gupta, A., Kumar, A., Srivastava, A.K., Srivastava, M., Kumar Tiwari, P., Ullah, M.A., 2022. Evolving long short-term memory network-based text classification. *Comput. Intell. Neurosci.* 2022, 4725639. <http://dx.doi.org/10.1155/2022/4725639>.
- Somu, N., Gauthama Raman, M.R., Ramamritham, K., 2020. A hybrid model for building energy consumption forecasting using long short term memory networks. *Appl. Energy* 261, 114131. <http://dx.doi.org/10.1016/j.apenergy.2019.114131>.
- Sumiea, E.H., Abdulkadir, S.J., Alhussian, H.S., Al-Selwi, S.M., Alqushaibi, A., Ragab, M.G., Fati, S.M., 2024. Deep deterministic policy gradient algorithm: A systematic review. *Heliyon* 10 (9), e30697. <http://dx.doi.org/10.1016/j.heliyon.2024.e30697>.
- Sumiea, E.H.H., Abdulkadir, S.J., Ragab, M.G., Al-Selwi, S.M., Fati, S.M., AlQushaibi, A., Alhussian, H., 2023. Enhanced deep deterministic policy gradient algorithm using grey wolf optimizer for continuous control tasks. *IEEE Access* 11, 139771–139784. <http://dx.doi.org/10.1109/ACCESS.2023.3341507>.
- Sun, S., Cao, Z., Zhu, H., Zhao, J., 2020a. A survey of optimization methods from a machine learning perspective. *IEEE Trans. Cybern.* 50 (8), 3668–3681. <http://dx.doi.org/10.1109/TCYB.2019.2950779>.
- Sun, P., Liu, P., Li, Q., Liu, C., Lu, X., Hao, R., Chen, J., 2020b. DL-IDS: Extracting features using CNN-LSTM hybrid network for intrusion detection system. *Secur. Commun. Netw.* 2020, 8890306. <http://dx.doi.org/10.1155/2020/8890306>.
- Surakhi, O., Serhan, S., Salah, I., 2020. On the ensemble of recurrent neural network for air pollution forecasting: Issues and challenges. *Adv. Sci. Technol. Eng. Syst. J.* 5 (2), 512–526. <http://dx.doi.org/10.25046/aj050265>.
- Talpur, N., Abdulkadir, S.J., Alhussian, H., Hasan, M.H., Aziz, N., Bamhdi, A., 2023. Deep neuro-fuzzy system application trends, challenges, and future perspectives: a systematic survey. *Artif. Intell. Rev.* 56 (2), 865–913. <http://dx.doi.org/10.1007/s10462-022-10188-3>.
- Tan, H.H., Lim, K.H., 2022. Two-phase switching optimization strategy in deep neural networks. *IEEE Trans. Neural Netw. Learn. Syst.* 33 (1), 330–339. <http://dx.doi.org/10.1109/TNNLS.2020.3027750>.

- Tan, K.S., Lim, K.M., Lee, C.P., Kwek, L.C., 2022. Bidirectional long short-term memory with temporal dense sampling for human action recognition. *Expert Syst. Appl.* 210, 118484. <http://dx.doi.org/10.1016/j.eswa.2022.118484>.
- Tanhadoust, A., Yang, T., Dabbaghi, F., Chai, H., Mohseni, M., Emadi, S., Nasrollahpour, S., 2023. Predicting stress-strain behavior of normal weight and lightweight aggregate concrete exposed to high temperature using LSTM recurrent neural network. *Constr. Build. Mater.* 362, 129703. <http://dx.doi.org/10.1016/j.conbuildmat.2022.129703>.
- Thakkar, A., Chaudhari, K., 2021. A comprehensive survey on deep neural networks for stock market: The need, challenges, and future directions. *Expert Syst. Appl.* 177, 114800. <http://dx.doi.org/10.1016/J.ESWA.2021.114800>.
- Tieleman, T., Hinton, G., 2012. Rmsprop: Divide the gradient by a running average of its recent magnitude. *course: Neural networks for machine learning. COURSE: Neural Netw. Mach. Learn.* 17.
- Torres, J.F., Hadjout, D., Sebaa, A., Martínez-Álvarez, F., Troncoso, A., 2021. Deep learning for time series forecasting: a survey. *Big Data* 9 (1), 3–21. <http://dx.doi.org/10.1089/big.2020.0159>.
- Torres-Tello, J., Guaman, A.V., Ko, S.-B., 2020. Improving the detection of explosives in a MOX chemical sensors array with LSTM networks. *IEEE Sens. J.* 20 (23), 14302–14309. <http://dx.doi.org/10.1109/JSEN.2020.3007431>.
- Tortora, S., Ghidoni, S., Chisari, C., Micera, S., Artoni, F., 2020. Deep learning-based BCI for gait decoding from EEG with LSTM recurrent neural network. *J. Neural Eng.* 17 (4), 046011. <http://dx.doi.org/10.1088/1741-2552/ab9842>.
- Trujillo-Guerrero, M.F., Román-Niemes, S., Jaén-Vargas, M., Cadiz, A., Fonseca, R., Serrano-Olmedo, J.J., 2023. Accuracy comparison of CNN, LSTM, and transformer for activity recognition using IMU and visual markers. *IEEE Access* 11, 106650–106669. <http://dx.doi.org/10.1109/ACCESS.2023.3318563>.
- Van Houdt, G., Mosquera, C., Nápoles, G., 2020. A review on the long short-term memory model. *Artif. Intell. Rev.* 53, 5929–5955. <http://dx.doi.org/10.1007/s10462-020-09838-1>.
- Vanitha, S., Jayashree, R., 2023. Towards finding the impact of deep learning in educational time series datasets – A systematic literature review. *Int. J. Adv. Comput. Sci. Appl.* 14 (3), <http://dx.doi.org/10.14569/IJACSA.2023.0140392>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. In: *Advances in Neural Information Processing Systems*, vol. 30, Curran Associates, Inc., <http://dx.doi.org/10.48550/arXiv.1706.03762>.
- Wang, L., Liu, Y., Li, T., Xie, X., Chang, C., 2020. Short-term PV power prediction based on optimized VMD and LSTM. *IEEE Access* 8, 165849–165862. <http://dx.doi.org/10.1109/ACCESS.2020.3022246>.
- Wang, S., Wang, X., Wang, S., Wang, D., 2019. Bi-directional long short-term memory method based on attention mechanism and rolling update for short-term load forecasting. *Int. J. Electr. Power Energy Syst.* 109, 470–479. <http://dx.doi.org/10.1016/j.jepes.2019.02.022>.
- Wang, X., Wang, X., Zhang, S., 2023. Adverse drug reaction detection from social media based on quantum bi-LSTM with attention. *IEEE Access* 11, 16194–16202. <http://dx.doi.org/10.1109/ACCESS.2022.3151900>.
- Wang, Y.-B., You, Z.-H., Li, X., Jiang, T.-H., Cheng, L., Chen, Z.-H., 2018. Prediction of protein self-interactions using stacked long short-term memory from protein sequences information. *BMC Syst. Biol.* 12 (8), 107–115. <http://dx.doi.org/10.1186/s12918-018-0647-x>.
- Weerakody, P.B., Wong, K.W., Wang, G., Ela, W., 2021. A review of irregular time series data handling with gated recurrent neural networks. *Neurocomputing* 441, 161–178. <http://dx.doi.org/10.1016/j.neucom.2021.02.046>.
- Wei, H., Gao, M., Zhou, A., Chen, F., Qu, W., Wang, C., Lu, M., 2019. Named entity recognition from biomedical texts using a fusion attention-based BiLSTM-CRF. *IEEE Access* 7, 73627–73636. <http://dx.doi.org/10.1109/ACCESS.2019.2920734>.
- Wei, X., Zhang, L., Yang, H., Zhang, L., Yao, Y., 2021. Machine learning for pore-water pressure time-series prediction: Application of recurrent neural networks. *Geosci. Front.* 12 (1), 453–467. <http://dx.doi.org/10.1016/J.GSF.2020.04.011>.
- Werbos, P.J., 1988. Generalization of backpropagation with application to a recurrent gas market model. *Neural Netw.* 1 (4), 339–356. [http://dx.doi.org/10.1016/0893-6080\(88\)90007-X](http://dx.doi.org/10.1016/0893-6080(88)90007-X).
- Wu, P., Huang, Z., Pian, Y., Xu, L., Li, J., Chen, K., 2020. A combined deep learning method with attention-based LSTM model for short-term traffic speed forecasting. *J. Adv. Transp.* 2020, 1–15. <http://dx.doi.org/10.1155/2020/8863724>.
- Xu, Z., Guo, Y., Saleh, J.H., 2021. Advances toward the next generation fire detection: Deep LSTM variational autoencoder for improved sensitivity and reliability. *IEEE Access* 9, 30636–30653. <http://dx.doi.org/10.1109/ACCESS.2021.3060338>.
- Xue, F., Ji, H., Zhang, W., Cao, Y., 2019. Attention-based spatial-temporal hierarchical convlstm network for action recognition in videos. *IET Comput. Vis.* 13, 708–718. <http://dx.doi.org/10.1049/iet-cvi.2018.5830>.
- Yang, X.-S., 2010. Firefly algorithm, stochastic test functions and design optimisation. *Int. J. Bio-Inspired Comput.* 2 (2), 78–84. <http://dx.doi.org/10.1504/IJBIC.2010.032124>.
- Yang, B., Cao, J., Wang, N., Liu, X., 2019. Anomalous behaviors detection in moving crowds based on a weighted convolutional autoencoder-long short-term memory network. *IEEE Trans. Cogn. Dev. Syst.* 11 (4), 473–482. <http://dx.doi.org/10.1109/TCDS.2018.2866838>.
- Yang, L., Xie, K., Wen, C., He, J.-B., 2021. Speech emotion analysis of netizens based on bidirectional LSTM and PGCDN. *IEEE Access* 9, 59860–59872. <http://dx.doi.org/10.1109/ACCESS.2021.3073234>.
- Ying, R., Shou, Y., Liu, C., 2021. Prediction model of dow jones index based on LSTM-adaboost. In: *2021 International Conference on Communications, Information System and Computer Engineering. CISCE*, pp. 808–812. <http://dx.doi.org/10.1109/CISCE52179.2021.9445928>.
- Yu, Y., Si, X., Hu, C., Zhang, J., 2019. A review of recurrent neural networks: LSTM cells and network architectures. *Neural Comput.* 31 (7), 1235–1270. [http://dx.doi.org/10.1162/neco\\_a\\_01199](http://dx.doi.org/10.1162/neco_a_01199).
- Zeebaree, D., Mohsin Abdulazeez, A., Abdulrahman, L., Abas Hasan, D., Kareem, O., 2021. The prediction process based on deep recurrent neural networks: A review. *Asian J. Res. Comput. Sci.* 11, 29–45. <http://dx.doi.org/10.9734/AJRCOS/2021/v11i230259>.
- Zeiler, M.D., 2012. Adadelta: an adaptive learning rate method. <http://dx.doi.org/10.48550/arXiv.1212.5701>, arXiv preprint [arXiv:1212.5701](https://arxiv.org/abs/1212.5701).
- Zhang, Y., Chen, L., Li, Y., Zheng, X., Chen, J., Jin, J., 2021a. A hybrid approach for remaining useful life prediction of lithium-ion battery with Adaptive Levy Flight optimized Particle Filter and Long Short-Term Memory network. *J. Energy Storage* 44, 103245. <http://dx.doi.org/10.1016/j.est.2021.103245>.
- Zhang, H., Huang, H., Han, H., 2021b. Attention-based convolution skip bidirectional long short-term memory network for speech emotion recognition. *IEEE Access* 9, 5332–5342. <http://dx.doi.org/10.1109/ACCESS.2020.3047395>.
- Zhang, M., Lucas, J., Ba, J., Hinton, G.E., 2019a. Lookahead optimizer: k steps forward, 1 step back. In: *Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché Buc, F., Fox, E., Garnett, R. (Eds.), Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc., pp. 9593–9604.
- Zhang, C., Ma, Y., 2012. *Ensemble Machine Learning: Methods and Applications*. Springer, <http://dx.doi.org/10.1007/978-1-4419-9326-7>.
- Zhang, Y., Qu, C., Wang, Y., 2020. An indoor positioning method based on CSI by using features optimization mechanism with LSTM. *IEEE Sens. J.* 20 (9), 4868–4878. <http://dx.doi.org/10.1109/JSEN.2020.2965590>.
- Zhang, Z., Ye, L., Qin, H., Liu, Y., Wang, C., Yu, X., Yin, X., Li, J., 2019b. Wind speed prediction method using shared weight long short-term memory network and Gaussian process regression. *Appl. Energy* 247, 270–284. <http://dx.doi.org/10.1016/j.apenergy.2019.04.047>.
- Zhao, F., Zeng, G.-Q., Lu, K.-D., 2020. EnLSTM-WPEO: Short-term traffic flow prediction by ensemble LSTM, NNCT weight integration, and population extremal optimization. *IEEE Trans. Veh. Technol.* 69 (1), 101–113. <http://dx.doi.org/10.1109/TVT.2019.2952605>.
- Zhao, E., Zhou, N., Liu, C., Su, H., Liu, Y., Cong, J., 2024. Time-aware MADDPG with LSTM for multi-agent obstacle avoidance: a comparative study. *Complex Intell. Syst.* 1–15. <http://dx.doi.org/10.1007/s40747-024-01389-0>.
- Zhou, W., Wang, P., Heidari, A.A., Wang, M., Zhao, X., Chen, H., 2021. Multi-core sine cosine optimization: Methods and inclusive analysis. *Expert Syst. Appl.* 164, 113974. <http://dx.doi.org/10.1016/j.eswa.2020.113974>.
- Zhou, S., Zhou, L., Mao, M., Xi, X., 2020. Transfer learning for photovoltaic power forecasting with long short-term memory neural network. In: *2020 IEEE International Conference on Big Data and Smart Computing. BigComp*, pp. 125–132. <http://dx.doi.org/10.1109/BigComp48618.2020.00-87>.
- Zhun, X., Liyun, X., Xufeng, L., 2021. An improved pigeon-inspired optimization algorithm for solving dynamic facility layout problem with uncertain demand. *Procedia CIRP* 104, 1203–1208. <http://dx.doi.org/10.1016/j.procir.2021.11.202>.